

On One-Shot Signatures, Quantum vs Classical Binding, and Obfuscating Permutations

Omri Shmueli¹ and Mark Zhandry^{1,2}

¹NTT Research

²Stanford University

Abstract

One-shot signatures (OSS) were defined by Amos, Georgiou, Kiayias, and Zhandry (STOC’20). These allow for signing exactly one message, after which the signing key self-destructs, preventing a second message from ever being signed. While such an object is impossible classically, Amos et al observe that OSS may be possible using quantum signing keys by leveraging the no-cloning principle. OSS has since become an important conceptual tool with many applications in decentralized settings and for quantum cryptography with classical communication. OSS are also closely related to separations between classical-binding and collapse-binding for post-quantum hashing and commitments. Unfortunately, the only known OSS construction due to Amos et al. was only justified in a classical oracle model, and moreover their justification was ultimately found to contain a fatal bug. Thus, the existence of OSS, even in a classical idealized model, has remained open.

We give the first standard-model OSS, with provable security assuming (sub-exponential) indistinguishability obfuscation (iO) and LWE. This also gives the first standard-model separation between classical and collapse-binding post-quantum commitments/hashing, solving a decade-old open problem. Along the way, we also give the first construction with unconditional security relative to a classical oracle. To achieve our standard-model construction, we develop a notion of permutable pseudorandom permutations (permutable PRPs), and show how they are useful for translating oracle proofs involving random permutations into obfuscation-based proofs. In particular, obfuscating permutable PRPs gives a trapdoor one-way permutation that is *full-domain*, solving another decade-old-problem of constructing this object from (sub-exponential) iO and one-way functions.

Contents

1	Introduction	3
1.1	Results and Paper Organization	5
1.2	Motivation and Other Related Work	7
1.3	The Bug in the previous work of AGKZ	9
1.4	On the Lower Bound for OWPs from iO	10
1.5	Directions for Future Work	10
2	Technical Overview	11
2.1	Definitions	11
2.2	OSS Relative to a Classical Oracle (Section 4)	13
2.3	Obfuscating PRPs (Section 5)	18
2.4	OSS In the Standard Model (Section 6)	24
3	Cryptographic Tools	27
3.1	Two iO Techniques	29
3.2	Information-Theoretical Hardness of Hidden Subspace Detection	31
3.3	Cryptographic Hardness of Hidden Subspace Detection	35
4	One-Shot Signatures Relative to a Classical Oracle	40
4.1	Bloating the Dual	44
4.2	Simulating the Dual	47
4.3	Hardness of the Dual-free Case from 2-to-1 Collision-Resistance	50
5	Permutable PRPs and Applications	54
5.1	How to use OP-PRPs with Indistinguishability Obfuscation	58
5.2	ONS-Merges	62
5.3	From ONS-Merge to ONS-PRP	67
5.4	Achieving OP-PRPs for Decomposable Permutations Using Obfuscation	69
6	One-Shot Signatures in the Standard Model	70
6.1	Bloating the Dual	72
6.2	Simulating the Dual	78
6.3	Hardness of the Dual-free Case from LWE and iO	81

1 Introduction

One-shot signatures (OSS) were originally proposed by Amos, Georgiou, Kiayias, and Zhandry [AGKZ20]. Here, we have a signer who generates a verification key/signing key pair, publishes the verification key, but keeps the signing key secret. Later, using the secret signing key, the signer can sign any message of their choosing. However, doing so *provably* destroys their signing key, rendering it impossible to sign even a second message relative to the same verification key. Note that the signer has full control over generating the verification key, signing key, message, and signature, and even with all this control they can never produce a second signature.

Such a protocol is clearly impossible classically. Using quantum protocols, however, it is conjectured by [AGKZ20] that an OSS is possible. Namely, the setup procedure now produces a *quantum* signing key, and the (quantum) signing algorithm is such that it requires measuring the key, which destroys it and prevents subsequent signatures. Verification keys, messages, signatures, and even the verification algorithm itself remain classical. Such an object is not trivially impossible, since the measurement principle of quantum mechanics means that computing the signature may irreversibly destroy the signing key.

OSS yields many applications that are not otherwise known: smart contracts without any blockchain [Sat22], overcoming lower-bounds in consensus protocols, a solution to the blockchain scalability problem [CS20], quantum money with classical communication, and more. Because of these advantages, OSS has gained significant interest within the blockchain community, were a major focus of recent workshops [QSi24, NTT23], and it has been claimed that practical OSS would “completely change the endgame of blockchains” [Dra23]. See Section 1.2 for more discussion.

Unfortunately, the status of OSS has been unclear. OSS lives atop a hierarchy of notions relating to quantum money, with even milder versions being notoriously difficult to construct. [AGKZ20] observe that prior work of [ARU14, Unr16b] can be used to give OSS relative to a *quantum* oracle¹, but there was no known way to actually instantiate this oracle, even heuristically. [AGKZ20] then propose a construction relative to a *classical* oracle², which can then be heuristically instantiated using (post-quantum) indistinguishability obfuscation (iO). To date, this was the only plausible candidate construction.

To justify the plausibility of their construction, [AGKZ20] prove the security of their scheme in the classical oracle model. Unfortunately, their proof turned out to have a bug discovered by [Bar23], which appears fatal and the proof in [AGKZ20] has been retracted. See Section 1.3 for discussion. This bug does not indicate an actual attack, but has left the existence of OSS uncertain.

Post-quantum hashing and commitments. Even before quantum computers will enable powerful quantum protocols, they will pose a threat to current classical cryptography. Sometimes, even if the underlying building blocks are replaced with “post-quantum” equivalents, security vulnerabilities may remain. One notable example is the security of commitments: as observed by Unruh [Unr16b], the classical notion of binding for commitments is insufficient against quantum attacks. That is, even if the security game for commitments is “upgraded” to allow the adversary to run a quantum computer but is otherwise unchanged, this is not enough to guarantee meaningful security in a quantum world. Instead a stronger, inherently quantum, notion of *collapse*-binding is needed. When instantiating commitments from hashing, (post-quantum) collision-resistance is

¹That is, an oracle that performs general unitary transformations.

²That is, an oracle implementing a classical function but which can be queried in superposition.

likewise not enough, and instead a stronger notion of *collapsing* is needed.

One may wonder if classical-binding / collision-resistance actually *implies* collapse-binding / collapsing. Unruh argues that this is likely *not* the case, by noting that a prior work of [ARU14] gives counterexample relative to a *quantum* oracle. However, there is no known standard-model separation, or even a classical-oracle separation. It is entirely consistent with existing results that classical binding implies collapse-binding in a non-relativizing way. As evidence for this, known *positive* results [Unr16a, Zha22] have given collapse-binding commitments/ hashes from essentially all of the same post-quantum assumptions known to imply classical binding. Nevertheless, our understanding of the relationship between classical-binding and quantum collapsing-binding is far from complete.

Interestingly, [AGKZ20, DS23] shows that a separation between classical and collapse-binding actually *implies* a one-shot signature, showing that these questions are closely linked.³ This is in fact how the construction in [AGKZ20] works.

One-way permutations from obfuscation? The concept of cryptographically useful *program obfuscation* dates to the pioneering work of Diffie-Hellman [DH76]. Obfuscation allows for embedding cryptographic secrets into public software. The proposal of [DH76], though not phrased in this language, is the following: Obfuscate a pseudorandom permutation (PRP) to get a *trapdoor* one-way permutation. More generally, obfuscation heuristically translates security using oracles into standard-model security, by actually giving out the obfuscated code of the oracles.

It ultimately took over 30 years for cryptographically useful general-purpose obfuscation to emerge [GGH⁺13]. Unfortunately, by then it was shown that obfuscation cannot in general translate oracles into the standard model [BGI⁺01]. For example, obfuscating an arbitrary PRP provably cannot guarantee any meaningful security. Instead, the community has settled on a much weaker but precise notion of *indistinguishability obfuscation* (iO), which says that the obfuscations of programs with the same functionality are computationally indistinguishable [BGI⁺01].

Though much weaker than ideal, numerous techniques have been developed to use iO. Most revolve around the use of pseudorandom *functions* (PRFs). While PRFs, just like PRPs, in general cannot be obfuscated, a strengthening known as a *puncturable* PRF [KPTZ13, BW13, BGI14] actually does give provable guarantees when using iO [SW14], leading to numerous positive results.

Despite many successes, some major open questions remain. Notably, Diffie and Hellman’s original proposal of obtaining a trapdoor permutation by obfuscating a PRP remains open. In fact, to the best of our knowledge, there are no known positive iO results that obfuscate PRPs to achieve *any* goal. One key challenge is that there has been no construction of a puncturable *PRP* or analogous object, and some evidence suggests that they may be *impossible* [BKW17].

Note that [BPW16, GPSZ17] construct trapdoor permutations using iO, though these permutations are not *full-domain*, i.e., where if n is the number of bits needed to represent any element in the domain (formally, the permutation domain χ is such that $\chi \subseteq \{0, 1\}^n$), then we have the equality $\chi = \{0, 1\}^n$. These works use a very different structure and do not simply obfuscate a PRP. Moreover, these solutions have a major drawback that the domain χ of the permutation is a sparse set in $\{0, 1\}^n$ that cannot be directly sampled from nor efficiently recognized. This complicates their use in applications (see for example [CL18]) and it is unknown how to use them in quantum settings [MY23], where one further wants to efficiently generate quantum superpositions

³The first connection between such a separation and unclonable cryptography was due to [Zha19], who shows that a separation implies the weaker object called quantum lightning. These works improved the implication to OSS.

over subsets of the domain. An open question is if full-domain (trapdoor) OWPs are possible from iO, whether by obfuscating a PRP or by other means. There is some indication that such a OWP may in fact be impossible [AS16].

1.1 Results and Paper Organization

There are two parts to our work.

Part 1: Classical Oracle OSS. The first part proves Theorems 1 and 2 given below. It includes an oracle construction of a non-collapsing hash function and an unconditional proof of collision resistance. A proof overview is given in Section 2 and the full proof is in Section 4.

Theorem 1. *Relative to a classical oracle, secure OSS exists.*

Similarly to the blueprint suggested in [AGKZ20], we prove Theorem 1 through separating quantum hashing notions:

Theorem 2. *Relative to a classical oracle, there exist post-quantum collision-resistant hash functions that are non-collapsing, and there exist post-quantum classically-binding commitments that are not collapse-binding.*

We do not actually know how to prove security for the oracle in [AGKZ20]. Instead, we prove Theorems 1 and 2 by a new construction. Our construction is inspired by the previous work of [AGKZ20] and a proposal made by [Bar23] at the NTT Research Quantum Money Workshop [NTT23]. We prove security through a new technique, showing the random self-reducibility of our collision problem, and a sequence of reductions to simpler problems. Ultimately we show collision finding in our hash is no easier than collision finding in plain 2-to-1 random functions, which is known to be hard. This gives the first classical-oracle construction of OSS⁴, and also the first such separation of classical- and collapse-binding. The construction is even the first classical-oracle construction of *quantum lightning*, a weaker notion than OSS, proposed in [Zha19].

Part 2: Standard-model OSS. The second part of this paper proves Theorems 3, 4, 5 and 6 below. It includes the development of a new cryptographic notion we call permutable pseudorandom permutations (permutable PRPs). We show how permutable PRPs can be used to prove the security of our construction in the standard model, and also for solving a number of long-standing open problems in cryptography, as elaborated below.

Theorem 3. *There exists secure OSS assuming each of the following: (1) sub-exponentially-secure indistinguishability obfuscation, (2) sub-exponentially-secure one-way functions, and (3) (polynomially-secure) LWE with a sub-exponential noise-modulus ratio.*

Theorem 4. *Under the same assumptions as Theorem 3, there exist post-quantum collision-resistant hash functions that are non-collapsing, and there exist post-quantum classically-binding commitments that are not collapse-binding.*

⁴By “classical-oracle construction,” we mean the first construction *provably* and *unconditionally* secure relative to a classical oracle.

As before, Theorem 3 follows from Theorem 4, which is proved in Section 6. This gives the first standard-model OSS⁵, and also the first standard-model separation between classical- and collapse-binding, solving this decade-old question⁶. Even for the weaker notion of quantum lightning, for which there exist a handful of candidates, this is the first construction with provable security under widely-studied assumptions.

The basic idea is to obfuscate the functions in our oracle construction rather than putting it in an oracle. The main technical gap between proving security in the oracle model and the standard model stems from the fact that the construction uses random permutations. Our oracle construction uses a (truly) random permutation and our standard model construction uses a pseudorandom permutation (PRP). Obfuscating PRPs and getting any formal security guarantees, however, is a known challenge in cryptography, independently of quantum computation.

We develop a new notion of PRPs, called *permutable* PRPs, that can be obfuscated using iO with provable security. A permutable PRP Π very roughly allows the following: Given a key k and a (known) permutation Γ in the form of having its circuit, one can produce a “permuted” key k^Γ , which allows for computing $\Gamma(\Pi(k, \cdot))$ (as well as the inverse $\Pi^{-1}(k, \Gamma^{-1}(\cdot))$). Moreover, the key k^Γ hides the fact that the outputs were permuted by Γ . See Section 2 for a more detailed explanation, and Section 5 for a formal definition, as well as a formal statement and proof of the following:

Theorem 5 (Informal). *There exist permutable PRPs for a large class of Γ assuming (1) sub-exponentially-secure iO and (2) sub-exponentially-secure one-way functions.*

Such PRPs can be seen as the PRP analogue of a puncturable pseudorandom function (PRF), which is one of the main techniques used to prove security in the iO literature. To the best of our knowledge, this is the first example which provably obfuscates a PRP. We also show that our techniques are quite general, and in Section 5 we prove the following:

Theorem 6. *There exist full-domain trapdoor one-way permutations (OWPs), assuming (1) sub-exponentially-secure iO and (2) sub-exponentially-secure one-way functions.*

Here, we remind that our notion of “full-domain” means that the permutation domain is just $\{0, 1\}^n$. We thus solve the decade-old problem of constructing full-domain (trapdoor) one-way permutations from iO⁷, and give an answer to the decades-old problem of obfuscating PRPs to obtain trapdoor permutations. In light of the impossibility in [AS16], Theorem 6 may seem surprising. However, we explain in Section 1.4 that it actually does *not* contradict their impossibility.

Our trapdoor OWP simply obfuscates a permutable PRP. The proof is straightforward given a permutable PRP; the bulk of the technical effort is then in constructing the permutable PRP in Theorem 5. This demonstrates the utility of our new PRP notion.

For another application, by plugging into the elegant proof of quantumness of [MY23], we immediately obtain:

Corollary 7. *Assuming sub-exponentially (classically) secure iO and sub-exponentially (classically) secure one-way functions, there exists a proof of quantumness protocol.*

This is the first proof of quantumness using iO, as the non-full-domain trapdoor permutations of [BPW16, GPSZ17] cannot be used in this construction.

⁵By “standard-model,” we mean with provable security under widely-used computational assumptions, as opposed to merely conjecturing that a construction is secure.

⁶[Unr16b] was first made public in early 2015.

⁷[BPW16] was first made public in early 2015.

Paper Organization. In the remainder of the introduction, we discuss additional motivation and related work (Section 1.2), why the bug in [AGKZ20] is unfixable (Section 1.3), and why the impossibility of [AS16] does not apply to our construction (Section 1.4). In Section 2, we provide an overview of our techniques. A reader only interested in our results on obfuscating PRPs, including our application to trapdoor permutations, can find an overview in Section 2.3. Section 2.3 is entirely classical and can be read independently of the rest of the paper without any background in quantum computing. A reader interested in our OSS construction should start with Section 2.2 which gives our oracle construction and an overview of the oracle proof. Then after developing our techniques for obfuscating PRPs, we explain how to translate our oracle construction into a standard-model construction in Section 2.4.

1.2 Motivation and Other Related Work

Quantum money and variants. Quantum money uses unclonable quantum states as currency to prevent counterfeiting. One-shot signatures lives at the top of a hierarchy of related concepts:

- **Secret key quantum money.** This was originally proposed by Wiesner [Wie83]. A major drawback, however, is that only the mint is able to verify, leading to a number of limitations.
- **Public key quantum money.** This was proposed by Aaronson [Aar09] to remedy the various issues with Wiesner’s scheme. Here, anyone can verify banknotes but only the mint can create new notes. It has been a major challenge to construct public key quantum money. Several candidate constructions exist [FGH⁺12, KSS22, LMZ23, Zha24]. It was also shown to exist in a classical oracle model [AC12], which was later improved to a standard-model construction using iO by [Zha19].
- **Quantum lightning.** This concept allows anyone to mint banknotes together with classical serial numbers such that anyone can verify pairs of a serial number and a quantum banknote, but ensures that no user can create two valid banknotes with the same serial number. This last property of unclonability (even for the state generator) allows to use quantum lightning in decentralized settings, in ways that standard (public-key) quantum money cannot generally be used. Quantum lightning was first suggested by [LAF⁺09] (under the name “collision-free quantum money”) and rigorously formalized by [Zha19]. It implies in particular a public-key quantum money scheme, where a banknote is a quantum lightning state/serial number pair $|\$, \sigma\rangle$, together with a signature on σ , signed using the mint’s (standard classical post-quantum) signing key. Some of the quantum money candidates are also quantum lightning [FGH⁺12, KSS22, LMZ23, Zha24]. But others, including the classical oracle and iO results mentioned above, are not quantum lightning. Prior to our work, all quantum lightning schemes required novel computational assumptions that had not been studied by the wider cryptography community. Moreover, no prior scheme has provable security in a classical oracle model. In fact, our OSS gives in particular the first quantum lightning scheme with provable security under widely-studied assumptions, and the first unconditional proof in a classical oracle model.
- **One-shot signatures (OSS).** Further strengthening quantum lightning, OSS treats the lightning state as a quantum signing key, which can sign a single message and then provably self-destructs. Among other things, the upgrade from quantum lightning to OSS adds the ability to use only classical communication and local quantum computation, decrease the needed

coherence times for quantum lightning states, and more. No known provable classical-oracle constructions nor standard-model instantiations (under *any* reasonable-sounding assumption) were known prior to this work.

Quantum cryptography with classical communication. An interesting application of OSS is to send quantum money using classical communication. This may sound impossible at first, but [AGKZ20] observe that it is nevertheless possible to send quantum money using a simple classical *interactive* protocol: for the mint to send a money state to Alice, Alice will create a lightning state/OSS signing key and serial number *for herself* $|\$_{\text{Alice}}\rangle, \sigma_{\text{Alice}}$, and send σ to the mint. The mint then signs the serial number, providing the signature $\sigma_{\text{mint} \rightarrow \text{Alice}}$, which the mint sends back to Alice. Now Alice’s money state is $|\$_{\text{Alice}}\rangle, \sigma_{\text{Alice}}, \sigma_{\text{mint} \rightarrow \text{Alice}}$, which was obtained by just sending classical messages! Then, if Alice wants to send money to Bob, Bob simply creates a new lightning state/OSS signing key and serial number *for himself* $|\$_{\text{Bob}}\rangle, \sigma_{\text{Bob}}$, sends σ_{Bob} to Alice, who then signs σ_{Bob} using her quantum lightning/OSS signing key, obtaining signature $\sigma_{\text{Alice} \rightarrow \text{Bob}}$, which she sends to Bob. Bob now has the money state $|\$_{\text{Bob}}\rangle, \sigma_{\text{Bob}}, \sigma_{\text{Alice} \rightarrow \text{Bob}}, \sigma_{\text{mint} \rightarrow \text{Alice}}$. By the OSS guarantee, Alice’s money state has now self-destructed, meaning she no longer has the money but Bob does. Bob can then send the money to Charlie, etc.⁸

The Mint-to-Alice step using classical communication was previously solved by [Shm22a] using iO and other tools, but the money could not be subsequently sent to Bob without quantum communication⁹. Our work allows for Alice to send money to any party in the system classically and directly. These works on quantum money with classical communication are part of a broader class of protocols for performing quantum cryptography using classical communication, such as tests of quantumness and certified randomness [BCM⁺18], position verification [LLQ22], and certified deletion [BKP23, BK23, KNY23, BGK⁺24].

Cryptocurrencies and Blockchains. One-shot signatures have numerous applications in the cryptocurrency and blockchain settings. For example, they can be used to give decentralized currency and even smart contracts without a blockchain at all [Zha19, AGKZ20, Sat22]. OSS are known [CS20] to provide a solution to the blockchain scalability problem¹⁰, using only classical communication. They also have various advantages for other blockchain-related tasks, such as decreasing the threshold for perfect finality, eliminating slashing and leakage risks in liquid staking, and more [Dra23].

Obfuscating pseudorandom objects. Perhaps the main technique in the literature for using iO is the punctured programming paradigm [SW14], which primarily utilizes a puncturable pseudorandom function [KPTZ13, BW13, BGI14]. These are functions where one can give out a “punctured” key, which allows for evaluating the PRF on all but a single point x . Meanwhile, even with this ability, the value on x remains pseudorandom.

⁸The Mint-to-Alice step actually only requires quantum lightning, but Alice-to-Bob, etc seem to require the stronger OSS.

⁹Using the results from [Shm22b] it is possible to use only classical communication and send money to Bob. However, this still necessitates Alice to communicate with the Bank repeatedly for every new transaction.

¹⁰The work of [CS20] uses a strengthening of quantum lightning, where there is an additional procedure to destroy quantum lightning states and produce a classical “proof of deletion”. This strengthening is known to follow from OSS, but not from standard quantum lightning.

Puncturable *invertible* functions were considered in [BKW17], though their construction expands the input size, so it is not a permutation but rather an (efficiently invertible) injection. They also discuss puncturable PRPs, and explain that they are impossible in some settings. In particular, in the setting where the domain is polynomial-sized, if the punctured key reveals the permutation on all points but x , it also reveals x .

Our notion of puncturable PRPs avoids this issue and even is valid in the small-domain setting. We can “puncture” a puncturable PRP at a point x with output y by choosing a random image y' , letting Γ be the transposition which swaps y and y' , and outputting the permuted key k^Γ . This allows for computing the PRP on all points except for x (and also the pre-image x' of y'), but hides the true value of y . The impossibility of punctured PRPs has come up in several settings (e.g. [SACM21, MZRA22, LMZ23, HPPY24]). It would be interesting to explore if our notion could be useful in these settings.

(Trapdoor) OWPs from iO. Trapdoor one-way permutations (OWP) were proposed by [Yao82] to abstract the ideas behind public key cryptosystems based on both RSA and Rabin. The domain and range of these objects are sets with algebraic structure. However, it became much simpler to describe applications in terms of a full-domain OWP where the domain and range are simply $\{0,1\}^n$. This simplification led to several conceptual errors (see [GR13] for explanation). To account for these errors, when using sparse-domain OWPs, one must often stipulate additional conditions (called “enhanced” or “doubly enhanced”) that are trivially satisfied by full-domain OWPs.

Once iO emerged as a powerful cryptographic tool [GGH⁺13], a natural question was whether iO could give a third way of building a trapdoor permutation. A particular motivation is to achieve post-quantum security, since RSA and Rabin are both quantumly insecure due to Shor’s algorithm [Sho94]. Constructions were given [BPW16, GPSZ17], but these constructions were “messy,” with sparse domains that could not be efficiently recognized and required cryptographic procedures to sample. An interesting question was whether a clean, full-domain trapdoor permutation was possible from iO. Our work mostly resolves this question, though we still have the limitation that the space of keys is sparse.

As a concrete application, our trapdoor OWPs can be plugged into the elegant proofs of quantumness of [MY23], to obtain proofs of quantumness from (classically-hard sub-exponential) iO and one-way functions¹¹. This is the first proof of quantumness based on iO. The prior OWPs of [BPW16, GPSZ17], despite being doubly enhanced, did not suffice for this application. More generally, permutations on simple domains like $\{0,1\}^n$ appear much more useful in the context of quantum cryptosystems, whereas permutations on highly structured domains break the delicate structure of quantum states.

1.3 The Bug in the previous work of AGKZ

The bug in [AGKZ20] as found by [Bar23] is rather technical, and we do not discuss it here. However, we argue that the bug is likely unfixable using the techniques employed by [AGKZ20]. The issue is that in [AGKZ20], the security of the OSS boils down to showing the collision resistance of a certain hash function, and the proof technique uses what is known as the inner-product adversary method – first developed in [AC12]. Unfortunately, the inner-product adversary method

¹¹The OWP part of our work does not need LWE.

is unlikely to be able to prove even the basic collision problem is hard, even before adding all the structure that [AGKZ20] need to obtain their OSS scheme. The reason is that the collision problem has a small *certificate complexity*, and it is known that the adversary method cannot prove good lower-bounds for problems with small certificate complexity, as shown by [Aar03]. This certificate complexity barrier extends to all “positive weight” adversary methods, including the inner-product method. Thus, it seems any fixed proof for [AGKZ20] must use additional techniques.¹²

Note that most of our oracle result is proved using standard adversary method techniques. However, our result assumes the standard collision lower bound as previously proved in [AS04, Zha15]. These results were proved using the polynomial method that is not subject to the certificate complexity barrier.

1.4 On the Lower Bound for OWP from iO

Asharov and Segev [AS16] show a barrier for constructing OWPs from iO and one-way functions. Specifically, they show that any “black-box” construction of one-way permutations from iO and one-way functions, cannot be “domain-invariant”. Here, black-box in the context of iO is a bit subtle, but their notion captures most known iO techniques, including ours. Recall that in a (keyed) OWP, we sample one permutation out of exponentially many permutations in an efficiently samplable family. As for domain-invariance, the abstract of [AS16] defines it as “each permutation [in the family] may have its own domain, but these domains are independent of the underlying building blocks”. At first glance, this would seem to contradict our full-domain trapdoor permutation, since the domain of all permutations in the family of our construction is just $\{0, 1\}^n$, clearly independent of any building block.

However, there are really two types of domain invariance: One for the key, and one for the actual input. The formal specification of the impossibility in [AS16] reveals that their notion of domain-invariance refers to schemes that satisfy *both* notions. In fact, in the model they propose, obfuscating a PRP actually does yield an *input*-domain-invariant (trapdoor) OWP; also, since the key is an obfuscated program and most strings do not correspond to valid programs / keys, it is not key-domain invariant.

Our (trapdoor) OWPs are similarly input-domain invariant but not key-domain invariant, since our key is likewise an obfuscated program, with the added benefit of provable security under iO. Thus, our result does not contradict the impossibility of [AS16]. Instead, it helps clarify the impossibility as being more about key-domain-invariance.

1.5 Directions for Future Work

Here are some natural follow-up directions from our work:

- Can OSS be achieved without using iO? While there are several approaches (using new hardness assumptions) that seem to give weaker objects like quantum lightning [FGH⁺12, KSS22, LMZ23, Zha24], they do not seem amenable to producing classical signatures, even heuristically.

¹²One caveat is that the barrier only applies to lower bounds for the number of queries to distinguish a function with collisions from a function that is injective. Such a lower-bound immediately implies a lower-bound for actually finding collisions. But the converse is not true, and the certificate complexity barrier does *not* seem to rule out directly proving the hardness of finding collisions. Nevertheless, it does not seem like the techniques of [AGKZ20] circumvent this barrier.

- Is it possible to remove the need for sub-exponential hardness in our constructions? Sub-exponential hardness arises in two key places. The first is in our PRP: we start with a “base” PRP (based just on one-way functions) which is permutable only for the class of “neighbor swaps” that exchange a given z with $z + 1$, and upgrade it to more general permutations Γ by decomposing such permutations into a sequence of neighbor swaps. The proof incurring a security loss for each step in the decomposition. General permutations, even just transpositions (which are enough to get trapdoor OWPs) require an exponential-sized decomposition. In order to get around this limitation, it seems that the “base” PRP needs to support a richer class than just neighbor swaps. Is there a base PRP that support, say, general transpositions, from just one-way functions? The second place sub-exponential hardness comes in is when we are switching from the construction to a simulated distribution; this utilizes a hybrid over all possible outputs. Perhaps there is a clever way to change all possible outputs in one go.
- Can we achieve a “clean iO” approach to OSS that uses just iO and generic primitives? Note that it is unlikely that OSS can be obtained from iO and one-way functions: OSS requires collision resistance, which is believed to not be possible from iO and one-way functions alone [AS15]. But perhaps iO and collision-resistance is enough. Or at a minimum, maybe our LWE assumption can be replaced with other algebraic techniques to give a diversity of assumptions.
- We introduced a new technique for obfuscating PRPs, and it would be interesting to see if this enables any new results, or at least streamlines old results.

Finally, we conclude with a fascinating complexity-theoretic question inspired by our techniques. Any permutation Γ on $\{0, 1\}^n$ can be decomposed into an exponentially long product of transpositions $\Gamma = (a_1 \ a_2) \circ (a_3 \ a_4) \circ \dots$. While such a decomposition is clearly inefficient, the following question asks if the partial products can be made small:

Question 8 (Efficient Permutation Decomposition Problem). *For any permutation Γ on $\{0, 1\}^n$ such that Γ, Γ^{-1} have circuits of size s , is it possible to decompose Γ into a product of transpositions $\Gamma = \tau_1 \circ \tau_2 \circ \dots \circ \tau_T$ such that each partial product $\Gamma_t := \tau_1 \circ \tau_2 \circ \dots \circ \tau_t$ for $t \in [T]$ and its inverse Γ_t^{-1} have circuits of size $\text{poly}(s, n)$?*

As part of constructing our permutable PRPs and applying them to construct OSS, we show that very general families of permutations can be efficiently decomposed in this way (see Figure 5 for a non-exhaustive list of such families). On the other hand, we do not know how to handle all efficient permutations, and even for very simple permutations like multiplying by a scalar mod N , we only know a general solution assuming the Extended Riemann Hypothesis.

2 Technical Overview

In this section we explain the main techniques shown in this work.

2.1 Definitions

We first give the formal definition of one-shot signatures.

Definition 9 (One-Shot Signature Scheme). *A one-shot signature (OSS) scheme is a tuple of algorithms (Setup, Gen, Sign, Ver) together with message-space $(\mathcal{M}_\lambda)_\lambda$ satisfying the following:*

- $\text{CRS} \leftarrow \text{Setup}(1^\lambda)$: *A classical probabilistic polynomial-time algorithm that given the security parameter, samples the classical common reference string (CRS).*
- $(\text{pk}, |\text{sk}\rangle) \leftarrow \text{Gen}(\text{CRS})$: *A quantum polynomial-time algorithm that takes the classical CRS and samples a classical public key pk and quantum secret key $|\text{sk}\rangle$.*
- $\sigma \leftarrow \text{Sign}(\text{CRS}, |\text{sk}\rangle, m)$: *A quantum polynomial-time algorithm that given CRS and the quantum key $|\text{sk}\rangle$, given any message $m \in \mathcal{M}_\lambda$ produces a classical signature σ .*
- $\text{Ver}(\text{CRS}, \text{pk}, m, \sigma) \in \{0, 1\}$: *A classical deterministic polynomial-time algorithm which verifies a message m and signature σ relative to a public key pk .*
- **Correctness:** *There exists a negligible function negl such that, for any λ and any $m \in \mathcal{M}_\lambda$ we have*

$$\Pr_{\substack{\text{CRS} \leftarrow \text{Setup}(1^\lambda), \\ (\text{pk}, |\text{sk}\rangle) \leftarrow \text{Gen}(\text{CRS}), \\ \sigma_m \leftarrow \text{Sign}(\text{CRS}, |\text{sk}\rangle, m)}} [\text{Ver}(\text{CRS}, \text{pk}, m, \sigma_m) = 1] \geq 1 - \text{negl}(\lambda) \quad .$$

- **Security:** *For any QPT algorithm $\mathcal{A}$, there exists a negligible function negl such that for all λ the following probability is bounded by $\leq \text{negl}(\lambda)$,*

$$\Pr[(m_0 \neq m_1) \wedge (\text{Ver}(\text{CRS}, \text{pk}, m_0, \sigma_0) = 1) \wedge (\text{Ver}(\text{CRS}, \text{pk}, m_1, \sigma_1) = 1)] \quad ,$$

where the above probability is over $\text{CRS} \leftarrow \text{Setup}(1^\lambda)$, $(\text{pk}, m_0, m_1, \sigma_0, \sigma_1) \leftarrow \mathcal{A}(\text{CRS})$.

It is straightforward to adapt the above definition to utilize an oracle. In this case, the oracle will play the role of CRS, and we will omit the algorithm Setup.

OSS from Non-collapsing Hashing. As observed informally by [AGKZ20], a collision-resistant but non-collapsing hash (or, non-collapsing CRH in short) function gives an OSS, and this was made formal and general by [DS23]. We next recall the definition of a non-collapsing CRH.

Definition 10 (Collision-Resistant Always-Non-Collapsing Hash). *A collision-resistant always-non-collapsing hash function is a pair of PPT algorithms (Setup, H) such that*

- $\text{CRS} \leftarrow \text{Setup}(1^\lambda)$: *A classical probabilistic polynomial-time algorithm that given the security parameter, samples the classical common reference string (CRS).*
- $y \leftarrow H(\text{CRS}, x)$: *A classical deterministic polynomial-time algorithm that given the CRS and input x , outputs y .*
- **Collision-resistance:** *For any QPT algorithm $\mathcal{A}$, there exists a negligible function negl such that for all λ ,*

$$\Pr_{\substack{\text{CRS} \leftarrow \text{Setup}(1^\lambda), \\ (x_0, x_1) \leftarrow \mathcal{A}(\text{CRS})}} [(x_0 \neq x_1) \wedge (H(\text{CRS}, x_0) = H(\text{CRS}, x_1))] \leq \text{negl}(\lambda) \quad .$$

- **Always non-collapsing:** *There exists a pair of QPT algorithms $(\mathcal{S}, \mathcal{D})$ and a negligible function $\text{negl}(\lambda)$ such that for all λ ,*

$$\left| \Pr \left[\mathcal{D}(x, \text{aux}) = 1 : \begin{array}{l} \text{CRS} \leftarrow \text{Setup}(1^\lambda), \\ (|\psi\rangle, \text{aux}) \leftarrow \mathcal{S}(\text{CRS}), \\ x \leftarrow \text{Measure}(|\psi\rangle) \end{array} \right] - \Pr \left[\mathcal{D}(|\psi_y\rangle, \text{aux}) = 1 : \begin{array}{l} \text{CRS} \leftarrow \text{Setup}(1^\lambda), \\ (|\psi\rangle, \text{aux}) \leftarrow \mathcal{S}(\text{CRS}), \\ |\psi_y\rangle \leftarrow \text{PartialMeasure}_H(\text{CRS}, \cdot)(|\psi\rangle) \end{array} \right] \right| \geq 1 - \text{negl}(\lambda) .$$

Here, $x \leftarrow \text{Measure}(|\psi\rangle)$ means to measure $|\psi\rangle$ in the computational basis arriving at measurement x . $|\psi_y\rangle \leftarrow \text{PartialMeasure}_H(\text{CRS}, \cdot)$ means to compute $H(\text{CRS}, \cdot)$ (in superposition, with output register on the side) with input register $|\psi\rangle$, and measuring the result, resulting in outcome y . Then the state $|\psi\rangle$ collapses to $|\psi_y\rangle$, which contains some superposition of preimages of y .

For the interested reader on how a non-collapsing CRH generically gives an OSS, we recall the proof sketch from [AGKZ20, DS23]. The idea is that Gen runs $\mathcal{S}$ to get $|\psi\rangle, \text{aux}$, and then applies $H(\text{CRS}, \cdot)$ to $|\psi\rangle$ and measures, obtaining y . We set $\text{pk} = y$, and $|\text{sk}\rangle = |\psi_y\rangle, \text{aux}$ where $|\psi_y\rangle$ is the post-measurement state. Observe that the support of $|\psi_y\rangle$ are strings x that hash to y .

For a bit b , let $|\psi_{y,b}\rangle$ be the state post-selecting on x whose first bit is b . One can create $|\psi_{y,b}\rangle$ for a random choice of b by simply measuring the first qubit. Then very roughly it is shown in [AGKZ20, DS23] how to use the distinguisher $\mathcal{D}$ to move back and forth between $|\psi_{y,0}\rangle$ and $|\psi_{y,1}\rangle$.

To sign a bit b , one obtains $|\psi_{y,b}\rangle$ and measures, obtaining a string x beginning with the bit b that hashes to y . This is a signature on b , which is easy to verify. Moreover, the collision-resistance of H implies that it is computationally infeasible to find two signatures. Thus, we have an OSS for a single bit. Then by parallel repetition, we obtain an OSS for arbitrary messages.

Remark 11 (Direct Construction of OSS). *We note that the specific construction from our work which we will next see, gives both a direct construction of OSS and a direct construction of a non-collapsing CRH. That is, we construct a non-collapsing CRH where there is no need for the transformation of [DS23] to get an OSS. Due to the fact that a non-collapsing CRH is formally a stronger primitive, in this work we focus on its construction. For the direct construction of OSS out of the non-collapsing CRH, we describe it formally in Section 4, Construction 31.*

2.2 OSS Relative to a Classical Oracle (Section 4)

Since OSS follows from non-collapsing collision-resistant hash functions, we will focus on constructing the latter.

The Construction From [AGKZ20]. The OSS of [AGKZ20] utilizes what we will call a *coset partition function* in this work. This is a function Q that is many-to-1, and where the pre-image set of any image is a coset of a linear subspace: That is, the pre-image sets have the form $Q^{-1}(y) := \{\mathbf{A}_y \cdot \mathbf{r} + \mathbf{b}_y : \mathbf{r} \in \mathbb{Z}_2^\ell\}$, where $\mathbf{A}_y \in \mathbb{Z}_2^{k \times \ell}, \mathbf{b}_y \in \mathbb{Z}_2^k$ are a matrix/vector pair that depends on the image y . The function Q will be the hash function, provided as an oracle.

In order to be non-collapsing, [AGKZ20] employ the hidden subspaces approach of [AC12], and additionally provide a separate oracle D , which provides membership testing for the linear space

$\mathbf{A}^\perp := \{\mathbf{z} \in \mathbb{Z}_2^k : \mathbf{z}^T \cdot \mathbf{A}_y = 0^\ell\}$. This allows for testing if a state is in the uniform superposition $|Q^{-1}(y)\rangle := \sum_{x \in Q^{-1}(y)} |x\rangle$: First use Q to test that the state is contained in the support of $Q^{-1}(y)$, then apply the quantum Fourier transform (QFT), and use D to test that the resulting state has support on $\mathbf{A}^\perp$. The only state that passes both verifications is the state $|Q^{-1}(y)\rangle$.

In order to obtain a non-collapsing hash/OSS scheme, then one needs to prove that Q is collision-resistant, given the oracles for Q and D . As mentioned, the proof provided in [AGKZZ20] contained a fatal bug, and while the scheme is plausibly collision-resistant, it remains unclear how to prove this.

Relaxing the structure. A key challenge with the approach in [AGKZZ20] is that it is non-trivial come up with a coset partition function that is not trivially insecure. The cosets need to have very different orientations ($\mathbf{A}_y$ values), lest the function Q be periodic and therefore subject to quantum period-finding algorithms [Sho94]. But how do we perfectly partition the domain into cosets that are all of different orientations, but no overlaps? The coset partition function used in [AGKZZ20] is derived in a very specific way, and the various pre-image sets are highly correlated, making reasoning about them quite challenging. This is further complicated by the D oracle, which contains even more information about the cosets. An interesting proposal was made by [Bar23]: Simply have the hash function be a random function H , but provide an additional oracle which maps each pre-image set $H^{-1}(y)$ to a random coset S_y embedded in a much larger space. The point is that, the cosets S_y for different y 's no longer need to partition the space they live in, as they are independent of each other.

Our following construction is inspired by the above general principles. We sample a random secret permutation Π on $\{0, 1\}^n$, and let $H(x), J(x)$ denote the first r bits and last $n - r$ bits of $\Pi(x)$, respectively. $H : \{0, 1\}^n \rightarrow \{0, 1\}^r$ will be our hash function. For each $y \in \{0, 1\}^r$, we also choose a random coset $S_y \subseteq \{0, 1\}^k$ of dimension $n - r$, described by a matrix/vector pair $\mathbf{A}_y \in \mathbb{Z}_2^{k \times (n-r)}, \mathbf{b}_y \in \mathbb{Z}_2^k$ as $S_y = \{\mathbf{A}_y \cdot \mathbf{r} + \mathbf{b}_y : \mathbf{r} \in \{0, 1\}^{n-r}\}$. We then provide an oracle $\mathcal{P}(x)$ which outputs $(H(x), \mathbf{u} = \mathbf{A}_{H(x)} \cdot J(x) + \mathbf{b}_{H(x)})$. In other words, it computes $y := H(x)$, then uses this information to determine a coset, and then uses the remaining permutation information $J(x)$ to determine the point $\mathbf{u}$ in that coset that x maps to. We likewise provide the oracle $\mathcal{D}(y, \mathbf{v})$ which checks if $\mathbf{v} \in S_y^\perp$, where $S_y^\perp$ is the kernel of $\mathbf{A}_y$.

We have to provide one more oracle, which is denoted $\mathcal{P}^{-1}(y, \mathbf{u})$, which inverts the oracle $\mathcal{P}$: It outputs x if $\mathcal{P}(x) = (y, \mathbf{u})$ and otherwise outputs a special symbol $\perp$, indicating that $(y, \mathbf{u})$ is not in the range of $\mathcal{P}$. This oracle is necessary in order to actually preserve the non-collapsing property: Given the uniform superposition of pre-images $|H^{-1}(y)\rangle$, applying the oracle $\mathcal{P}$ gives $\sum_{x \in H^{-1}(y)} |x, \mathcal{P}(x)\rangle = \sum_{x \in H^{-1}(y)} |x, y, \mathbf{A}_y \cdot J(x) + \mathbf{b}_y\rangle$. We would like to apply the QFT to the register containing $\mathbf{A}_y \cdot J(x) + \mathbf{b}_y$, but this will not work: since the register x remains around and is entangled with the register containing $\mathbf{A}_y \cdot J(x) + \mathbf{b}_y$, performing the QFT will actually yield random junk. The oracle $\mathcal{P}^{-1}$ allows us to un-compute x , thereby allowing verification to work as desired.

We now need to prove the collision-resistance of H . Unfortunately, quantum query lower-bounds for oracles with inverses is a notorious challenging problem. This problem, together with the presence of the oracle $\mathcal{D}$, were the primary barriers to proving the security of this construction.

In what follows, we describe our proof in the oracle setting. At a very high-level, our proof will gradually eliminate parts of the oracles $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}$, until all that remains is a simple hash function oracle, and we can then invoke the known query lower-bounds for collision-finding to conclude

security.

Warm-up: A Random Self-Reduction. As a warm-up that will lead to our proof, we introduce a random self-reduction for the oracles $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}$. Given an instance of the oracles $\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}, \overline{\mathcal{D}}$ (with underlying arbitrary permutation $\overline{\Pi}$ on $\{0, 1\}^n$ and arbitrary matrix/vector pairs $\overline{\mathbf{A}}_y \in \mathbb{Z}_2^{k \times (n-r)}, \overline{\mathbf{b}}_y \in \mathbb{Z}_2^k$ for all $y \in \{0, 1\}^r$, where all we know is that $\overline{\mathbf{A}}_y$ is full-rank), we can construct another instance as follows. Choose a random permutation Γ on $\{0, 1\}^n$ and for every $y \in \{0, 1\}^r$, choose a random full-rank $\mathbf{C}_y \in \{0, 1\}^{k \times k}$ and random $\mathbf{d}_y \in \{0, 1\}^k$. Note that $\mathbf{C}_y, \mathbf{d}_y$ define a random affine permutation $L_y(\mathbf{u}) = \mathbf{C}_y \cdot \mathbf{u} + \mathbf{d}_y$ on $\{0, 1\}^k$. Then define:

- $\mathcal{P}(x)$: Compute $\bar{x} \leftarrow \Gamma(x)$, $(y, \bar{\mathbf{u}}) \leftarrow \overline{\mathcal{P}}(\bar{x})$, $\mathbf{u} \leftarrow L_y(\bar{\mathbf{u}})$ and output $(y, \mathbf{u})$.
- $\mathcal{P}^{-1}(y, \mathbf{u})$: Compute $\bar{\mathbf{u}} \leftarrow L_y^{-1}(\mathbf{u})$, $\bar{x} \leftarrow \overline{\mathcal{P}}^{-1}(y, \bar{\mathbf{u}})$, $x \leftarrow \Gamma^{-1}(\bar{x})$ and output x .
- $\mathcal{D}(y, \mathbf{v})$: Output $\overline{\mathcal{D}}(y, \mathbf{v}^T \cdot \mathbf{C}_y)$.

The resulting oracle implicitly sets $\Pi := \overline{\Pi} \circ \Gamma$, and for every $y \in \{0, 1\}^r$ sets $\mathbf{A}_y := \mathbf{C}_y \cdot \overline{\mathbf{A}}_y$, $\mathbf{b}_y := \mathbf{C}_y \cdot \overline{\mathbf{b}}_y + \mathbf{d}_y$, which all distribute independently of the underlying $\overline{\Pi}$, $\overline{\mathbf{A}}_y$, $\overline{\mathbf{b}}_y$. Thus this random self-reduction turns any instance $\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}, \overline{\mathcal{D}}$ into a fresh independent instance $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}$. Moreover, if we let H and $\overline{H}$ be the functions outputting the first r bits of $\mathcal{P}$ and $\overline{\mathcal{P}}$ respectively, we can turn a collision for H into a collision for $\overline{H}$ by applying Γ . Thus, we can derive the collision-resistance for random instances based on the collision resistance of any fixed instance. We note that there does not appear to be an analogous random self-reduction for the oracles of [AGKZ20]. We will use variants of this self-reduction to gradually remove components from our oracle.

Step 1: Bloating the Dual. The first step of our actual proof, inspired by techniques of [Zha19], is to “bloat the dual”. This means, for each y , we choose a random super-space $T_y^\perp$ of $S_y^\perp := \text{ColSpan}(\mathbf{A}_y)^\perp$, and replace $\mathcal{D}$ with the oracle $\mathcal{D}'$ which checks for membership in $T_y^\perp$. The super-space $T_y^\perp$ is chosen at random such that (1) it is a sparse subset of $\{0, 1\}^k$ and (2) $S_y^\perp$ is a sparse subset of $T_y^\perp$. We can prove that $\mathcal{D}'$ is indistinguishable from $\mathcal{D}$, since the points in $T_y^\perp$ but not in $S_y^\perp$ are random sparse points hidden from the adversary. Thus an adversary that finds a collision given the more informative $\mathcal{D}$, will also find a collision given $\mathcal{D}'$.

In [Zha19], this technique was employed to reduce the security of the quantum money scheme of [AC12] to an information-theoretic statement. Crucially in that proof, *both* the primal S and dual $S^\perp$ were bloated, which then implies that the original space S is information-theoretically hidden. In our case, however, the original subspaces S_y are still part of the oracles $\mathcal{P}, \mathcal{P}^{-1}$, and it is unclear how to bloat them. This key difference is that in [Zha19], there was only a single sparse space S , and it could be bloated by consuming some of the abundant “free” ambient space. However, in our case, the entire domain is partitioned into sets $H^{-1}(y)$, and it seems that we would need to bloat each of them. But this is impossible, since there is no “free” space left as each point is already part of some subspace. Nevertheless, in the following, we will see that the dual-bloating is still useful.

Step 2: Simulating the Dual. We will now completely eliminate the dual oracle $\mathcal{D}'$. To do so, we use a version of the random self-reduction described above, but starting from a smaller instance $(\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}, \overline{\mathcal{D}})$. We will show that embedding the smaller instance lets us (without querying the

smaller instance) know some information about the subspaces $S_y^\perp$, namely a random super-space $T_y^\perp$ of $S_y^\perp$. This allows us to simulate $\mathcal{D}'$ *without querying $\overline{\mathcal{D}}$ at all*, and in turn to eliminate the dual oracle $\overline{\mathcal{D}}$ entirely.

In more detail, let $s = \log_2(|T_y^\perp|/|S_y^\perp|)$, which is how much larger (in terms of dimension) $T_y^\perp$ is from $S_y^\perp$. Let $\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}$ be an instance of our oracle, but with input space $\{0, 1\}^{n-(n-r-s)} = \{0, 1\}^{r+s}$ and output space $\{0, 1\}^r \otimes \{0, 1\}^{k-(n-r-s)}$. In other words, we shrink the input x and the vector part of the output $\mathbf{u}$ each by $n - r - s$ bits, but we keep the y part of the output (the actual hash function output) the same length. We will not have access to the oracle $\overline{\mathcal{D}}$.

We simulate $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'$ using $\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}$. However, since $\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}$ is a smaller instance, we first expand it into a full-sized instance (i.e., without considering how our mapping distributes). Essentially, we just pass the first $r + s$ bits of x into $\overline{\mathcal{P}}$, and the last $n - r - s$ bits we output in the clear as part of $\mathbf{u}$. In more detail:

- $\mathcal{P}(x)$: Break the input $x \in \{0, 1\}^n$ into two parts: $x := (\bar{x} \in \mathbb{Z}_2^{r+s}, \tilde{x} \in \mathbb{Z}_2^{n-r-s})$ and we moreover interpret $\tilde{x}$ as a vector of dimension $n - r - s$. Now query $(y, \bar{\mathbf{u}}) \leftarrow \overline{\mathcal{P}}(\bar{x})$, and set $\mathbf{u} = (\bar{\mathbf{u}}, \tilde{x})$. Output $(y, \mathbf{u})$.
- $\mathcal{P}^{-1}(y, \mathbf{u})$: Write $\mathbf{u} = (\bar{\mathbf{u}} \in \mathbb{Z}_2^{k-(n-r-s)}, \tilde{x} \in \mathbb{Z}_2^{n-r-s})$, compute $\bar{x} \leftarrow \overline{\mathcal{P}}^{-1}(y, \bar{\mathbf{u}})$ and output $x = (\bar{x}, \tilde{x})$.
- $\mathcal{D}'(y, \mathbf{v})$: Write $\mathbf{v} = (\bar{\mathbf{v}} \in \mathbb{Z}_2^{k-(n-r-s)}, \tilde{x} \in \mathbb{Z}_2^{n-r-s})$. Output 1 if and only if $\tilde{x} = 0^{n-r-s}$.

Now this clearly does not simulate the correct distribution of oracles $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'$ since for example the last $n - r - s$ bits of input are clearly visible in the output, and $\mathcal{D}'$ checks a fixed subspace. However, it is a valid instance of the oracles $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'$, in the sense that there is *some* choice permutation Π , cosets S_y and spaces $T_y^\perp$ that gives these oracles, or in other words, we output an oracle that's inside the support of $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'$. Note that our output oracle satisfies that for every $y \in \{0, 1\}^r$ we have that $T_y^\perp$ is just the space of vectors whose last $n - r - s$ entries are 0 – we'll use this fact later. We can then apply our random self-reduction to generate a correctly distributed instance.¹³

To re-randomize $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}')$, let $\overline{H}$ the hash function in the oracles $\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}$, let H the hash function in the oracles $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}')$ and let $\tilde{H}$ the hash function in the oracles $(\tilde{\mathcal{P}}, \tilde{\mathcal{P}}^{-1}, \tilde{\mathcal{D}})$, which are generated by applying the random self reducibility on $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}')$. By the random self-reducibility, we know that any algorithm which finds collisions for $\tilde{H}$ given access to $(\tilde{\mathcal{P}}, \tilde{\mathcal{P}}^{-1}, \tilde{\mathcal{D}})$ will find collisions for the simulated H relative to $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}')$. To complete our elimination of the dual oracle, we need to show that such collisions in H actually yield collisions in $\overline{H}$. For simplicity, we will work with the non-random-self reduced oracles described above $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}')$. First, observe that if we write $x = (\bar{x} \in \mathbb{Z}_2^{r+s}, \tilde{x} \in \mathbb{Z}_2^{n-r-s})$, then $H(x) = \overline{H}(\bar{x})$. There are two ways a collision x, x' in H can occur:

- $\bar{x} \neq \bar{x}'$. In this case $\bar{x}$ and $\bar{x}'$ form a collision in $\overline{H}$, as desired.

¹³Note that in our warm-up, the random self-reduction re-randomized an instance of the actual construction $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D})$, but we are now re-randomizing an instance $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}')$, when the dual is bloated. The same random self-reduction works just as well in this setting.

- $\bar{x} = \bar{x}'$. We will call these “bad” collisions, which we will now handle. Observe that in these cases, since $x \neq x'$, we must have $\tilde{x} \neq \tilde{x}'$. Also, since $\bar{x} = \bar{x}'$, we have that $\bar{\mathbf{u}} = \bar{\mathbf{u}}'$. But letting $\mathbf{u} = (\bar{\mathbf{u}} \in \mathbb{Z}_2^{k-(n-r-s)}, \tilde{x} \in \mathbb{Z}_2^{n-r-s})$ and $\mathbf{u}' = (\bar{\mathbf{u}} \in \mathbb{Z}_2^{k-(n-r-s)}, \tilde{x}' \in \mathbb{Z}_2^{n-r-s})$, this means that $\mathbf{u} - \mathbf{u}'$ is a non-zero vector whose first $k - (n - r - s)$ entries are 0.

In that case, let T_y be the linear space that is dual to $T_y^\perp$. Recall that in the non-re-randomized case, $T_y^\perp$ is the space of vectors whose last $n - r - s$ entries are 0; this means that T_y is the space of vectors whose *first* $k - (n - r - s)$ entries are 0. Thus, “bad” collisions give $(\mathbf{u} - \mathbf{u}') \in T_y$. This property is moreover preserved by applying the random self-reduction. In contrast, a general collision will have $\mathbf{u} - \mathbf{u}'$ in a much larger space, namely S_y , the analogous linear space dual to $S_y^\perp$.¹⁴

Thus, we see that bad collisions cause $\mathbf{u} - \mathbf{u}'$ to concentrate in a much smaller space than general collisions. Following a technique introduced in [Shm22b] (which we necessarily make optimal, in Lemmas 23 and 27), we will use this fact to show that an algorithm which produces bad collisions actually distinguishes the bloated from un-bloated cases, contradicting our earlier proof of the indistinguishability of these two cases.

In slightly more detail, this is because we can run an adversary several times (using a separate random self-reduction each time) to collect several independent vectors $\mathbf{u} - \mathbf{u}'$. If the vectors are always concentrated in T_y , the span of them will be contained in T_y and therefore have dimension $n - r - s$. On the other hand, if we do the same for the original un-bloated oracles, we argue (again using self-reducibility) that the vectors $\mathbf{u} - \mathbf{u}'$ will be random in S_y and hence with enough of them we span the whole larger space of dimension $n - r$. Looking at the dimension of the spanned space therefore distinguishes the original and bloated duals, which we already showed was impossible. Thus, the original algorithm must at least *occasionally* output collisions that are not bad.

Step 3: Reducing to a Coset-Partition Function. We have now reduced our problem to proving the collision resistance of H when only given the oracles $\mathcal{P}, \mathcal{P}^{-1}$ but where the adversary does not have access to the oracle $\mathcal{D}$. We now prove such collision resistance, assuming the collision-resistance of some coset-partition function Q . This may seem counter-productive; after all, we deliberately moved away from the coset-partition structure of [AGKZ20]. Looking ahead, we will see that, since we have now stripped away the dual oracle, we actually *can* analyze the coset-partition structure.

We explain that given a coset-partition function $Q : \{0, 1\}^n \rightarrow \{0, 1\}^r$, we can readily construct an instance of $\mathcal{P}, \mathcal{P}^{-1}$: $\mathcal{P}(x)$ sets $y = Q(x)$, and $\mathbf{u} = (x, 0^{k-n})$. This also makes $\mathcal{P}^{-1}(y, (x, 0^{k-n}))$ trivial as x is already present in the clear; we just need to verify that $Q(x) = y$ before outputting x (and outputting $\perp$ otherwise). This gives a valid instance of the oracles $\mathcal{P}, \mathcal{P}^{-1}$ since the pre-image sets of Q are already cosets, so outputting x padded with 0’s satisfies the structure of $\mathcal{P}$. The oracles $\mathcal{P}, \mathcal{P}^{-1}$ clearly do not have the correct distribution, but we can once more apply the random self-reduction to simulate a proper random instance of $\mathcal{P}, \mathcal{P}^{-1}$.

Importantly, observe that the reduction which simulate $\mathcal{P}, \mathcal{P}^{-1}$ only require access to Q *in the forward direction*. This means we have reduced our problem to the collision resistance of Q , given only forward queries to Q .

¹⁴ S_y contains, and is s dimensions larger than T_y , since $S_y^\perp$ is contained, and is s dimensions smaller than $T_y^\perp$.

Step 4: Constructing Hard Coset-Partition Functions. It may appear that we are still stuck: while we have eliminated the dual and inverse oracles, we have re-introduced the extra structure of a coset partition function. Thankfully, our reduction from the previous Step 3 actually shows that *any* coset partition function Q can be used to simulate the oracles $\mathcal{P}, \mathcal{P}^{-1}$, as long as Q maps from n to r bits and each preimage set, which is a coset, has size 2^{n-r} . This means that finding any distribution over such Q that is provably collision resistant, will constitute the collision resistance of our construction.

We now explain a simple construction of such a function Q . We start by observing that a 2-to-1 function is trivially a coset partition function. This is because any pre-image set, which is 2 points x, x' , is automatically a coset of the 1-dimensional subspace $S := \{0^n, x \oplus x'\} \subseteq \mathbb{Z}_2^n$ with a constant shift of x (or equivalently, a constant shift of x'). Also, the known quantum collision lower-bounds [AS04, Zha15] imply that a random 2-to-1 function is provably collision resistant. Now, just any random 2-to-1 function is not enough for us, but the following can be done.

We observe that a straightforward combination of several existing results proves that a random 2-to-1 function, with the added property that it is shrinking by 1 bit, is also collision resistant. Given such shrinking ℓ i.i.d. random 2-to-1 functions $H_1 : \{0, 1\}^{n'} \rightarrow \{0, 1\}^{n'-1}, \dots, H_\ell : \{0, 1\}^{n'} \rightarrow \{0, 1\}^{n'-1}$, take $Q : \{0, 1\}^{n' \cdot \ell} \rightarrow \{0, 1\}^{n' \cdot \ell - \ell}$ to be the ℓ -wise parallel application of them. The pre-image sets are then the direct sums of ℓ pre-image sets of the underlying 2-to-1 functions, and direct sums of cosets are cosets. Parallel application also preserves collision resistance. Putting everything together we set $n := n' \cdot \ell$, $r := n' \cdot \ell - \ell$, which proves the oracle-security of our construction, thereby proving Theorems 1 and 2.

2.3 Obfuscating PRPs (Section 5)

Our next goal is to turn our oracle proof into a standard-model proof. The natural approach is, instead of providing oracles for the various functions $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}$, to provide obfuscations of these functions. To make the functions efficient, we will replace the random permutation Π with a pseudorandom permutation, and the random choices of $\mathbf{A}_y, \mathbf{b}_y$ with values generated by a pseudorandom function.

But we immediately run into a problem: There are simply no known techniques for proving the security of obfuscated pseudorandom permutations when using the standard notion of indistinguishability obfuscation (iO). This is because iO only provides a seemingly very weak guarantee: Functionally-equivalent programs are indistinguishable when obfuscated. In order to use iO, some of the program transformations actually need happen outside of the iO, which in turn requires other cryptographic techniques. Most of the iO literature follows the punctured programming approach [SW14], which uses the notion of a “punctured PRF” [KPTZ13, BW13, BGI14]. These, very roughly, allow for giving out a program that either computes the PRF correctly, *or* the program computes the PRF everywhere but a single (known) point, and for that one point the output is uniformly random. Security says that these two programs are indistinguishable.

Permutable PRPs. Unfortunately, pseudorandom permutations provably cannot be punctured in the same sense as PRFs, as explained by [BKW17]. Roughly, the reason is that replacing the output at a single point with a uniform random value means the function is no longer a permutation.

We instead define the notion of a *permutable* PRP. Here, given $k \in \{0, 1\}^\lambda$ a PRP secret key and some fixed permutation Γ on $\{0, 1\}^n$, it is possible to produce a circuit which computes either (1) $\Pi(k, \cdot)$ and its inverse correctly, or (2) $\Gamma(\Pi(k, \cdot))$ and its inverse. Security requires that

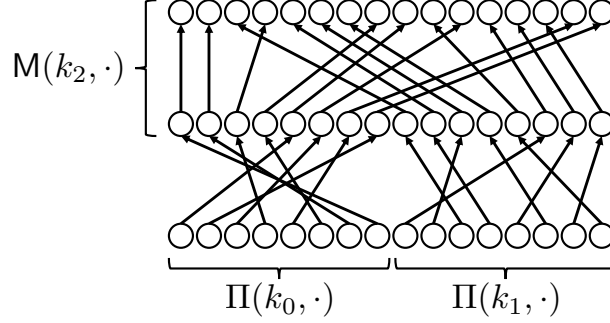


Figure 1: The PRP construction $\Pi(k, \cdot)$ of [GP07]. There are 2^n input nodes on the bottom, one per input, and 2^n output nodes. The left and right halves correspond to inputs starting with 0 and 1 respectively. Here, k_0, k_1, k_2 are keys pseudorandomly derived from k . Then $M(k_2, \cdot)$ is a pseudorandom object we call a Merge which preserves the order of each half but otherwise pseudorandomly scrambles them. $\Pi(k_0, \cdot)$ and $\Pi(k_1, \cdot)$ are then recursive calls to the construction on inputs of length $n - 1$.

(1) and (2) are indistinguishable, even if Γ is known. This avoids the impossibility of [BKW17] since we always maintain that the program being computed is a permutation. When used in iO proofs, permutable PRPs readily give that $\text{iO}(\Pi(k, \cdot))$ is computationally indistinguishable from $\text{iO}(\Gamma(\Pi(k, \cdot)))$. This even holds true if given obfuscated programs for the inverses, and even for more complicated programs that may query the permutations several times. Observe that a similar statement for puncturable pseudorandom functions readily follows from [CLTV15], but that case inherently has no inverse.

Constructing Permutable PRPs for Neighbor Swaps. For now we will focus on an extremely simple setting, where we restrict Γ to be a swap between $z \in \{0, 1\}^n$ and $(z + 1) \in \{0, 1\}^n$ for some value z , but leave all other points unaffected. We will call such Γ a *neighbor swap*.

Even for the simple case of a neighbor swap, a permutable PRP is non-trivial. An “obvious” choice is to take a PRP built from a pseudorandom *function* (PRF), and instantiate the PRF with a puncturable PRF. But it is not at all clear a priori how puncturing the underlying PRF allows for swapping outputs.

In fact, this strategy *cannot* work in general. For example, we argue that it cannot work for Luby-Rackoff PRPs. To see this, we observe that in the setting where the domain size is polynomial, a neighbor-swap PRP actually is also a standard PRP. This is because the neighbor-swap property ensures that you can computationally undetectably swap z and $z+1$ in the output truth table. Since any permutation on a polynomial-domain can be decomposed into a polynomial-length sequence of neighbor swaps, this says that you can apply a *random* permutation un-detectably. But composing with a random permutation actually gives a truly random permutation, thus showing that the truth table of the PRP is computationally indistinguishable from the truth table of a random permutation. We can also scale this argument up, and show that if the neighbor-swap PRP is sub-exponentially secure, then it must be secure against a “truth-table” adversary that is provided the entire (exponential-sized) truth table. We then recall that standard PRP constructions such as Luby-Rackoff *cannot* achieve security in the small-domain/ truth-table setting [Pat01].

Guided by the above discussion, we look to the literature on small-domain permutations, specifi-

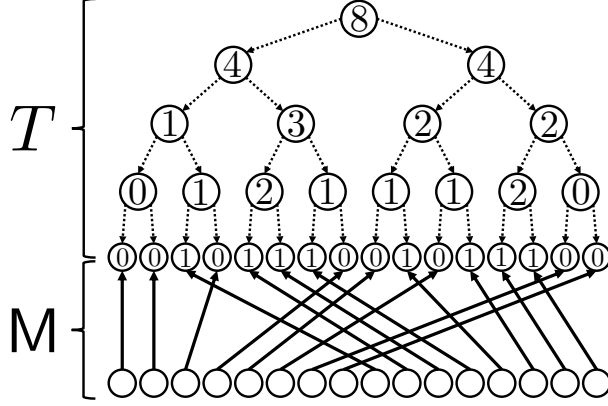


Figure 2: A merge M and the associated tally tree T , for the case $N_0 = N_1 = 8$, $N = 16$.

cally the work of [GP07]. While [GP07] present their construction in a very procedural way involving “permutators”, “splitters” and “repartitors”, we will present the idea in a more conceptual way. We will then explain how our permuting algorithm executes neighbor swaps efficiently for any (possibly exponential) domain size. The first observation is that one can construct a random permutation as follows: Divide into two equal sized piles (those strings starting with 0 and 1, respectively), and then recursively, randomly permute each pile independently. Then randomly merge the two piles together; this step preserves the order in each pile (which were already shuffled, so this is fine), but randomly determines how the two piles are interleaved. It is not hard to see that every permutation uniquely corresponds to a triple containing a merge (determining the interleaving) and two recursive permutations. Thus this process perfectly simulates a random permutation. See Figure 1.

It turns out that permutations structured in this way can be evaluated efficiently, even for an exponential-size domain. To evaluate on a point, we only need to efficiently evaluate *one* of the recursive permutations for the pile it belongs to, and then the merge operation (which we simply call a Merge). As there will be n levels of the recursion and each level only makes a single recursive call, this means that we can evaluate the overall permutation using n Merge evaluations. We show how to compute the Merge operation efficiently using a random data-structure we call a tally tree. A tally tree is a full binary tree on 2^n nodes, where leaf z corresponds to an *output* of the Merge, and is given the value of the first bit of the pre-image of z . Then the internal nodes count the total number of 1’s in the leaves of the sub-tree rooted at that node. An example tally tree T for a merge M is given in Figure 2.

Using a tally tree we can efficiently evaluate the Merge. We start by describing the inverse: since the Merge preserves the order within each half, we can compute the inverse of z just knowing two pieces of information: (1) which half the pre-image of z belongs to, and (2) how many nodes to the left of z go to the same half. This can be computed easily using the tally tree. Then the forward direction can be computed using a binary search algorithm which makes queries to the inverse and again exploits the order-preserving property.

The next observation is, rather than choosing a random merge and computing the tree in a bottom-up manner, we can actually sample the tree in a top-down manner. In this view, the value in each node follows an appropriate hypergeometric distribution based on the value of its parent.

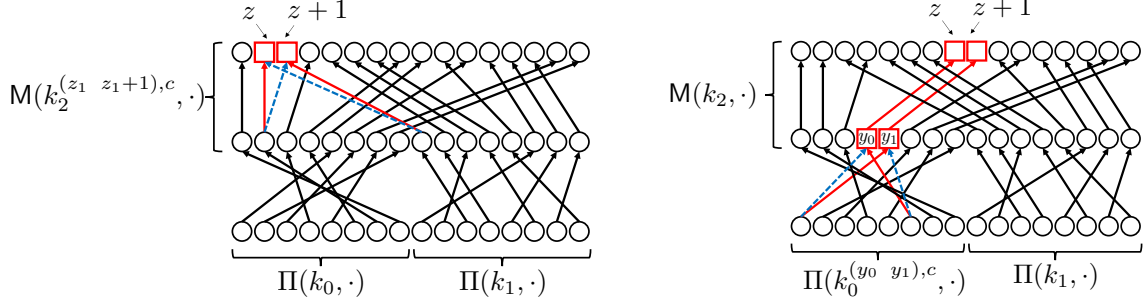


Figure 3: Our permuting algorithm. Here, $k^{(z, z+1), c}$ means the permuted key, where $c = 0$ means no swapping occurs, and $c = 1$ means z and $z+1$ are swapped. Red squares indicate the points $z, z+1$, red solid arrows indicate the original permutation ($c = 0$), and blue dashed lines indicate the permuted permutation ($c = 1$). Let y_0, y_1 be the pre-images of $z, z+1$ in the Merge. Left: The case where y_0, y_1 lie in different halves, and we permute $z, z+1$ by permuting the merge M . Right: The case where y_0, y_1 lie in the same half, and we permute $z, z+1$ by permuting the recursive application of Π , as indicated by the red squares.

Then the random choices can be simulated using a pseudorandom function (PRF). The values in the tree are left implicitly determined by the PRF key, to be computed on the fly as needed during evaluation (which only visits a polynomial-sized portion of the tree since evaluation is efficient). The result is that keys can be small (independent of the domain size), and evaluation takes time poly-logarithmic in the domain size.

We start with this construction, and first show that we can permute z and $z+1$ by either permuting the Merge or by permuting one of the recursive calls to Π . In particular, if the pre-images of z and $z+1$ lie in different halves of the domain, then we can permute within the Merge. If the pre-images lie in the same half, then we cannot permute the Merge since this would violate the ordering of elements in that half. In this case, however, the order-preserving property of the Merge guarantees that the pre-images of $z, z+1$ are adjacent, and so we can instead permute within the recursive Π call for that half. See Figure 3.

Thus, we have reduced the task of permuting Π to permuting the Merge, and we only need to concern ourselves with the case where the pre-images of $z, z+1$ lie in different halves. If we look at the leaf nodes $z, z+1$ in the tally tree, since they have pre-images in different halves, this means one of the nodes has a value of 0 and the other a value of 1. Moreover, permuting $z, z+1$ corresponds to exchanging which is a 0 and which is a 1.

What we show is that such permuting can be accomplished by puncturing the underlying pseudorandom function (PRF) that is used to generate the tally tree. We use a puncturable PRF that can be punctured at several points, resulting in those points being replaced with random values. Concretely, we puncture at the nodes $z, z+1$, as well as all nodes on the paths from these nodes to the root, and also the siblings of those nodes. By analyzing the induced hypergeometric distributions, we conclude after puncturing that the values at $z, z+1$ are actually statistically equally likely to be 0 or 1. Thus, we can swap their values without detection. See Figure 4 for an example.

Remark 12. [GP07] is not particularly efficient from a practical perspective, due to having to

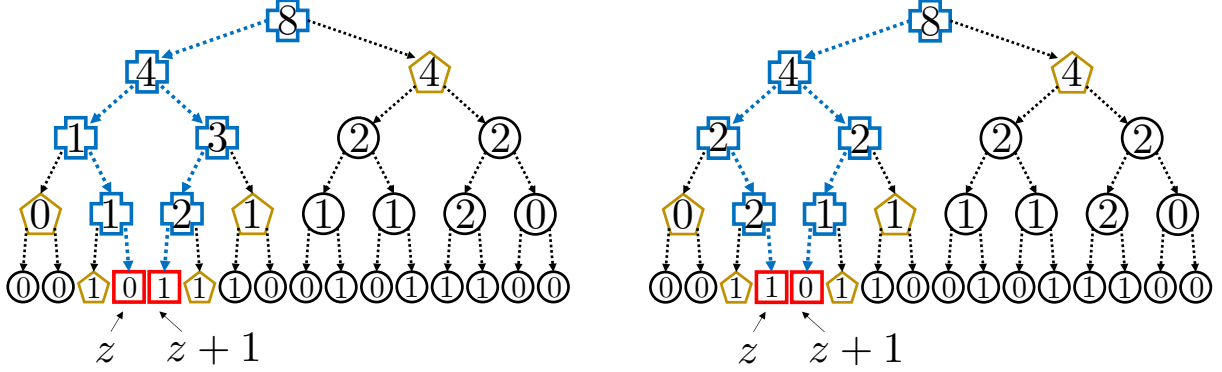


Figure 4: Left: The tally tree for a permuted merge key $k^{(z, z+1), 0}$, where no swapping happens. The red squares are the positions to be swapped $z, z+1$. The blue crosses are the paths from root to these nodes and the gold pentagons are the siblings of these paths; these nodes have all been punctured. Right: The tally tree for the permuted merge key $k^{(z, z+1), 1}$. This is identical to the left tree, except that we swapped the bits at z and $z+1$, and accordingly updated the tallies in the paths to $z, z+1$ (the blue crosses). The gold pentagons and all their descendants remain unchanged.

sample from hypergeometric distributions. Other works have given more efficient constructions of small-domain PRPs [Mor05, HMR12, RY13, MR14], but it is unclear if these also give permutable PRPs when instantiated with a puncturable PRF.

Extending to More General Permutations. Neighbor swaps are not sufficient for most applications. We therefore explain how to extend the construction to handle much more general permutations Γ .

The idea is simple, and builds upon the intuition that neighbor swaps are enough to generate all permutations. To get the circuits computing $\Pi(k, \cdot)$ or $\Gamma(\Pi(k, \cdot))$, we simply obfuscate those programs using iO. To show that the two cases are indistinguishable, we decompose Γ into a sequence of (exponentially-many) neighbor swaps, and perform a sequence of hybrids where we apply one neighbor-swap at a time.

There are two caveats to this approach. One is that we will need exponentially-many hybrids, and therefore we must rely on sub-exponentially-secure iO. The underlying puncturable PRF must also be sub-exponentially secure, which is implied by sub-exponential one-way functions.

The more challenging caveat is the following. If we decompose Γ as $\Gamma = \tau_N \circ \tau_{N-1} \circ \dots \circ \tau_1$ for neighbor-swaps τ_i , we actually require that each partial permutation $\Gamma_t := \tau_N \circ \tau_{N-1} \circ \dots \circ \tau_t$ is computable by a small circuit. These need to be small because, at the appropriate hybrid, they will be hard-coded into the program that gets obfuscated. To maintain security given the obfuscated inverse circuits, we also need each Γ_t^{-1} to have small circuits. But in general, even if Γ, Γ^{-1} have small circuits, it is not obvious that Γ can be decomposed in a way which guarantees that each Γ_t, Γ_t^{-1} have small circuit. This leads to our Efficient Permutation Decomposition (EPD) Problem (Question 8), which asks whether such a guarantee is possible in general.¹⁵

¹⁵Note that Question 8 was about decomposing into transpositions rather than neighbor swaps. But as we will see, transpositions can be decomposed into neighbor swaps, and thus the two versions are equivalent.

Transpositions $i \leftrightarrow j$.
 Linear cycles $j \rightarrow j+1 \rightarrow j+2 \rightarrow \dots \rightarrow \ell \rightarrow j$.
 Affine mod-2 permutations $\mathbf{x} \rightarrow \mathbf{A} \cdot \mathbf{x} + \mathbf{v} \bmod 2$ if $\det(\mathbf{A}) \bmod 2 = 1$.
 Products $(x, y) \rightarrow (\Gamma_0(x), \Gamma_1(x))$ for decomposable Γ_0, Γ_1 .
 Compositions $\Gamma = \Gamma_0 \circ \Gamma_1$ for decomposable Γ_0, Γ_1 .
 Controlled permutations $(x, y) \rightarrow (x, \Gamma_x(y))$ for decomposable Γ_x .
 Efficient involutions $(\Gamma \circ \Gamma = \mathbf{I})$.
 Conjugations $\Lambda^{-1} \circ \Gamma \circ \Lambda$ for decomposable Γ and efficient Λ, Λ^{-1} .
 Permutations with ancillas $(x, 0^n) \rightarrow (\Gamma(x), 0^n)$ for efficient Γ, Γ^{-1} .

Figure 5: A non-exhaustive list of decomposable permutations. Note that for conjugations, Λ need not be decomposable. For ancillas, $\Gamma : \{0, 1\}^n \rightarrow \{0, 1\}^n$ need not be decomposable, but the permutation $(x, 0^n) \rightarrow (\Gamma(x), 0^n)$ is only a partial function that is not specified on the rest of the domain. The permutable PRP domain in this case is the entire space including ancillas.

We do not know how to resolve the EPD problem in general. Instead, we define a notion of “decomposable” circuits, which are Γ that can be decomposed into a (potentially exponentially-long) sequence in such a way that the Γ_t, Γ_t^{-1} all have small circuits.¹⁶ Fortunately, decomposable circuits capture many natural types of permutations, such as general (non-neighbor) transpositions, linear cycles $j \mapsto j+1 \mapsto \dots \mapsto \ell-1 \mapsto \ell \mapsto j$, involutions (i.e., such that $\Gamma \circ \Gamma$ is the identity), and more. See Figure 5 for a longer list of examples of decomposable permutations, which are shown to be decomposable in Section 5. Our proof shows that an obfuscated neighbor-swap PRP is a permutable PRP for all decomposable circuits. This gives the informal Theorem 5.

Remark 13. *Interestingly, despite showing that a wide range of permutations are decomposable, even for very simple permutations, decompositions can be tricky. For example, $x \mapsto 2x \bmod N$ for odd N is decomposable, though our proof uses the fact that the map is a merge with an efficiently computable tally tree, and shows that such tally trees allow for decomposition. This generalizes to $x \mapsto ax \bmod N$ for polynomial a . But the only way we know how to handle general linear maps requires decomposing a into the product (mod N) of polynomially-bounded scalars. This in turn requires that such polynomial scalars generate $\mathbb{Z}_N^*$. This is implied by the Extended Riemann Hypothesis [Bac90]. However, we leave a general unconditional decomposition, even for scalar multiplication, as an intriguing open question.*

Applications. We show that permutable PRPs can be very useful in obfuscation applications.¹⁷ Concretely, we show that obfuscating a permutable PRP for general transpositions (which in particular are decomposable) immediately gives a *full-domain* trapdoor one-way permutation, yielding Theorem 6. The proof goes through several hybrids:

¹⁶Note that there may be many possible decompositions into neighbor-swaps, and some decompositions may give small circuits while others may not.

¹⁷As we describe it, our applications will obfuscate a permutable PRP, where our permutable PRPs are built using obfuscation. We can equivalently simply obfuscate the underlying neighbor swap PRP, and all the hybrids in the proof of security for the permutable PRP can instead be carried out at the level of the application. However, for conceptual simplicity, we believe it is beneficial to describe our results using the permutable PRP abstraction.

- Hybrid 0: The adversary is given an obfuscation of $\Pi(k, \cdot)$ and a challenge point y^* , and has to find x^* such that $y^* = \Pi(k, x^*)$.
- Hybrid 1: We choose a random output y' , and “puncture” the program so if $\Pi(k, x) = y'$, the program outputs y^* instead of y' . Otherwise the program is unchanged. This means there are now two points x^*, x' that map to y^* . This change follows standard iO techniques.
- Hybrid 2: We use permuted key $k^{(x^* \ x'),0} \leftarrow \text{Permute}(k, (x^* \ x'), 0)$ but where the transposition $(x^* \ x')$ is still turned off. Recall per Figure 5 that transpositions are decomposable. We use the permuted key instead of the real key k for all purposes of this hybrid. Since these are functionally equivalent, by the security of the iO, the transition to this hybrid is computationally undetectable.
- Hybrid 3: Now we compose the PRP with the transposition swapping x^*, x' . That is, we sample $k^{(x^* \ x'),1} \leftarrow \text{Permute}(k, (x^* \ x'), 1)$ and use it instead of $k^{(x^* \ x'),0}$, which was used in the previous hybrid. By the security of the permutable PRP this change is computationally indistinguishable. Note that, importantly we transpose x^* and x' both within the obfuscated program and also to verify the adversary’s output (where we check that the circuit’s output equals y^*). This swap means that now the adversary has to find x' instead of x^* . That is, after we move to this hybrid, we can make another change to how we check the success of this hybrid: Take the original PRP key k and simply check that $y' = \Pi(k, x)$.
- Hybrid 4: Now we move to using the original key k inside the obfuscated circuit instead of the permuted key $k^{(x^* \ x'),1}$. Note that the functionality of the outside circuit (which is also obfuscated by iO) is unchanged because it is still the case that on both, x^*, x' it outputs y^* . This change is computationally indistinguishable by the security of the iO.

Finalizing the reduction. In the setting of the last hybrid experiment, we sample a random PRP key k and a uniformly random y' . We then obfuscate the circuit P_1 , which computes the PRP with key k , checks if the output was y' , and if so outputs y^* instead of y' (or in other words, does something unrelated to the original problem). The adversary then finds x' which eventually leads to finding y' . By standard iO techniques, finding such uniformly random y' is computationally intractable, even given the key k , the value y^* and the obfuscation of the program P_1 .

An elaborated formal proof is found in Section 5 (Theorem 44).

2.4 OSS In the Standard Model (Section 6)

We finally turn to constructing OSS in the standard model. The idea is to simply replace the random permutation Π with a permutable PRP $\Pi(k, \cdot)$ and to have the matrix/vector pairs $\mathbf{A}_y, \mathbf{b}_y$ be generated pseudorandomly from a (puncturable) pseudorandom function. Fortunately, the preceding sections have already set up most of the ideas we need for the proof, as our sequence of reductions for proving our oracle construction actually have natural cryptographic analogs.

For our random self-reduction, we need to argue that the simulated obfuscated program is indistinguishable from the real distribution. In the oracle model, we have perfect indistinguishability.

In the standard model case, we no longer have perfect indistinguishability since the simulated permutation Π is now the composition of two permutable PRPs, and the matrix/vector pairs $\mathbf{A}_y, \mathbf{b}_y$ are derived from underlying terms $\overline{\mathbf{A}}_y, \overline{\mathbf{b}}_y, \mathbf{C}_y, \mathbf{d}_y$, each of which are pseudorandomly generated. Fortunately, a standard hybrid over every y , utilizing punctured PRF security lets us move to the correct distribution over $\mathbf{A}_y, \mathbf{b}_y$, and permutable PRP security lets us replace the composition of two permutable PRPs with a single PRP.

For dual-bloating, we follow the approach of [Zha19], which shows how to increase the size of obfuscated subspaces using iO and one-way functions. We can likewise eliminate the dual as in the oracle case, as our argument works just as well in the standard model. Finally, we can embed a coset-partition function just as we did in the oracle case, but now using our computational random self-reduction.

There are, however, two main hurdles to getting everything to work. The first is that, in order to show that the obfuscation of the composition of two permutable PRPs is indistinguishable from a single permutable PRP, we need our permutable PRPs itself to be decomposable. In Theorem 41 we construct permutable PRPs by our permuting algorithm, while carefully also maintaining their decomposability. This allows for overall composition between permutable PRPs.

Another caveat is that we need a post-quantum collision-resistant 2-to-1 function L , since we can no longer rely on the unconditional existence relative to an oracle. It turns out that we also need this L to have some extra properties, that were not present in the oracle case. Specifically, when we showed that embedding Q (the parallel repetition of L) results in a valid instance of the oracles $\mathcal{P}, \mathcal{P}^{-1}$, we actually argued that this means there *exists* an implicit permutation Π determined by Q . Very roughly, since Q has collisions it therefore loses information, and Π outputs Q plus some additional bits to recover the lost information. For the oracle proof, we only needed the existence of Π to argue that the simulated oracles had the same distribution as the real oracles. But for our standard-model argument, we will actually have a hybrid where Π is hard-coded into the obfuscated program. That means we need Π to have a small circuit. Unfortunately, Π essentially requires inverting Q , and so in general is computationally inefficient. Note that the inefficient circuit is not needed in normal usage, but just in the proof.

As a result, we need the function Q to have a number of specific properties, which mainly boil down to having L a collision-resistant 2-to-1 function that has a trapdoor. Fortunately, we can construct such “trapdoor” 2-to-1 hash functions from lattices, following a similar approach as the claw-free trapdoor functions built from LWE [BCM⁺18].

However, focusing on the LWE-based hash functions leads into the third and fourth caveats. For the third: The range of the LWE-based hash function is actually larger than the domain. In contrast, for our oracle construction described above, the function Q needs to be surjective in order for filling in the lost information to actually yield a permutation. But this means the range of Q must be smaller than the domain since it is many-to-one. This also presents a problem for invoking permutable PRP security, since permutable PRPs allow for composing with *permutations*, but our LWE-based hash function is no longer a permutation.

Fortunately, we can expand the range by padding y with 0’s, and composing the output of the construction with *another* permutable PRP. This leads to our ultimate construction, which we describe visually in Figure 6. We can also split up the evaluation of Q into multiple stages, where each stage is a (decomposable) permutation which allows us to invoke permutable PRP security.

The fourth caveat is that the LWE-based hash function L is really only 2-to-1 on an overwhelming fraction of the domain, but it is easy to devise points where the function is only 1-to-1 on those

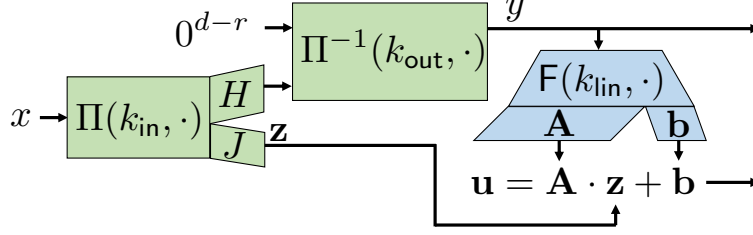


Figure 6: The program P that gets obfuscated to give the program $\mathcal{P}$. The programs P^{-1}, D are defined accordingly. Here, Π, Π^{-1} is a permutable PRP, and F is a puncturable pseudorandom function. The output of H is r bits, $\mathbf{z}$ is $n - r$ bits, and y is d bits.

points. Intuitively, this makes the parallel repetition Q of L be, accordingly, only an *approximate* coset partition function (i.e., it is not the case that for every output point of Q , the preimage set is a coset of the same dimensions). This breaks the straightforward iO proof: When we embed Q into the programs, the programs will actually not have the same functionality due to the 1-to-1 points. This prevents us from naively using iO, which requires functionally equivalent programs.

To account for this, we add a trigger at all the sparse “bad” 1-to-1 points, and if this trigger occurs, the program behaves differently. We could try to embed this trigger in the original construction, but it is not clear if we would be able to carry out all of our re-randomization steps; even if it could work, it would significantly complicate each of the steps. Instead, we add the trigger only in the proof. Standard iO techniques allow for adding sparse triggers on *random* points that are otherwise independent of the program, but not fixed triggers like we need.¹⁸ Here, we use our permutable PRPs again. We show that, as long as the fixed trigger is sandwiched between two permutable PRPs (which they are, for our case, because we are already composing another PRP on the output) and meets some other technical conditions, we can un-detectably add the trigger. See Lemma 45 in Section 5 for a precise statement, and Figure 7 for a visual representation of the kinds of program changes which we can make.

Our proof of the sparse trigger works as follows. To trigger at a single *fixed* point y , we actually first add a trigger at a *random* point y' . Adding random triggers follows from standard iO techniques. Then, we use the permutable PRP property to exchange y and y' , meaning now the trigger occurs at y . This step requires a permutable PRP both before and after the trigger, since after the trigger, we need to return y and y' to their original values. With a bit more work, we can extend this to interval triggers, as long as the interval is at most a sub-exponential fraction of the domain.

Once we add the trigger, then embedding Q actually yields an equivalent program and allows the proof to go through. By piecing all of the steps together, we obtain Theorems 3 and 4.

Remark 14. One may wonder if there are other instantiations of the needed 2-to-1 collision-resistant hash function. Aside from needing the structure of being 2-to-1, another requirement that limits instantiations is the apparent need for a trapdoor. This seems to prevent us from using hash functions based on LPN [BLVW19, YZW⁺19] or super-singular isogeny graphs [CLG09]. One can always make a somewhat tautological assumption that obfuscating the function which discards the last bit of a (permutable) PRP gives such a function (the trapdoor being the un-obfuscated key),

¹⁸Observe that the “bad” points are fixed once the hash function is chosen.

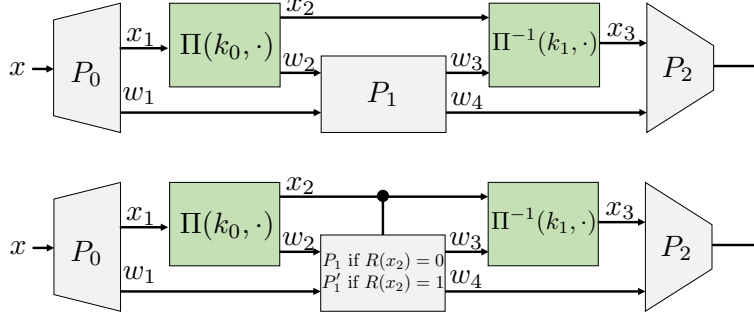


Figure 7: The original program P (top) and triggered program P' (bottom). Here, P_0, P_1, P'_1, P_2 are arbitrary programs as long as they are independent of the keys k_0, k_1 . Our indistinguishability result shows that for the case where $R(x_2)$ tests if x_2 (or some component thereof) is in a fixed sparse interval, then obfuscating these two programs yields computationally indistinguishable programs. Note that R is fixed and known, but due to the application of the permutable PRPs, the points that hit the trigger are computationally hidden once obfuscated.

though we do not know how to prove this based on any standard assumption. Another possibility is to look at constructions from group actions such as [AMR22], which can plausibly be post-quantum instantiated using ordinary isogenies over elliptic curves. Their construction has a trapdoor and can be split up into decomposable parts analogous to the LWE-based construction, and has most of the domain being 2-to-1. However, a problem is that a non-negligible fraction of the domain is 1-to-1. This breaks the step where we embed a trigger into the obfuscated program, since the trigger is no longer sparse. An interesting open question is whether their construction can be modified so that an exponentially-small fraction of the domain is 1-to-1.

3 Cryptographic Tools

We present the general cryptographic tools and techniques which are used in this work. Some of the results in this section are new and may be of independent interest.

Cryptographic Preliminaries. We use the following known primitives and notions. Both are implicitly classical primitives with security holding against quantum algorithms.

Definition 15 (Puncturable PRFs). *A puncturable pseudorandom function (P-PRF) is a pair of efficient algorithms $(F, \text{Punc}, \text{Eval})$ with associated output-length function $m(\lambda)$ such that:*

- $F : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \{0, 1\}^{m(\lambda)}$ is a deterministic polynomial-time algorithm.
- $\text{Punc}(k, S)$ is a probabilistic polynomial-time algorithm which takes as input a key $k \in \{0, 1\}^\lambda$ and a set of points $S \subseteq \{0, 1\}^*$. It outputs a punctured key k^S .
- $\text{Eval}(k^S, x)$ is a deterministic polynomial-time algorithm.
- **Correctness:** For any $\lambda \in \mathbb{N}$, $S \subseteq \{0, 1\}^*$, $k \in \{0, 1\}^\lambda$, $x \notin S$, and k^S in the support of $\text{Punc}(k, S)$, we have that $\text{Eval}(k^S, x) = F(k, x)$.

- **Security:** For any quantum polynomial-time algorithm $\mathcal{A}$, there exists a negligible function ϵ such that the following experiment with $\mathcal{A}$ outputs 1 with probability at most $\frac{1}{2} + \epsilon(\lambda)$:
 - $\mathcal{A}(1^\lambda)$ produces a set $S \subseteq \{0, 1\}^*$.
 - The experiment chooses a random $k \leftarrow \{0, 1\}^\lambda$ and computes $k^S \leftarrow \text{Punc}(k, S)$. For each $x \in S$, it also sets $y_x^0 := F(k, x)$ and samples $y_x^1 \leftarrow \{0, 1\}^{m(\lambda)}$ uniformly at random. Then it chooses a random bit b . It finally gives $k^S, \{(x, y_x^b)\}_{x \in S}$ to $\mathcal{A}$.
 - $\mathcal{A}$ outputs a guess b' for b . The experiment outputs 1 if $b' = b$.

Different security levels. For arbitrary functions $f_0, f_1 : \mathbb{N} \rightarrow \mathbb{N}$, we say that the P-PRF is $(f_0, \frac{1}{f_1})$ -secure, if in the above security part of the definition, we ask that the indistinguishability holds for every adversary of size $\leq f_0(\lambda)$ and we swap $\epsilon(\lambda)$ with $\frac{1}{f_1(\lambda)}$. Concretely, a sub-exponentially secure P-PRF scheme would be one such that there exists a positive real constant $c > 0$ such that the scheme is $(2^{\lambda^c}, \frac{1}{2^{\lambda^c}})$ -secure.

Definition 16 (Indistinguishability Obfuscation (iO)). *An indistinguishability obfuscator (iO) for Boolean circuits is a probabilistic polynomial-time algorithm $\text{iO}(\cdot, \cdot, \cdot)$ with the following properties:*

- **Correctness:** For all $\lambda, s \in \mathbb{N}$, Boolean circuits C of size at most s , and all inputs x to C ,

$$\Pr \left[\text{O}_C(x) = C(x) : \text{O}_C \leftarrow \text{iO} \left(1^\lambda, 1^s, C \right) \right] = 1 .$$

- **Security:** For every polynomial $\text{poly}(\cdot)$ there exists a negligible function ϵ such that the following holds. Let $\lambda, s \in \mathbb{N}$, and let C_0, C_1 two classical circuits of (1) the same functionality (i.e., for every possible input they have the same output) and (2) both have size $\leq s$.

$$\left\{ \text{O}_{C_0} : \text{O}_{C_0} \leftarrow \text{iO} \left(1^\lambda, 1^s, C_0 \right) \right\} \\ \approx_{(\text{poly}(\lambda), \epsilon(\lambda))} \left\{ \text{O}_{C_1} : \text{O}_{C_1} \leftarrow \text{iO} \left(1^\lambda, 1^s, C_1 \right) \right\} .$$

Different security levels. For arbitrary functions $f_0, f_1 : \mathbb{N} \rightarrow \mathbb{N}$, we say that an iO scheme is $(f_0, \frac{1}{f_1})$ -secure, if in the above security part of the definition, we swap poly with a specific function f_0 and the negligible function with the function $\frac{1}{f_1}$. Concretely, a sub-exponentially secure iO scheme would be one such that there exists a positive real constant $c > 0$ such that the scheme is $(2^{\lambda^c}, \frac{1}{2^{\lambda^c}})$ -secure.

Definition 17 (Lossy Functions). *A lossy function (LF) scheme consists of classical algorithms $(\text{LF.KeyGen}, \text{LF.F})$ with the following syntax.*

- $\text{pk} \leftarrow \text{LF.KeyGen}(1^\lambda, b, 1^\ell)$: a probabilistic polynomial-time algorithm that gets as input the security parameter $\lambda \in \mathbb{N}$, a bit b and a lossiness parameter $\ell \in \mathbb{N}$, $\lambda \geq \ell$. The algorithm outputs a public key.

- $y \leftarrow \text{LF.F}(\text{pk}, x)$: a deterministic polynomial-time algorithm that gets as input the security parameter $\lambda \in \mathbb{N}$, the public key pk and an input $x \in \{0, 1\}^\lambda$ and outputs a string $y \in \{0, 1\}^m$ for some $m \geq \lambda$.

The scheme satisfies the following guarantees.

- **Statistical Correctness for Injective Mode:** There exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda, \ell \in \mathbb{N}$,

$$\Pr_{\text{pk} \leftarrow \text{LF.KeyGen}(1^\lambda, 0, 1^\ell)} \left[\left| \text{Img}(\text{LF.F}(\text{pk}, \cdot)) \right| = 2^\lambda \right] \geq 1 - \text{negl}(\lambda) .$$

- **Statistical Correctness for Lossy Mode:** There exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda, \ell \in \mathbb{N}$,

$$\Pr_{\text{pk} \leftarrow \text{LF.KeyGen}(1^\lambda, 1, 1^\ell)} \left[\left| \text{Img}(\text{LF.F}(\text{pk}, \cdot)) \right| \leq 2^\ell \right] \geq 1 - \text{negl}(\lambda) .$$

- **Security:** For every polynomial $\text{poly}(\cdot)$ there exists a negligible function ϵ such that the following holds. Let $\lambda, \ell \in \mathbb{N}$, then (note that in the following computational indistinguishability, the security parameter is ℓ and not λ),

$$\left\{ \text{pk}_0 : \text{pk}_0 \leftarrow \text{LF.KeyGen}(1^\lambda, 0, \ell) \right\} \\ \approx_{(\text{poly}(\ell), \epsilon(\ell))} \left\{ \text{pk}_1 : \text{pk}_1 \leftarrow \text{LF.KeyGen}(1^\lambda, 1, \ell) \right\} .$$

Different security levels. For arbitrary functions $f_0, f_1 : \mathbb{N} \rightarrow \mathbb{N}$, we say that an LF scheme is $(f_0, \frac{1}{f_1})$ -secure, if in the above security part of the definition, we swap poly with a specific function f_0 and the negligible function with the function $\frac{1}{f_1}$. Concretely, a sub-exponentially secure LF scheme would be one such that there exists a positive real constant $c > 0$ such that the scheme is $(2^{\ell^c}, \frac{1}{2^{\ell^c}})$ -secure.

3.1 Two iO Techniques

Here, we recall two standard iO techniques that we will abstract as useful lemmas.

Sparse Random Triggers. Let P be some program and P' an arbitrary different program. Let R be a function with range $[N]$ for some N that is exponential in the security parameter. Let J_y (for ‘join’) be the following program:

$$J_y(x) = \begin{cases} P(x) & \text{if } R(x) \neq y \\ P'(x) & \text{if } R(x) = y \end{cases} .$$

Lemma 18. Suppose one-way functions exist. For sufficiently large polynomial s and for y chosen uniformly in $\{0, 1\}^\lambda$, $\text{iO}(1^\lambda, 1^s, P)$ and $\text{iO}(1^\lambda, 1^s, J_y)$ are computationally indistinguishable even given the description of P . Moreover, y is computationally unpredictable given $P, \text{iO}(1^\lambda, 1^s, J_y)$

Proof. We first prove indistinguishability through a sequence of hybrid programs:

- **Hyb₀:** The original obfuscation of P .

Here, the adversary is given $\text{iO}(1^\lambda, 1^s, P)$.

- **Hyb₁:** Adding a uniformly random trigger that applies only if it is in the image of a sparse PRG.

We assume an injective length-doubling pseudorandom generator $\text{PRG} : [N] \rightarrow [N]^2$. These can be replaced by injective one-way functions using standard techniques, which in turn follow from plain one-way functions and iO [BPW16]. Here, we choose a random $w \leftarrow [N]^2$. The adversary is given $\text{iO}(1^\lambda, 1^s, J'_w)$ where $J'_w(x) = \begin{cases} P(x) & \text{if } \text{PRG}(R(x)) \neq w \\ P'(x) & \text{if } \text{PRG}(R(x)) = w \end{cases}$. Note that since PRG is length-doubling, we have that with overwhelming probability over the choice of w , the second line of J'_w will never be triggered. Therefore, J'_w is functionally equivalent to P . Therefore, by iO security, as long as s is larger than the size of J'_w (which is larger than the size of P), hybrids 0 and 1 are indistinguishable.

- **Hyb₂:** Changing the trigger to be a random element inside the image of the PRG.

Here, we switch to $w = \text{PRG}(y)$ for a random y . Indistinguishability from Hybrid 1 follows immediately from the pseudorandomness of PRG .

- **Hyb₃:** Dropping the use of the PRG and checking its image directly.

Now the adversary is given $\text{iO}(1^\lambda, 1^s, J_y)$ for a random y . Observe that since $w = \text{PRG}(y)$ and PRG is injective, J_y and J'_w have equivalent functionalities. Therefore, by iO security, hybrids 2 and 3 are indistinguishable. This completes the proof of indistinguishability.

For the computational unpredictability of y , consider an adversary starting in hybrid 3 which outputs y with probability ϵ . Since the indistinguishability of hybrids 2 and 3 did not rely on the randomness of y , we can switch to hybrid 2 and still obtain an adversary that outputs y with probability at least $\epsilon - \text{negl}$. Now we observe that the view of the adversary only depends on $w = \text{PRG}(y)$, and in the end the adversary produces y with non-negligible probability. Thus, by a straightforward reduction to the one-wayness of PRG , we conclude that $\epsilon - \text{negl}$, and hence ϵ itself, must be negligible. $\square$

Swapping distributions. We now move to the next standard technique. Let $\{D_0^x\}_x, \{D_1^x\}_x$ be two families of distributions over the same domain $\mathcal{Y}$, which can also be thought of as deterministic functions $D_0(x; r), D_1(x; r)$ that take as input an index x and some random coins r . Let P be a program that makes queries to an oracle $O : \mathcal{X} \rightarrow \mathcal{Y}$ for some set $\mathcal{X}$. Then we have the following:

Lemma 19. *Let (F, Punc) be a (f_F, δ_F) -secure puncturable PRF and iO be a $(f_{\text{iO}}, \delta_{\text{iO}})$ -secure iO . Let $\mathcal{X}$ a finite set and let $D_0 := \{D_{0,x}\}_{x \in \mathcal{X}}, D_1 := \{D_{1,x}\}_{x \in \mathcal{X}}$ two ensembles of distributions, such that for every $x \in \mathcal{X}$, $D_{0,x}, D_{1,x}$ are (f_D, δ_D) -indistinguishable. Let $E_0(x) = D_0(x; F(k, x))$ and $E_1(x) = D_1(x; F(\bar{k}, x))$. Then for a sufficiently large polynomial s , $\text{iO}(1^\lambda, 1^s, P^{E_0})$ and $\text{iO}(1^\lambda, 1^s, P^{E_1})$ are $(\min(f_F, f_{\text{iO}}, f_D), O(|\mathcal{X}| \cdot (\delta_F + \delta_{\text{iO}} + \delta_D)))$ -computationally indistinguishable, where $k, \bar{k} \leftarrow \{0, 1\}^\lambda$ are uniformly random keys.*

Proof. We assume $\mathcal{X} = [N]$ by giving some ordering to the set. We prove security through a sequence of hybrids.

Hyb_{*i*,0}: The adversary gets $\text{iO}(1^\lambda, 1^s, P^{E_{i,0}})$, where $E_{i,0}$ has $k, \bar{k}$ hard-coded and is defined as

$$E_{i,0}(x) = \begin{cases} D_0(x; F(k, x)) & \text{if } x \geq i \\ D_1(F(x; \bar{k}, x)) & \text{if } x < i \end{cases}.$$

Hyb_{*i*,1}: The adversary gets $\text{iO}(1^\lambda, 1^s, P^{E_{i,1}})$, where to generate $E_{i,1}$, we compute $k^i \leftarrow \text{Punc}(k, i)$, $\bar{k}^i \leftarrow \text{Punc}(\bar{k}, i)$, sample $y \leftarrow D_0(i; F(k, i))$ and let $E_{i,1}(x) = \begin{cases} D_0(x; F(k, x)) & \text{if } x > i \\ y & \text{if } x = i. \text{ Observe} \\ D_1(x; F(\bar{k}, x)) & \text{if } x < i \end{cases}$

that by our choice of y , $E_{i,1}$ is identical to $E_{i,0}$, and hence the programs $P^{E_{i,0}}$ and $P^{E_{i,1}}$ are equivalent. Thus, by the security of iO , Hybrids $i.0$ and $i.1$ are indistinguishable except with probability δ_{iO} .

Hyb_{*i*,2}: Here, we still obfuscate $P^{E_{i,1}}$, but instead switch to $y \leftarrow D_0(i; r)$ for fresh random coins r . Observe that the entire experiment except for y is simulatable using just the punctured key k^i , and the only difference for y is that we replace $F(k, i)$ with a random string. Thus Hybrids $i.1$ and $i.2$ are indistinguishable except with probability δ_F .

Hyb_{*i*,3}: Now we change to $y \leftarrow D_1(r)$. Hybrids $i.2$ and $i.3$ are indistinguishable except with probability ϵ .

Hyb_{*i*,4}: Now we change to $y \leftarrow D_1(F(\bar{k}, i))$. Hybrids $i.3$ and $i.4$ are indistinguishable except with probability δ_F .

Next, we observe that $E_{i,1}$, when using $y \leftarrow D_1(i; F(\bar{k}, i))$, is actually functionally equivalent to $E_{i+1,0}$. Thus, we see that the programs $P^{E_{i,1}}$ and $P^{E_{i+1,0}}$ are functionally equivalent. By iO security, we therefore have that Hybrid $i.4$ and $(i+1).0$ are indistinguishable except with probability δ_{iO} .

The proof then follows by observing that Hybrid 1.0 corresponds to $\text{iO}(1^\lambda, 1^s, P^{D_0(F(k, \cdot))})$ and Hybrid $(N+1).0$ corresponds to $\text{iO}(1^\lambda, 1^s, P^{D_1(F(\bar{k}, \cdot))})$

□

3.2 Information-Theoretical Hardness of Hidden Subspace Detection

One of our central objects in this paper are quantumly accessible classical functions that check membership in some secret linear subspace $S \subseteq \mathbb{Z}_2^k$.

Information-Theoretical Subspace Hiding. We start with a quantum lower bound for detecting a change between two oracles: One allows access to membership check for some given (known) subspace S , and the other allows access to membership check in a random superspace T of S . This is an information-theoretical version of the subspace-hiding obfuscation introduced in [Zha19].

Lemma 20. *Let $k, r, s \in \mathbb{N}$ such that $r + s \leq k$ and let $S \subseteq \mathbb{Z}_2^k$ a subspace of dimension r . Let S_s the uniform distribution over subspaces T of dimension $r + s$ such that $S \subseteq T \subseteq \mathbb{Z}_2^k$. For any subspace $S' \subseteq \mathbb{Z}_2^k$ let $\mathcal{O}_{S'}$ the oracle that checks membership in S' (outputs 1 if and only if the input is inside S').*

Then, for every oracle-aided quantum algorithm $\mathcal{A}$ making at most q quantum queries, we have

the following indistinguishability over oracle distributions.

$$\{\mathcal{O}_S\} \approx \frac{q \cdot s}{\sqrt{2^{k-r-s}}} \{\mathcal{O}_T : T \leftarrow \mathcal{S}_s\} .$$

Proof. We prove the claim by a hybrid argument, increasing the dimension of the random superspace T by 1 in each step, until we made an increase of s dimensions. In the first step we consider a matrix $\mathbf{B} \in \mathbb{Z}_2^{k \times (k-r)}$, the columns of which form a basis for $S^\perp$. Note that the oracle $\mathcal{O}_S$ can be described as accepting $\mathbf{x} \in \mathbb{Z}_2^k$ iff $\mathbf{x}^T \cdot \mathbf{B} = 0^{k-r}$.

Next we sample a uniformly random $\mathbf{a} \in \mathbb{Z}_2^{k-r}$ and consider the oracle $\mathcal{O}_{S_1}$ that accepts $\mathbf{x} \in \mathbb{Z}_2^k$ iff either $\mathbf{x}^T \cdot \mathbf{B} = 0^{k-r}$ or $\mathbf{x}^T \cdot \mathbf{B} = \mathbf{a}$. Two things can be verified: (1) Due to the randomness of $\mathbf{a}$, by standard quantum lower bounds, $\{\mathcal{O}_S\} \approx \frac{q}{\sqrt{2^{k-r}}} \{\mathcal{O}_{S_1} : \mathbf{a} \leftarrow \mathbb{Z}_2^{k-r}\}$, and (2) The set S_1 is a random superspace of S with dimension $r+1$.

This proves our claim for $s=1$. For a general s we can make an s -step hybrid argument, where at each step we have S_i which is a random $(r+i)$ -dimensional superspace of S . Overall we get that for a q -query algorithm $\mathcal{A}$ the distinguishing advantage between $\{\mathcal{O}_S\}$ and $\{\mathcal{O}_{S_1} : T \leftarrow \mathcal{S}_s\}$ is

$$\sum_{i \in [s]} \frac{q}{\sqrt{2^{k-r-(i-1)}}} \leq \frac{q \cdot s}{\sqrt{2^{k-r-s}}} ,$$

as needed. $\square$

We will also use the following corollary of Lemma 20. The corollary says that it is still hard to distinguish membership check between S and T , also when we duplicate the oracle access ℓ times. The corollary follows by a direct simulation reduction.

Corollary 21. *Let $k, r, s \in \mathbb{N}$ such that $r+s \leq k$ and let $S \subseteq \mathbb{Z}_2^k$ a subspace of dimension r . Let $\mathcal{S}_s$ the uniform distribution over subspaces T of dimension $r+s$ such that $S \subseteq T \subseteq \mathbb{Z}_2^k$. For any subspace $S' \subseteq \mathbb{Z}_2^k$ let $\mathcal{O}_{S'}$ the oracle that checks membership in S' (outputs 1 if and only if the input is inside S').*

Then, for every oracle-aided quantum algorithm $\mathcal{A}$ making at most q quantum queries, we have the following indistinguishability over oracle distributions.

$$\{\mathcal{O}_S^1, \dots, \mathcal{O}_S^\ell\} \approx \frac{q \cdot \ell \cdot s}{\sqrt{2^{k-r-s}}} \{\mathcal{O}_T^1, \dots, \mathcal{O}_T^\ell : T \leftarrow \mathcal{S}_s\} .$$

An additional corollary which follows by combining the above, with a standard hybrid argument on the first statement 20 is as follows. Note that in the below statement, the first oracle distribution is where each of the ℓ oracles samples an i.i.d. superspace T_i , and the second oracle distribution is where we sample T once, and then duplicate its oracle.

Corollary 22. *Let $k, r, s \in \mathbb{N}$ such that $r+s \leq k$ and let $S \subseteq \mathbb{Z}_2^k$ a subspace of dimension r . Let $\mathcal{S}_s$ the uniform distribution over subspaces T of dimension $r+s$ such that $S \subseteq T \subseteq \mathbb{Z}_2^k$. For any subspace $S' \subseteq \mathbb{Z}_2^k$ let $\mathcal{O}_{S'}$ the oracle that checks membership in S' (outputs 1 if and only if the input is inside S').*

Then, for every oracle-aided quantum algorithm $\mathcal{A}$ making at most q quantum queries, we have the following indistinguishability over oracle distributions.

$$\begin{aligned} & \{\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_\ell} : \forall i \in [\ell] : T_i \leftarrow \mathcal{S}_s\} \\ & \approx \frac{q \cdot \ell \cdot s}{\sqrt{2^{k-r-s}}} \{\mathcal{O}_T^1, \dots, \mathcal{O}_T^\ell : T \leftarrow \mathcal{S}_s\} . \end{aligned}$$

The below is an information theoretical version of our Lemma 27 and corresponding proof.

Lemma 23. *Let $k, r, s \in \mathbb{N}$ such that $r + s \leq k$ and let $S \subseteq \mathbb{Z}_2^k$ a subspace of dimension r . Let $\mathcal{S}_s$ the uniform distribution over subspaces T of dimension $r + s$ such that $S \subseteq T \subseteq \mathbb{Z}_2^k$. For any subspace $S' \subseteq \mathbb{Z}_2^k$ let $\mathcal{O}_{S'}$ the oracle that checks membership in S' (outputs 1 if and only if the input is inside S').*

Assume there is an oracle-aided quantum algorithm $\mathcal{A}$ making at most q quantum queries and outputting a vector $\mathbf{u} \in \mathbb{Z}_2^k$ at the end of its execution, such that

$$\Pr \left[\mathcal{A}^{\mathcal{O}_T} \in (T^\perp \setminus \{0\}) : T \leftarrow \mathcal{S}_s \right] \geq \epsilon .$$

Also, denote $t := k - r - s$, $\ell := \frac{k(t+1)}{\epsilon}$ and assume (1) $\frac{t}{2^{s-t}} \leq \frac{\epsilon}{2}$ and (2) $\frac{q \cdot \ell^2 \cdot s}{\sqrt{2}^t} \leq \frac{1}{2}$. Then, it is necessarily the case that

$$\Pr \left[\mathcal{A}^{\mathcal{O}_T} \in (S^\perp \setminus T^\perp) : T \leftarrow \mathcal{S}_s \right] \geq \frac{\epsilon}{16 \cdot k \cdot (t+1)} .$$

Proof. We start with defining the following reduction $\mathcal{B}$, that will use the circuit $\mathcal{A}$ as part of its machinery.

The reduction $\mathcal{B}$. The input to $\mathcal{B}$ contains $\ell := \frac{k \cdot (t+1)}{\epsilon}$ samples of oracles $(\mathcal{O}^{(1)}, \dots, \mathcal{O}^{(\ell)})$, for $t := k - r - s$. Given the ℓ oracles, execute $\mathcal{A}^{\mathcal{O}^{(i)}}$ for every $i \in [\ell]$ and obtain ℓ vectors $\{u_1, \dots, u_\ell\}$. Then, take only the vectors $\{v_1, \dots, v_m\}$ that are inside $S^\perp$, and then compute the dimension of their span, $D := \dim(\text{Span}(v_1, \dots, v_m))$. Note that the number of queries that $\mathcal{B}$ makes is $q \cdot \ell$.

Executing $\mathcal{B}$ on the oracle distribution $\mathcal{D}_1$. Consider the following distribution $\mathcal{D}_1$: Sample ℓ i.i.d superspaces $T_1, \dots, T_\ell$, and for each of them, give access to its membership check oracle: $\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_\ell}$. Let us see what happens when we execute $\mathcal{B}$ on a sample from the distribution $\mathcal{D}_1$.

Consider the ℓ vectors $\{u_1, \dots, u_\ell\}$ obtained by executing $\mathcal{A}$ on each of the input oracles. Recall that $\ell := \frac{1}{\epsilon} \cdot k \cdot (t+1)$ and consider a partition of the vectors into $t+1$ consecutive sequences (or buckets), accordingly, each of length $\frac{1}{\epsilon} \cdot k$. In order to show that the probability for the reduction $\mathcal{B}$ to have $D \geq t+1$ is high, we show that with high probability, in each bucket $j \in [t+1]$ there is a vector u_i that's inside the corresponding dual $T_i^\perp$, but such that also the intersection between $T_i^\perp$ and each of the previous $j-1$ dual subspaces that were hit by $\mathcal{A}$, is only the zero vector 0^k . Note that the last condition indeed implies $D \geq t+1$.

For every $i \in [\ell]$ we define the probability p_i . We start with defining it for the indices in the first bucket, and then proceed to define it recursively for the rest of the buckets. For indices $i \in [\frac{1}{\epsilon} \cdot k]$ in the first bucket, p_i is the probability that given access to $\mathcal{O}_{T_i}$, the output of $\mathcal{A}$ is $u_i \in (T_i^\perp \setminus \{0\})$, and in such case we define the i -th execution as successful. We denote by $T_{(1)}$ the first subspace in the first bucket where a successful execution happens (and define $T_{(1)} := \perp$ if no success happened). For any i inside any bucket $j \in ([t+1] \setminus \{1\})$ that is not the first bucket, we define p_i as the probability that (1) $u_i \in (T_i^\perp \setminus \{0\})$ and also (2) the intersection between $T_i^\perp$ and each of the dual subspaces of the previous winning subspaces $T_{(1)}, \dots, T_{(j-1)}$, is only $\{0^k\}$. That is, p_i is the probability that the output of the adversary hits the dual subspace, and also the dual does not have a non-trivial intersection with any of the previous successful duals. Similarly to the first bucket, we denote by $T_{(j)}$ the first subspace in bucket j with a successful execution.

We prove that with high probability, all $t + 1$ buckets have at least one successful execution. To see this, we define the following probability p' which we show lower bounds p_i , and is defined as follows. First, let $\bar{T}_1, \dots, \bar{T}_t$ any t subspaces, each of dimension $r + s$, thus the duals $\bar{T}_1^\perp, \dots, \bar{T}_t^\perp$ are such that each has dimension t . $p'_{(\bar{T}_1, \dots, \bar{T}_t)}$ is the probability that (1) when sampling $T^\perp$, the intersection of $T^\perp$ with each of the t dual subspaces $\bar{T}_1^\perp, \dots, \bar{T}_t^\perp$ was only the zero vector, and also (2) the output of the adversary $\mathcal{A}$ was inside $T^\perp$. p' is defined as the minimal probability taken over all possible choices of t subspaces $\bar{T}_1, \dots, \bar{T}_t$. After one verifies that indeed for every i we have $p' \leq p_i$, it is sufficient to lower bound p' .

Lower bound for the probability p' . The probability p' is for an event that's defined as the logical AND of two events, and as usual, equals the product between the probability p'_0 of the first event (the trivial intersection between the subspaces), times the conditional probability p'_1 of the second event (that $\mathcal{A}$ hits a non-zero vector in the dual $T^\perp$), conditioned on the first event.

First we lower bound the probability p'_0 by upper bounding the complement probability, that is, we show that the probability for a non-trivial intersection is small. Consider the random process of choosing a basis for a subspace T and note that it is equivalent to choosing a basis for the dual $T^\perp$. The process of choosing a basis for the dual has t steps, and in each step we choose a random vector in $S^\perp$ that's outside the span we aggregated so far. Given a dual subspace $\bar{T}^\perp$ of dimension t , what is the probability for the two subspaces to have only a trivial intersection? It is exactly the sum over $z \in [t]$ (which we think of as the steps for sampling $T^\perp$) of the following event: In the t -step process of choosing a basis for $T^\perp$, index z was the first to cause the subspaces to have a non-zero intersection. Recall that for each $z \in [t]$, the probability that z was such first index to cause an intersection, equals the probability that the z -th sampled basis vector for $T^\perp$ is a vector that's inside the unified span of $\bar{T}^\perp$ and the aggregated span of $T^\perp$ so far, after $z - 1$ samples. This amounts to the probability

$$\begin{aligned} \sum_{z \in [t]} \frac{|\bar{T}^\perp| \cdot 2^{z-1}}{|S^\perp|} &= \sum_{z \in [t]} \frac{2^t \cdot 2^{z-1}}{2^{k-r}} = 2^{-s} \cdot \sum_{z \in \{0, 1, \dots, t-1\}} 2^z \\ &= 2^{-s} \cdot (2^t - 1) < 2^{t-s}. \end{aligned}$$

Since the above is an upper bound on the probability for a non-trivial intersection between $T^\perp$ and one more single subspace, by union bound, the probability for $T^\perp$ to have a non-trivial intersection with at least one of the t subspaces $\bar{T}_1^\perp, \dots, \bar{T}_t^\perp$ is upper bounded by $t \cdot 2^{t-s}$. This means that $p'_0 \geq 1 - t \cdot 2^{t-s}$.

The lower bound for the conditional probability p'_1 is now quite easy: Note that since $\Pr[A|B] \geq \Pr[A] - \Pr[\neg B]$, letting A the event that $\mathcal{A}$ outputs a vector in the dual $T^\perp$ and B the event that $T^\perp$ has only a trivial intersection with all other t subspaces, we get $p'_1 \geq \epsilon - t \cdot 2^{t-s}$. By our assumption that $\frac{t}{2^{s-t}} \leq \frac{\epsilon}{2}$, we have $p'_1 \geq \frac{\epsilon}{2}$. Overall we got $p' := p'_0 \cdot p'_1 \geq (1 - t \cdot 2^{t-s}) \cdot \frac{\epsilon}{2} > \frac{\epsilon}{4}$.

Finally, to see why we get an overall high probability for $D \geq t + 1$ on a sample from $\mathcal{D}_1$, observe the following. In each bucket there are $\frac{k}{\epsilon}$ attempts, each succeeds with probability at least $\frac{\epsilon}{4}$ and thus the overall success probability in a bucket is $\geq 1 - e^{-\Omega(k)}$. Accordingly, the probability to succeed at least once in each of the $t + 1$ buckets (and thus to satisfy $D \geq t + 1$) is $\geq 1 - (t + 1) \cdot e^{-\Omega(k)}$, by considering the complement probability and applying union bound. Overall the probability for $D \geq t + 1$ is thus $\geq 1 - e^{-\Omega(k)}$.

Executing $\mathcal{B}$ on the distribution $\mathcal{D}_2$. Consider a different distribution $\mathcal{D}_2$: Sample T once, then allow an ℓ -oracle access to it, $\mathcal{O}_T^{(1)}, \dots, \mathcal{O}_T^{(\ell)}$. Note that $\mathcal{B}$ is a $q \cdot \ell$ -query algorithm and thus by Corollary 22 there is the following indistinguishability of oracles with respect to $\mathcal{B}$:

$$\mathcal{D}_1 \approx \frac{q \cdot \ell^2 \cdot s}{\sqrt{2^{k-r-s}}} \mathcal{D}_2 .$$

Since given a sample oracle from $\mathcal{D}_1$, the algorithm $\mathcal{B}$ outputs $D \geq t+1$ with probability $\geq 1 - e^{-\Omega(k)}$, by the above indistinguishability, whenever we execute $\mathcal{B}$ on a sample from $\mathcal{D}_2$, then with probability at least $\geq 1 - e^{-\Omega(k)} - \frac{q \cdot \ell^2 \cdot s}{\sqrt{2^{k-r-s}}} \geq 1 - \frac{q \cdot \ell^2 \cdot s}{\sqrt{2^{k-r-s}}}$ we have $D \geq t+1$. By our assumption in the Lemma that $\frac{q \cdot \ell^2 \cdot s}{\sqrt{2^{k-r-s}}} \leq \frac{1}{2}$, with probability at least $\frac{1}{2}$ we have $D \geq t+1$ given a sample from $\mathcal{D}_2$. By an averaging argument, it follows that with probability at least $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$ over sampling the superspace T , the probability p_T for the event where $D \geq t+1$, is at least $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$. Let us call this set of superspaces T , "the good set" of samples, which by definition has fraction at least $\frac{1}{4}$. Recall two facts: (1) the dimension of $T^\perp$ is t , (2) The dimension D aggregates vectors inside $S^\perp$. The two facts together imply that in the event $D \geq t+1$, it is necessarily the case that there exists an execution index $i \in [\ell]$ in the reduction $\mathcal{B}$ where $\mathcal{A}$ outputs a vector in $(S^\perp \setminus T^\perp)$, given membership check in T .

For every T inside the good set we thus know that with probability $\frac{1}{4}$, one of the output vectors of $\mathcal{A}$ will be in $(S^\perp \setminus T^\perp)$. Since these are ℓ i.i.d. executions of $\mathcal{A}$, by union bound, for every T inside the good set, when we prepare an oracle access to T and execute $\mathcal{A}$, we will get $\mathcal{A}^{\mathcal{O}_T} \in (S^\perp \setminus T^\perp)$ with probability $\frac{1}{4\ell}$. We deduce that for a uniformly random T which we then prepare oracle access to, the probability for $\mathcal{A}^{\mathcal{O}_T} \in (S^\perp \setminus T^\perp)$ is at least the probability for this event and also that T is inside the good set, which in turn is at least

$$\frac{1}{4} \cdot \frac{1}{4 \cdot \ell} = \frac{1}{16 \cdot \ell} := \frac{\epsilon}{16 \cdot k \cdot (t+1)} ,$$

which finishes our proof. $\square$

3.3 Cryptographic Hardness of Hidden Subspace Detection

In this subsection we prove the cryptographic analogues of the information theoretical lower bounds from the previous section.

We start with stating the subspace-hiding obfuscation property of indistinguishability obfuscators from [Zha19].

Lemma 24. *Let $k, r, s \in \mathbb{N}$ such that $r+s \leq k$ and let $S \subseteq \mathbb{Z}_2^k$ a subspace of dimension r . Let $\mathcal{S}_s$ the uniform distribution over subspaces T of dimension $r+s$ such that $S \subseteq T \subseteq \mathbb{Z}_2^k$. For any subspace $S' \subseteq \mathbb{Z}_2^k$ let $C_{S'}$ some canonical classical circuit that checks membership in S' , say be Gaussian elimination. Let iO an indistinguishability obfuscation scheme that is $(f_{\text{iO}}(\lambda), \epsilon_{\text{iO}}(\lambda))$ -secure, and assume that $(f_{\text{OWF}}(\lambda), \epsilon_{\text{OWF}}(\lambda))$ -secure injective one-way functions exist.*

Then, for every security parameter λ such that $\lambda \leq k - r - s$ and sufficiently large $p := p(\lambda)$ polynomial in the security parameter, we have the following indistinguishability for $f(\lambda) := \min(f_{\text{OWF}}(\lambda), f_{\text{iO}}(\text{poly}(\lambda)))$, $\epsilon(\lambda) := \max(\epsilon_{\text{OWF}}(\lambda), \epsilon_{\text{iO}}(\text{poly}(\lambda)))$,

$$\{\mathcal{O}_S : \mathcal{O}_S \leftarrow \text{iO}(1^\lambda, 1^p, C_S)\} \approx_{(f(\lambda) - \text{poly}(\lambda), s \cdot \epsilon(\lambda))}$$

$$\{\mathcal{O}_T : T \leftarrow \mathcal{S}_s, \mathcal{O}_T \leftarrow \text{iO}(1^\lambda, 1^p, C_T)\} .$$

The following generalization of Lemma 24 is derived by a random self-reducibility argument and is formally proved by using an additional layer of indistinguishability obfuscation.

Lemma 25. *Let $k, r, s \in \mathbb{N}$ such that $r + s \leq k$ and let $S \subseteq \mathbb{Z}_2^k$ a subspace of dimension r . Let $\mathcal{S}_s$ the uniform distribution over subspaces T of dimension $r + s$ such that $S \subseteq T \subseteq \mathbb{Z}_2^k$. For any subspace $S' \subseteq \mathbb{Z}_2^k$ let $C_{S'}$ some canonical classical circuit that checks membership in S' , say be Gaussian elimination. Let iO an indistinguishability obfuscation scheme that is $(f_{\text{iO}}(\lambda), \epsilon_{\text{iO}}(\lambda))$ -secure, and assume that $(f_{\text{OWF}}(\lambda), \epsilon_{\text{OWF}}(\lambda))$ -secure injective one-way functions exist.*

Then, for every security parameter λ such that $\lambda \leq k - r - s$ and sufficiently large $p := p(\lambda)$ polynomial in the security parameter, we have the following indistinguishability for $f(\lambda) := \min(f_{\text{OWF}}(\lambda), f_{\text{iO}}(\text{poly}(\lambda)))$, $\epsilon(\lambda) := \max(\epsilon_{\text{OWF}}(\lambda), \epsilon_{\text{iO}}(\text{poly}(\lambda)))$,

$$\{\mathcal{O}_S^1, \dots, \mathcal{O}_S^\ell : \forall i \in [\ell], \mathcal{O}_S^i \leftarrow \text{iO}(1^\lambda, 1^p, C_S)\} \approx_{(f(\lambda) - \ell \cdot \text{poly}(\lambda), (2 \cdot \ell + s) \cdot \epsilon(\lambda))}$$

$$\{\mathcal{O}_T^1, \dots, \mathcal{O}_T^\ell : T \leftarrow \mathcal{S}_s, \forall i \in [\ell], \mathcal{O}_T^i \leftarrow \text{iO}(1^\lambda, 1^p, C_T)\} .$$

Proof. We first observe that as long as the circuit size parameter 1^p is sufficiently large, and specifically, larger than the size of an obfuscated version of the plain circuit C , then it is indistinguishable to tell whether said circuit is obfuscated under one or two layers of obfuscation. More precisely, due to the perfect correctness of obfuscation, the circuit C and an obfuscated $\mathcal{O}_C \leftarrow \text{iO}(1^\lambda, 1^p, C)$ have the same functionality (for every sample out of the distribution of obfuscated versions of C), and thus, as long as $p \geq |\mathcal{O}_C|$, then the distributions $\mathcal{D}_0 := \{\mathcal{O}_C \leftarrow \text{iO}(1^\lambda, 1^p, C)\}$ and $\mathcal{D}_1 := \{\mathcal{O}_C^2 \leftarrow \text{iO}(1^\lambda, 1^p, \mathcal{D}_0)\}$ are $(f(\lambda), \epsilon(\lambda))$ -indistinguishable.

An implication of the above is that for a sufficiently large parameter p , the distribution

$$\{\mathcal{O}_S^1, \dots, \mathcal{O}_S^\ell : \forall i \in [\ell], \mathcal{O}_S^i \leftarrow \text{iO}(1^\lambda, 1^p, C_S)\}$$

is $(f(\lambda), \ell \cdot \epsilon(\lambda))$ -indistinguishable from a distribution where C_S is swapped with an obfuscation of it (let us denote this modified distribution with $\mathcal{D}_S$), and the distribution

$$\{\mathcal{O}_T^1, \dots, \mathcal{O}_T^\ell : T \leftarrow \mathcal{S}_s, \forall i \in [\ell], \mathcal{O}_T^i \leftarrow \text{iO}(1^\lambda, 1^p, C_T)\}$$

is $(f(\lambda), \ell \cdot \epsilon(\lambda))$ -indistinguishable from a distribution where C_T is swapped with an obfuscation of it (let us denote this modified distribution with $\mathcal{D}_T$). To complete our proof we will show that $\mathcal{D}_S$ and $\mathcal{D}_T$ are appropriately indistinguishable, and will get our proof by transitivity of computational distance.

The indistinguishability between $\mathcal{D}_S$ and $\mathcal{D}_T$ follows almost readily from the the subspace hiding Lemma 24: One can consider the reduction $\mathcal{B}$ that gets a sample $\mathcal{O}$ which is either from $\mathcal{D}_0 := \{\mathcal{O}_S \leftarrow \text{iO}(1^\lambda, 1^p, C_S)\}$ or from $\mathcal{D}_1 := \{\mathcal{O}_T \leftarrow \text{iO}(1^\lambda, 1^p, C_T)\}$ for an appropriately random superspace T of S . $\mathcal{B}$ then generates ℓ i.i.d obfuscations $\{\mathcal{O}^{(1)}, \dots, \mathcal{O}^{(\ell)}\}$ of the (already obfuscated) circuit $\mathcal{O}$ and executes $\mathcal{A}$ on the ℓ obfuscations. One can see that when the input sample $\mathcal{O}$ for $\mathcal{B}$ came from $\mathcal{D}_0$ then the output sample of the reduction comes from the distribution $\mathcal{D}_S$, and when the input sample $\mathcal{O}$ for $\mathcal{B}$ came from $\mathcal{D}_1$ then the output sample of the reduction comes from the distribution $\mathcal{D}_T$. Since the reduction executes in complexity $\ell \cdot \text{poly}(\lambda)$, this means that

$$\mathcal{D}_S \approx_{(f(\lambda) - \ell \cdot \text{poly}(\lambda), s \cdot \epsilon(\lambda))} \mathcal{D}_T .$$

To conclude, by transitivity of computational indistinguishability, we get that the two distributions in our Lemma's statement are $(f(\lambda) - \ell \cdot \text{poly}(\lambda), (2 \cdot \ell + s) \cdot \epsilon(\lambda))$ -indistinguishable, as needed. $\square$

A corollary which follows by combining the above Lemma 25, with a standard hybrid argument on the first lemma 24 is as follows.

Corollary 26. *Let $k, r, s \in \mathbb{N}$ such that $r + s \leq k$ and let $S \subseteq \mathbb{Z}_2^k$ a subspace of dimension r . Let $\mathcal{S}_s$ the uniform distribution over subspaces T of dimension $r + s$ such that $S \subseteq T \subseteq \mathbb{Z}_2^k$. For any subspace $S' \subseteq \mathbb{Z}_2^k$ let $C_{S'}$ some canonical classical circuit that checks membership in S' , say be Gaussian elimination. Let iO an indistinguishability obfuscation scheme that is $(f_{\text{iO}}(\lambda), \epsilon_{\text{iO}}(\lambda))$ -secure, and assume that $(f_{\text{OWF}}(\lambda), \epsilon_{\text{OWF}}(\lambda))$ -secure injective one-way functions exist.*

Then, for every security parameter λ such that $\lambda \leq k - r - s$ and sufficiently large $p := p(\lambda)$ polynomial in the security parameter, we have the following indistinguishability for $f(\lambda) := \min(f_{\text{OWF}}(\lambda), f_{\text{iO}}(\text{poly}(\lambda)))$, $\epsilon(\lambda) := \max(\epsilon_{\text{OWF}}(\lambda), \epsilon_{\text{iO}}(\text{poly}(\lambda)))$,

$$\{\mathbf{O}_{T_1}, \dots, \mathbf{O}_{T_\ell} : T_i \leftarrow \mathcal{S}_s \forall i \in [\ell], \mathbf{O}_{T_i} \leftarrow \text{iO}(1^\lambda, 1^p, C_{T_i})\}$$

$$\approx_{(f(\lambda) - \ell \cdot \text{poly}(\lambda), (2 \cdot \ell \cdot s) \cdot \epsilon(\lambda))} \{\mathbf{O}_T^1, \dots, \mathbf{O}_T^\ell : T \leftarrow \mathcal{S}_s, \forall i \in [\ell], \mathbf{O}_T^i \leftarrow \text{iO}(1^\lambda, 1^p, C_T)\} .$$

Improved Results on Hardness of Concentration in Dual of Obfuscated Subspace. As part of this work we strengthen the main technical lemma (Lemma 5.1) from [Shm22b]. Roughly speaking, in [Shm22b] it is shown that an adversary $\mathcal{A}$ that gets an obfuscation $\mathbf{O}_T$ for a random superspace T of S , and manages to output a (non-zero) vector in the dual $T^\perp \setminus \{\mathbf{0}\}$ with probability ϵ , has to sometimes output vectors in $S^\perp \setminus T^\perp$, i.e. with probability at least $\Omega(\epsilon^2) / \text{poly}(k)$. Below we strengthen the probability to $\Omega(\epsilon) / \text{poly}(k)$.

Lemma 27 (IO Dual Subspace Anti-Concentration). *Let $k, r, s \in \mathbb{N}$ such that $r + s \leq k$ and let $S \subseteq \mathbb{Z}_2^k$ a subspace of dimension r . Let $\mathcal{S}_s$ the uniform distribution over subspaces T of dimension $r + s$ such that $S \subseteq T \subseteq \mathbb{Z}_2^k$. For any subspace $S' \subseteq \mathbb{Z}_2^k$ let $C_{S'}$ some canonical classical circuit that checks membership in S' , say be Gaussian elimination. Let iO an indistinguishability obfuscation scheme that is $(f_{\text{iO}}(\lambda), \epsilon_{\text{iO}}(\lambda))$ -secure, and assume that $(f_{\text{OWF}}(\lambda), \epsilon_{\text{OWF}}(\lambda))$ -secure injective one-way functions exist.*

Let $\lambda \in \mathbb{N}$ the security parameter such that $\lambda \leq k - r - s$ and let $p := p(\lambda)$ a sufficiently large polynomial in the security parameter. Denote $f(\lambda) := \min(f_{\text{OWF}}(\lambda), f_{\text{iO}}(\text{poly}(\lambda)))$, $\epsilon(\lambda) := \max(\epsilon_{\text{OWF}}(\lambda), \epsilon_{\text{iO}}(\text{poly}(\lambda)))$. Denote by $\mathcal{O}_{\lambda, p, s}$ the distribution over obfuscated circuits that samples $T \leftarrow \mathcal{S}_s$ and then $\mathbf{O}_T \leftarrow \text{iO}(1^\lambda, 1^p, C_T)$.

Assume there is a quantum algorithm $\mathcal{A}$ of complexity $T_{\mathcal{A}}$ such that,

$$\Pr \left[\mathcal{A}(\mathbf{O}_T) \in (T^\perp \setminus \{\mathbf{0}\}) : \mathbf{O}_T \leftarrow \mathcal{O}_{\lambda, p, s} \right] \geq \epsilon .$$

Also, denote $t := k - r - s$, $\ell := \frac{k(t+1)}{\epsilon}$ and assume (1) $\frac{t}{2s-t} \leq \frac{\epsilon}{2}$ and (2) $\frac{\ell \cdot (k^3 + \text{poly}(\lambda) + T_{\mathcal{A}})}{f(\lambda)} \leq \frac{1}{8}$. Then, it is necessarily the case that

$$\Pr \left[\mathcal{A}(\mathbf{O}_T) \in (S^\perp \setminus T^\perp) : \mathbf{O}_T \leftarrow \mathcal{O}_{\lambda, p, s} \right] \geq \frac{\epsilon}{16 \cdot k \cdot (t+1)} .$$

Proof. We start with defining the following reduction $\mathcal{B}$, that will use the circuit $\mathcal{A}$ as part of its machinery.

The reduction $\mathcal{B}$. The input to $\mathcal{B}$ contains $\ell := \frac{k \cdot (t+1)}{\epsilon}$ samples of obfuscations $(\mathcal{O}^{(1)}, \dots, \mathcal{O}^{(\ell)})$, for $t := k - r - s$. Given the ℓ obfuscations, execute $\mathcal{A}(\mathcal{O}^{(i)})$ for every $i \in [\ell]$ and obtain ℓ vectors $\{u_1, \dots, u_\ell\}$. Then, take only the vectors $\{v_1, \dots, v_m\}$ that are inside $S^\perp$, and then compute the dimension of their span, $D := \dim(\text{Span}(v_1, \dots, v_m))$. Note that the running time of $\mathcal{B}$ is $\ell \cdot T_{\mathcal{A}} + \ell \cdot k^3$, where $\ell \cdot T_{\mathcal{A}}$ is for producing the ℓ outputs of $\mathcal{A}$ and $\ell \cdot k^3$ is for (naively) executing Gaussian elimination ℓ times, to repeatedly check whether the new vector v_i adds a dimension i.e., whether it is outside of the span $\text{Span}(v_1, \dots, v_{i-1})$ of the previous vectors.

Executing $\mathcal{B}$ on the distribution $\mathcal{D}_1$. Consider the following distribution $\mathcal{D}_1$: Sample ℓ i.i.d superspaces $T_1, \dots, T_\ell$, and for each of them, send an obfuscation of it: $\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_\ell}$. Let us see what happens when we execute $\mathcal{B}$ on a sample from the distribution $\mathcal{D}_1$.

Consider the ℓ vectors $\{u_1, \dots, u_\ell\}$ obtained by executing $\mathcal{A}$ on each of the input obfuscations. Recall that $\ell := \frac{1}{\epsilon} \cdot k \cdot (t+1)$ and consider a partition of the vectors into $t+1$ consecutive sequences (or buckets), accordingly, each of length $\frac{1}{\epsilon} \cdot k$. In order to show that the probability for the reduction $\mathcal{B}$ to have $D \geq t+1$ is high, we show that with high probability, in each bucket $j \in [t+1]$ there is a vector u_i that's inside the corresponding dual $T_i^\perp$, but such that also the intersection between $T_i^\perp$ and each of the previous $j-1$ dual subspaces that were hit by $\mathcal{A}$, is only the zero vector 0^k . Note that the last condition indeed implies $D \geq t+1$.

For every $i \in [\ell]$ we define the probability p_i . We start with defining it for the indices in the first bucket, and then proceed to define it recursively for the rest of the buckets. For indices $i \in [\frac{1}{\epsilon} \cdot k]$ in the first bucket, p_i is the probability that given $\mathcal{O}_{T_i}$, the output of $\mathcal{A}$ is $u_i \in (T_i^\perp \setminus \{0\})$, and in such case we define the i -th execution as successful. We denote by $T_{(1)}$ the first subspace in the first bucket where a successful execution happens (and define $T_{(1)} := \perp$ if no success happened). For any i inside any bucket $j \in ([t+1] \setminus \{1\})$ that is not the first bucket, we define p_i as the probability that (1) $u_i \in (T_i^\perp \setminus \{0\})$ and also (2) the intersection between $T_i^\perp$ and each of the dual subspaces of the previous winning subspaces $T_{(1)}, \dots, T_{(j-1)}$, is only $\{0^k\}$. That is, p_i is the probability that the output of the adversary hits the dual subspace, and also the dual does not have a non-trivial intersection with any of the previous successful duals. Similarly to the first bucket, we denote by $T_{(j)}$ the first subspace in bucket j with a successful execution.

We prove that with high probability, all $t+1$ buckets have at least one successful execution. To see this, we define the following probability p' which we show lower bounds p_i , and is defined as follows. First, let $\bar{T}_1, \dots, \bar{T}_t$ any t subspaces, each of dimension $r+s$, thus the duals $\bar{T}_1^\perp, \dots, \bar{T}_t^\perp$ are such that each has dimension t . $p'_{(\bar{T}_1, \dots, \bar{T}_t)}$ is the probability that (1) when sampling $T^\perp$, the intersection of $T^\perp$ with each of the t dual subspaces $\bar{T}_1^\perp, \dots, \bar{T}_t^\perp$ was only the zero vector, and also (2) the output of the adversary $\mathcal{A}$ was inside $T^\perp$. p' is defined as the minimal probability taken over all possible choices of t subspaces $\bar{T}_1, \dots, \bar{T}_t$. After one verifies that indeed for every i we have $p' \leq p_i$, it is sufficient to lower bound p' .

Lower bound for the probability p' . The probability p' is for an event that's defined as the logical AND of two events, and as usual, equals the product between the probability p'_0 of the first event (the trivial intersection between the subspaces), times the conditional probability p'_1 of the second event (that $\mathcal{A}$ hits a non-zero vector in the dual $T^\perp$), conditioned on the first event.

First we lower bound the probability p'_0 by upper bounding the complement probability, that is, we show that the probability for a non-trivial intersection is small. Consider the random process

of choosing a basis for a subspace T and note that it is equivalent to choosing a basis for the dual $T^\perp$. The process of choosing a basis for the dual has t steps, and in each step we choose a random vector in $S^\perp$ that's outside the span we aggregated so far. Given a dual subspace $\bar{T}^\perp$ of dimension t , what is the probability for the two subspaces to have only a trivial intersection? It is exactly the sum over $z \in [t]$ (which we think of as the steps for sampling $T^\perp$) of the following event: In the t -step process of choosing a basis for $T^\perp$, index z was the first to cause the subspaces to have a non-zero intersection. Recall that for each $z \in [t]$, the probability that z was such first index to cause an intersection, equals the probability that the z -th sampled basis vector for $T^\perp$ is a vector that's inside the unified span of $\bar{T}^\perp$ and the aggregated span of $T^\perp$ so far, after $z-1$ samples. This amounts to the probability

$$\begin{aligned} \sum_{z \in [t]} \frac{|\bar{T}^\perp| \cdot 2^{z-1}}{|S^\perp|} &= \sum_{z \in [t]} \frac{2^t \cdot 2^{z-1}}{2^{k-r}} = 2^{-s} \cdot \sum_{z \in \{0,1,\dots,t-1\}} 2^z \\ &= 2^{-s} \cdot (2^t - 1) < 2^{t-s} . \end{aligned}$$

Since the above is an upper bound on the probability for a non-trivial intersection between $T^\perp$ and one more single subspace, by union bound, the probability for $T^\perp$ to have a non-trivial intersection with at least one of the t subspaces $\bar{T}_1^\perp, \dots, \bar{T}_t^\perp$ is upper bounded by $t \cdot 2^{t-s}$. This means that $p'_0 \geq 1 - t \cdot 2^{t-s}$.

The lower bound for the conditional probability p'_1 is now quite easy: Note that since $\Pr[A|B] \geq \Pr[A] - \Pr[\neg B]$, letting A the event that $\mathcal{A}$ outputs a vector in the dual $T^\perp$ and B the event that $T^\perp$ has only a trivial intersection with all other t subspaces, we get $p'_1 \geq \epsilon - t \cdot 2^{t-s}$. By our assumption that $\frac{t}{2^{s-t}} \leq \frac{\epsilon}{2}$, we have $p'_1 \geq \frac{\epsilon}{2}$. Overall we got $p' := p'_0 \cdot p'_1 \geq (1 - t \cdot 2^{t-s}) \cdot \frac{\epsilon}{2} > \frac{\epsilon}{4}$.

Finally, to see why we get an overall high probability for $D \geq t+1$ on a sample from $\mathcal{D}_1$, observe the following. In each bucket there are $\frac{k}{\epsilon}$ attempts, each succeeds with probability at least $\frac{\epsilon}{4}$ and thus the overall success probability in a bucket is $\geq 1 - e^{-\Omega(k)}$. Accordingly, the probability to succeed at least once in each of the $t+1$ buckets (and thus to satisfy $D \geq t+1$) is $\geq 1 - (t+1) \cdot e^{-\Omega(k)}$, by considering the complement probability and applying union bound. Overall the probability for $D \geq t+1$ is thus $\geq 1 - e^{-\Omega(k)}$.

Executing $\mathcal{B}$ on the distribution $\mathcal{D}_2$. Consider a different distribution $\mathcal{D}_2$: Sample T once, then sample ℓ i.i.d. obfuscations of the same circuit C_T , denoted $\mathbf{O}_T^{(1)}, \dots, \mathbf{O}_T^{(\ell)}$. By Corollary 26,

$$\mathcal{D}_1 \approx_{\left(f(\lambda) - \ell \cdot \text{poly}(\lambda), \frac{2 \cdot s \cdot \ell}{f(\lambda)}\right)} \mathcal{D}_2 .$$

Recall that the running time of $\mathcal{B}$ is $\ell \cdot T_{\mathcal{A}} + \ell \cdot k^3$ and by our Lemma's assumptions, the complexity of $\mathcal{B}$ is $\leq f(\lambda) - \ell \cdot \text{poly}(\lambda)$. Since given a sample oracle from $\mathcal{D}_1$, the algorithm $\mathcal{B}$ outputs $D \geq t+1$ with probability $\geq 1 - e^{-\Omega(k)}$, by the above indistinguishability, whenever we execute $\mathcal{B}$ on a sample from $\mathcal{D}_2$, then with probability at least $\geq 1 - e^{-\Omega(k)} - \frac{2 \cdot s \cdot \ell}{f(\lambda)} \geq 1 - \frac{4 \cdot s \cdot \ell}{f(\lambda)}$ we have $D \geq t+1$. By our assumption in the Lemma that $\frac{s \cdot \ell}{f(\lambda)} \leq \frac{1}{8}$, with probability at least $\frac{1}{2}$ we have $D \geq t+1$ given a sample from $\mathcal{D}_2$. By an averaging argument, it follows that with probability at least $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$ over sampling the superspace T , the probability p_T for the event where $D \geq t+1$, is at least $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$. Let us call this set of superspaces T , "the good set" of samples, which by definition has fraction at least $\frac{1}{4}$. Recall two facts: (1) the dimension of $T^\perp$ is t , (2) The dimension D aggregates vectors

inside $S^\perp$. The two facts together imply that in the event $D \geq t + 1$, it is necessarily the case that there exists an execution index $i \in [\ell]$ in the reduction $\mathcal{B}$ where $\mathcal{A}$ outputs a vector in $(S^\perp \setminus T^\perp)$.

For every T inside the good set we thus know that with probability $\frac{1}{4}$, one of the output vectors of $\mathcal{A}$ will be in $(S^\perp \setminus T^\perp)$. Since these are ℓ i.i.d. executions of $\mathcal{A}$, by union bound, for every T inside the good set, when we prepare an obfuscation $\mathbf{O}_T$ of T and execute $\mathcal{A}$, we will get a vector in $(S^\perp \setminus T^\perp)$ with probability $\frac{1}{4\ell}$. We deduce that for a uniformly random T , the probability for $\mathcal{A}(\mathbf{O}_T) \in (S^\perp \setminus T^\perp)$ is at least the probability for $\mathcal{A}(\mathbf{O}_T) \in (S^\perp \setminus T^\perp)$ intersecting with the event that T is inside the good set, which in turn is at least

$$\frac{1}{4} \cdot \frac{1}{4 \cdot \ell} = \frac{1}{16 \cdot \ell} := \frac{\epsilon}{16 \cdot k \cdot (t + 1)} ,$$

which finishes our proof. □

4 One-Shot Signatures Relative to a Classical Oracle

In this section we present our construction of non-collapsing collision-resistant hash functions (which imply OSS) with respect to a classical oracle and proof of security. We first describe our scheme in 28.

Construction 28. Let $\lambda \in \mathbb{N}$ the statistical security parameter. Define $s := 16 \cdot \lambda$ and let $n, r, k \in \mathbb{N}$ such that $r := s \cdot (\lambda - 1)$, $n := r + \frac{3}{2} \cdot s$, $k := n$.

Let $\Pi : \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a random permutation and let $F : \{0, 1\}^r \rightarrow \{0, 1\}^{k \cdot (n-r+1)}$ a random function. Let $H(x)$ denote the first r output bits of $\Pi(x)$, and $J(x)$ denote the remaining $n - r$ bits, which are interpreted as a vector in $\mathbb{Z}_2^{n-r}$. For each $y \in \{0, 1\}^r$, let $\mathbf{A}(y) \in \mathbb{Z}_2^{k \times (n-r)}$ be a random matrix with full column-rank, and $\mathbf{b}(y) \in \mathbb{Z}_2^k$ be uniformly random, both are generated by the output randomness of $F(y)$.

Then, we let $\mathcal{P} : \{0, 1\}^n \rightarrow (\{0, 1\}^r \times \mathbb{Z}_2^k)$, $\mathcal{P}^{-1} : (\{0, 1\}^r \times \mathbb{Z}_2^k) \rightarrow \{0, 1\}^n$, $\mathcal{D} : (\{0, 1\}^r \times \mathbb{Z}_2^k) \rightarrow \{0, 1\}$ be the following oracles:

$$\begin{aligned} \mathcal{P}(x) &= (y, \mathbf{A}(y) \cdot J(x) + \mathbf{b}(y)) \text{ where } y = H(x) \\ \mathcal{P}^{-1}(y, \mathbf{u}) &= \begin{cases} \Pi^{-1}(y, \mathbf{z}) & \exists \mathbf{z} \in \mathbb{Z}_2^{n-r} \text{ such that } \mathbf{A}(y) \cdot \mathbf{z} + \mathbf{b}(y) = \mathbf{u} \\ \perp & \text{else} \end{cases} \\ \mathcal{D}(y, \mathbf{v}) &= \begin{cases} 1 & \text{if } \mathbf{v}^T \cdot \mathbf{A}(y) = 0^{n-r} \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

We will denote the above distribution of oracles by $\mathcal{O}_{n,r,k}$. We define our hash function as H , which can be easily computed by querying $\mathcal{P}$, considering only the first r bits and discarding the second output.

Note that since $\mathbf{A}(y) \in \mathbb{Z}_2^{k \times (n-r)}$ is full column-rank, $\mathbf{z} \in \mathbb{Z}_2^{n-r}$ is unique if it exists. Thus, $\mathcal{P}^{-1}(\mathcal{P}(x)) = x$ and $\mathcal{P}^{-1}(y, \mathbf{u}) = \perp$ if $(y, \mathbf{u})$ is not in the range of $\mathcal{P}$. Thus, $\mathcal{P}^{-1}$ is the uniquely-defined inverse of the injective function $\mathcal{P}$.

Efficient Implementation. Given the description of Construction 28, it is straightforward to implement it, using pseudorandom functions and pseudorandom permutations, the existence of both of which follow from the existence of one-way functions [Zha21, Zha16]. Specifically, we swap the truly random permutation Π with a pseudorandom permutation PRP, and swap the truly random function F with a pseudorandom function PRF. One can easily verify that under the security of PRP, PRF for quantum queries, the classical efficient construction is computationally indistinguishable from the oracle distribution $\mathcal{O}_{n,r,k}$ described in Construction 28.

Non-collapsing. We prove that our hash function is non-collapsing.

Proposition 29 (Non-collapsing of H). *The hash function H defined in Construction 28 is (always) non-collapsing, as per Definition 10.*

Proof. We explain the non-collapsing property of our scheme by describing the algorithms $(\mathcal{S}_H, \mathcal{D}_H)$. The first algorithm $\mathcal{S}_H$ simply computes a uniform superposition $|+\rangle^{\otimes n}$ over n qubits (where n is the input size for H), and outputs it as $|\psi\rangle$. The second algorithm $\mathcal{D}_H$, given an unknown n -qubit state $|\phi\rangle := \sum_{x \in \mathbb{Z}_2^n} \alpha_x \cdot |x\rangle$, acts as follows.

1. Execute $\mathcal{P}$ in superposition to obtain

$$\sum_{x \in \mathbb{Z}_2^n} \alpha_x \cdot |x\rangle |y_x\rangle |\mathbf{u}_x\rangle .$$

2. Execute $\mathcal{P}^{-1}$ in superposition to un-compute the input register holding x , to obtain

$$\sum_{x \in \mathbb{Z}_2^n} \alpha_x \cdot |y_x\rangle |\mathbf{u}_x\rangle .$$

3. Execute a k -qubit Quantum Fourier Transform over $\mathbb{Z}_2$ on the rightmost k -qubit register (holding the vectors $\mathbf{u}_x$). This boils down to the execution of parallel Hadamard gates $H^{\otimes k}$, to obtain

$$\sum_{x \in \mathbb{Z}_2^n} \alpha_x \cdot |y_x\rangle \left(H^{\otimes k} \cdot |\mathbf{u}_x\rangle \right) .$$

4. Execute $\mathcal{D}(\cdot, \cdot)$ in superposition on the state and measure the output bit register. The output of $\mathcal{D}_H$ is identical to the output of $\mathcal{D}(\cdot, \cdot)$.

To finish the explanation for non-collapsing we will show that the pair $(\mathcal{S}_H, \mathcal{D}_H)$ gives a distinguishing advantage of $\geq 1 - 2^{-\Omega(\lambda)}$, between the cases of full measurement and partial measurement.

- In the first case, given $|\psi\rangle := |+\rangle^{\otimes n}$, it is measured in the computational basis and the algorithm $\mathcal{D}_H$ gets as input some classical $x \in \mathbb{Z}_2^n$. At the end of Step 2 of $\mathcal{D}_H$ the state is $|y_x\rangle |\mathbf{u}_x\rangle$. The execution of QFT in the next step will generate a uniform superposition over all elements in $\mathbb{Z}_2^k$, at the end of Step 3. The set of elements that are accepted by $\mathcal{D}(y_x, \cdot)$ (for every classical y_x) is a coset of dimension r and thus in particular a set of size 2^r . The probability that the k -qubit uniform superposition will be accepted by $\mathcal{D}(y_x, \cdot)$ is $\frac{2^r}{2^k} = 2^{-(k-r)} \leq 2^{-\Omega(\lambda)}$. It follows that the probability that $\mathcal{D}_H$ outputs 1 in the first case is $\leq 2^{-\Omega(\lambda)}$.

- In the second case, given $|\psi\rangle := |+\rangle^{\otimes n}$ we compute $\mathcal{P}$ in superposition and measure a value y on the side. The algorithm $\mathcal{D}_H$ thus gets as input the state $\sum_{x \in \mathbb{Z}_2^n : H(x)=y} \sqrt{2^{-(n-r)}} \cdot |x\rangle$ for some y . One can easily verify that at the end of Step 2, the state that $\mathcal{D}_H$ holds is

$$|y\rangle \otimes \left(\sum_{\mathbf{u} \in \text{ColSpan}(\mathbf{A}(y))} \sqrt{|\text{ColSpan}(\mathbf{A}(y))|^{-1}} \cdot |\mathbf{u} + \mathbf{b}(y)\rangle \right).$$

Next, by standard known properties of QFT applied to a quantum state that is in a uniform superposition over a coset, it follows that at the end of Step 3, the state that $\mathcal{D}_H$ holds is

$$|y\rangle \otimes \left(\sum_{\mathbf{v} \in \text{ColSpan}(\mathbf{A}(y))^\perp} \sqrt{|\text{ColSpan}(\mathbf{A}(y))^\perp|^{-1}} \cdot (-1)^{\langle \mathbf{b}(y), \mathbf{v} \rangle} \cdot |\mathbf{v}\rangle \right).$$

It follows that the probability that $\mathcal{D}_H$ outputs 1 in the second case is 1.

□

Security. Most of this section will be devoted to proving security. There are two main steps to our security proof. The first part of the proof will show Theorem 34 (Proved in Sections 4.1 and 4.2), which says that an adversary $\mathcal{A}$ that manages to find a collision in H given access to $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D})$ sampled from $\mathcal{O}_{n,r,k}$, can be transformed to an adversary $\mathcal{B}$ that finds a collision in H in the dual-free setting, i.e., having only access to $(\mathcal{P}, \mathcal{P}^{-1})$ and no access to the dual verification oracle $\mathcal{D}$. The second part of security will show Theorems 36 and 37 (both proved in Section 4.3), which together, show how a collision finder for the dual-free case can be turned into a collision finder for plain 2-to-1 random functions. We obtain the following main security theorem.

Theorem 30 (Collision Resistance of H). *Let $\mathcal{O}_{n,r,k}$ the distribution over oracles defined in Construction 28. Let $\mathcal{A}$ an oracle aided q -query (computationally unbounded) quantum algorithm. Then,*

$$\Pr \left[x_0 \neq x_1 \wedge H(x_0) = H(x_1) : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \mathcal{O}_{n,r,k} \\ (x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}} \end{array} \right] \leq O \left(\frac{\lambda^3 \cdot k^3 \cdot q^3}{2^\lambda} \right).$$

Proof. Assume towards contradiction that there is an oracle aided quantum algorithm $\mathcal{A}$, making q queries, that given a sample oracle $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \mathcal{O}_{n,r,k}$ outputs a collision (x_0, x_1) in H with probability ϵ , such that $\epsilon \geq \omega \left(\frac{\lambda^3 \cdot k^3 \cdot q^3}{2^\lambda} \right)$.

Note that by our parameter choices in Construction 28 and by our assumption towards contradiction $\epsilon \geq \omega \left(\frac{\lambda^3 \cdot k^3 \cdot q^3}{2^\lambda} \right)$, one can verify through calculation that (1) for $s - (n - r - s) := s'$ we have $\frac{k^3 \cdot q^3 \cdot \frac{1}{\epsilon^2}}{2^{s'}} \leq o(1)$ and also (2) $\frac{k^9 \cdot q^7 \cdot \frac{1}{\epsilon^4}}{\sqrt{2^{n-r-s}}} \leq o(1)$. This means that the conditions of Theorem 34 are satisfied, and it follows there is a q -query algorithm $\mathcal{B}$ that gets access only to $(\mathcal{P}, \mathcal{P}^{-1})$, sampled from $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \mathcal{O}_{r+s, r, k-(n-r-s)}$ that finds collisions in H with probability $\geq \frac{\epsilon}{2^6 \cdot k^2}$.

By Theorem 36, it follows there is a q -query algorithm $\mathcal{B}'$ that given oracle access to any $Q : \{0, 1\}^{r+s} \rightarrow \{0, 1\}^r$ an $(r+s, r, s)$ -coset partition function (as per Definition 35), finds a collision in Q with probability $\frac{\epsilon}{2^6 \cdot k^2}$.

The above is in particular true for any distribution $Q \leftarrow \mathcal{Q}$ over $(r+s, r, s)$ -coset partition functions. Since our parameter choices in Construction 28 imply $s \mid (r+s)$, we can consider the distribution $\mathcal{Q}$ over $(r+s, r, s)$ -coset partition functions generated by Theorem 37. It follows by Theorem 37 that

$$\frac{\epsilon}{2^6 \cdot k^2} \leq O\left(\frac{s^3 \cdot q^3}{2^{\frac{(r+s)}{s}}}\right),$$

which in turn implies

$$\epsilon \leq O\left(\frac{k^2 \cdot \lambda^3 \cdot q^3}{2^\lambda}\right),$$

in contradiction to $\epsilon \geq \omega\left(\frac{\lambda^3 \cdot k^3 \cdot q^3}{2^\lambda}\right)$. $\square$

Direct Construction of One-Shot Signatures from our Non-collapsing CRH. Before turning to the full security proof, we see a direct construction of one-shot signatures from the above Construction 28 of a non-collapsing CRH. The construction uses the CRH functions in a (quantumly queriable) black-box way.

Construction 31 (One-Shot Signature Construction Relative to a Classical Oracle). *Our one-shot signature (OSS) scheme (Setup, TokenGen, Sign, Ver), is defined for security parameter $\lambda \in \mathbb{N}$ and signs on one-bit messages. Our scheme is defined as follows.*

- $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \text{Setup}(1^\lambda)$: The oracle that the setup algorithm samples is exactly the oracles sampled by Construction 28.
- $(\text{pk}, |\text{sk}\rangle) \leftarrow \text{TokenGen}^{\mathcal{P}, \mathcal{P}^{-1}}$: To sample a signature token, start with a uniform superposition $|+\rangle^{\otimes n} := \sum_{x \in \{0,1\}^n} 2^{-\frac{n}{2}} |x\rangle$ and apply $\mathcal{P}$ to obtain

$$\frac{1}{2^{\frac{n}{2}}} \sum_{x \in \{0,1\}^n} |x, y, \mathbf{u}_x\rangle,$$

where $\mathcal{P}(x) := (y, \mathbf{u}_x)$.

Next, use $\mathcal{P}^{-1}$ to un-compute the register holding x and then measure the register holding $y \in \{0,1\}^r$, to obtain the state

$$|y\rangle \otimes \frac{1}{2^{\frac{n-r}{2}}} \sum_{\mathbf{u} \in (\text{ColSpan}(\mathbf{A}(y)) + \mathbf{b}(y))} |\mathbf{u}\rangle.$$

Finally, set $\text{pk} := y$ and $|\text{sk}\rangle := \frac{1}{2^{\frac{n-r}{2}}} \sum_{\mathbf{u} \in (\text{ColSpan}(\mathbf{A}(y)) + \mathbf{b}(y))} |\mathbf{u}\rangle$.

- $\sigma \leftarrow \text{Sign}^{\mathcal{P}^{-1}, \mathcal{D}}(\text{pk}, |\text{sk}\rangle, m \in \{0,1\})$: Parse $\text{pk} := y \in \{0,1\}^r$ and parse $|\text{sk}\rangle$ as the superposition $\frac{1}{2^{\frac{n-r}{2}}} \sum_{\mathbf{u} \in (\text{ColSpan}(\mathbf{A}(y)) + \mathbf{b}(y))} |\mathbf{u}\rangle$. Signatures on $m \in \{0,1\}$ are elements in $\text{ColSpan}(\mathbf{A}(y)) + \mathbf{b}(y)$ that start with m . Note that the oracle $\mathcal{P}^{-1}$ allows membership check in $\text{ColSpan}(\mathbf{A}) + \mathbf{b}(y)$ by simply checking that its output isn't $\perp$, and the oracle $\mathcal{D}$ allows membership check in $\text{ColSpan}(\mathbf{A}(y))^\perp$.

To conclude, we have a coset state $\frac{1}{2^{\frac{n-r}{2}}} \sum_{\mathbf{u} \in (\text{ColSpan}(\mathbf{A}(y)) + \mathbf{b}(y))} |\mathbf{u}\rangle$ and we want to project it to be a uniform superposition over elements in the coset which start with the bit m . For

this, we have access to membership check in the coset and its dual. This means we have the same setting for signing as in the previous work of [AGKZ20], and can use fixed-point Grover search, as described in Theorem 5 of [AGKZ20], to generate a valid signature.

- $\text{Ver}^{\mathcal{P}^{-1}}(\text{pk}, m \in \{0, 1\}, \sigma) \in \{0, 1\}$: Parse pk as $y \in \{0, 1\}^r$ and parse σ as $\mathbf{u}_\sigma \in \mathbb{Z}_2^k$. Output 1 iff both, $\mathcal{P}^{-1}(y, \mathbf{u}_\sigma) \neq \perp$ and the first bit of $\mathbf{u}_\sigma$ equals m .

Security of One-Shot Signatures. The above OSS construction has strong unforgeability. Specifically, one can see that *any* pair of two different signatures (even for the same message $m \in \{0, 1\}$) for the same public key pk , will constitute a collision in the function H from the oracles $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}$. However, in Theorem 30 we proved the collision resistance of H . This means that the above OSS has strong unforgeability.

4.1 Bloating the Dual

Let $\mathcal{O}'_{n,r,k,s}$ denote the following distribution over $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'$. The primal oracles $\mathcal{P}, \mathcal{P}^{-1}$ are defined identically to $\mathcal{O}_{n,r,k}$. However, now, for $s \leq n - r$, we let $\mathbf{A}(y)^{(0)} \in \mathbb{Z}_2^s$ denote the first s columns of $\mathbf{A}(y) \in \mathbb{Z}_2^{n-r}$ and $\mathbf{A}(y)^{(1)} \in \mathbb{Z}_2^{n-r-s}$ denote the remaining $n - r - s$ columns. Then, define $\mathcal{D}'$ as the oracle:

$$\mathcal{D}'(y, \mathbf{v}) = \begin{cases} 1 & \text{if } \mathbf{v}^T \cdot \mathbf{A}(y)^{(1)} = 0^{n-r-s} \\ 0 & \text{otherwise} \end{cases}$$

Observe that if $\mathbf{v}^T \cdot \mathbf{A}(y) = 0^{n-r}$, then $\mathbf{v}^T \cdot \mathbf{A}(y)^{(1)} = 0^{n-r-s}$. Thus $\mathcal{D}'$ accepts all points that are accepted by $\mathcal{D}$, but also accepts additional points as well, namely those for which $\mathbf{v}^T \cdot \mathbf{A}(y)^{(0)} \neq 0^s$ but $\mathbf{v}^T \cdot \mathbf{A}(y)^{(1)} = 0^{n-r-s}$. We call this action, of moving from $\mathcal{D}$ to a more relaxed oracle $\mathcal{D}'$, "bloating the dual".

Lemma 32. Suppose there is an oracle aided q -query quantum algorithm $\mathcal{A}$ such that

$$\Pr \left[(y_0 = y_1) \wedge (x_0 \neq x_1) : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \mathcal{O}_{n,r,k} \\ (x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}} \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \right] \geq \epsilon .$$

Also, let $s \leq n - r$, $s - (n - r - s) := s'$ such that,

1. $\frac{k^8 \cdot q^7 \cdot s}{\sqrt{2^{n-r-s}}} \leq o(\epsilon^4)$,
2. $\frac{k^2 \cdot q^3 \cdot (n-r-s)}{2^{s'}} \leq o(\epsilon^2)$.

Then,

$$\Pr \left[\begin{array}{l} (y_0 = y_1 := y) \wedge \\ (\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}(y)^{(1)}) \end{array} : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \mathcal{O}'_{n,r,k,s} \\ (x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'} \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \right] \geq \frac{\epsilon}{2^6 \cdot k^2} .$$

Note that $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}(y)^{(1)})$ means in particular that $\mathbf{u}_0, \mathbf{u}_1$, and hence x_0, x_1 , are distinct (this follows because for every $y \in \mathbb{Z}_2^r$, the mapping between preimages x and vectors $\mathbf{u}_x \in (\text{ColSpan}(\mathbf{A}(y)) + \mathbf{b}(y))$ is bijective). Thus, the second expression means that $\mathcal{A}$ is finding collisions, but these collisions satisfy an even stronger requirement.

Proof. Assume there is an oracle-aided q -query quantum algorithm $\mathcal{A}$ that given oracle access to $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \mathcal{O}_{n,r,k}$ outputs a pair (x_0, x_1) of n -bit strings. Denote by ϵ the probability that x_0, x_1 are both distinct and collide in $H(\cdot)$ (i.e., their y -values are identical). We next define a sequence of hybrid experiments, outputs and success probabilities for them, and explain why the success probability in each consecutive pair is statistically close.

- **Hyb₀**: The original execution of $\mathcal{A}$.

The process **Hyb₀** is the above execution of $\mathcal{A}$ on input oracles $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D})$. We define the output of the process as (x_0, x_1) and the process execution is considered as successful if x_0, x_1 are both distinct and collide in $H(\cdot)$. By definition, the success probability of **Hyb₀** is ϵ .

- **Hyb₁**: Simulating the oracles using only a bounded number of cosets $(\mathbf{A}(y), \mathbf{b}(y))$, by using small-range distribution.

Consider the function F which samples for every $y \in \mathbb{Z}_2^r$ the i.i.d. coset description $(\mathbf{A}(y), \mathbf{b}(y))$. These cosets are then used in all three oracles $\mathcal{P}, \mathcal{P}^{-1}$ and $\mathcal{D}$. The difference between the current hybrid and the previous hybrid is that we swap F with F' which is sampled as follows: We set $R := (300 \cdot q^3) \cdot \frac{2^r \cdot k^2}{\epsilon}$ and for every $y \in \mathbb{Z}_2^r$ we sample a uniformly random $i_y \leftarrow [R]$, then sample for every $i \in [R]$ a coset $(\mathbf{A}_i \in \mathbb{Z}_2^{k \times (n-r)}, \mathbf{b}_i \in \mathbb{Z}_2^k)$ as usual. For $y \in \mathbb{Z}_2^r$ we define $F'(y) := (\mathbf{A}_{i_y}, \mathbf{b}_{i_y})$.

By Theorem A.6 from [AGQY22], it follows that for every quantum algorithm making at most q queries and tries to distinguish between F and F' , the distinguishing advantage is bounded by $\frac{300 \cdot q^3}{R} < \frac{\epsilon}{8}$, which means in particular that the outputs of this hybrid and the previous one has statistical distance bounded by $\frac{\epsilon}{8}$. It follows in particular that the success probability of the current hybrid is $:= \epsilon_1 \geq \epsilon - \frac{\epsilon}{8} = \frac{7 \cdot \epsilon}{8}$.

- **Hyb₂**: Relaxing dual verification oracle to accept a larger subspace, by information-theoretical subspace hiding.

The difference between the current hybrid and the previous hybrid is that in the current hybrid we make the dual verification oracle $\mathcal{D}$ more relaxed, and accept more strings. Specifically, recall that as part of sampling our oracles $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D})$ we sample the function F' and in particular we sample R i.i.d. cosets: $(\mathbf{A}_i \in \mathbb{Z}_2^{k \times (n-r)}, \mathbf{b}_i \in \mathbb{Z}_2^k)_{i \in [R]}$. Recall that in the previous hybrid, given

input $(y \in \mathbb{Z}_2^r, \mathbf{v} \in \mathbb{Z}_2^k)$, the oracle $\mathcal{D}$ accepts iff $\mathbf{v} \in \text{ColSpan}(\mathbf{A}_{i_y})^\perp$. The change we make to the current hybrid is the following and applies only to the dual verification oracle $\mathcal{D}$: For each $i \in [R]$, we sample $T_i^\perp$ which is a uniformly random, $(k - n + r + s)$ -dimensional superspace of $\text{ColSpan}(\mathbf{A}_i)^\perp$ (which has dimension $k - n + r$). When we execute $\mathcal{D}$ we check membership in $T_i^\perp$ rather than in the more restrictive, $(k - n + r)$ -dimensional $\text{ColSpan}(\mathbf{A})^\perp$.

By Lemma 20, due to $k - (k - (n - r)) - s = n - r - s$, for every $i \in [R]$, changing $\mathcal{D}$ to check for membership in $T_i^\perp$ instead of $\text{ColSpan}(\mathbf{A}_i)^\perp$, is $\left(\frac{q \cdot s}{\sqrt{2^{n-r-s}}}\right)$ -indistinguishable, for any q -query algorithm. Since we use the above indistinguishability R times, we get $R \cdot \left(\frac{q \cdot s}{\sqrt{2^{n-r-s}}}\right)$ -indistinguishability. It follows that the success probability of the current hybrid is $:= \epsilon_2 \geq \epsilon_1 - R \cdot \left(\frac{q \cdot s}{\sqrt{2^{n-r-s}}}\right) \geq \frac{7 \cdot \epsilon}{8} - O\left(\frac{k^2 \cdot q^4 \cdot s \cdot \frac{1}{\epsilon}}{\sqrt{2^{n-r-s}}}\right)$, which in turn by Condition 1 in our Lemma's statement, is at least $\frac{3 \cdot \epsilon}{4}$.

- **Hyb₃**: Asking for the sum of collisions to be outside of T_i , by using dual-subspace anti-concentration.

In the current hybrid we change the success predicate of the experiment. Recall that as part of sampling the oracles in the previous hybrid, we sample R i.i.d. cosets $(\mathbf{A}_i, \mathbf{b}_i)_{i \in [R]}$ which are used in all three oracles $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D})$. We then sample R i.i.d. $(k - n + r + s)$ -dimensional superspaces $(T_i^\perp)_{i \in [R]}$ of the R corresponding duals $(\text{ColSpan}(\mathbf{A}_i)^\perp)_{i \in [R]}$. The change we make to the success predicate is the following: At the end of the execution we get a pair (x_0, x_1) from $\mathcal{A}$. We define the process as successful if $y_0 = y_1 := y$ and also $(\mathbf{u}_0 - \mathbf{u}_1) \notin T_{i_y}$, rather than only asking that $x_0 \neq x_1$.

Let ϵ_3 be the success probability of the current hybrid and note that $\mathcal{A}$ finds collisions with probability ϵ_2 in the previous hybrid **Hyb₂** (and since this hybrid is no different, the same goes for the current hybrid). For every value $i \in [R]$ denote by $\epsilon_2^{(i)}$ the probability to find a collision in index i , or formally, that $y_0 = y_1 := y$, $x_0 \neq x_1$ and also $i_y = i$. Observe that in such an event we also have $(\mathbf{u}_0 - \mathbf{u}_1) \in S_{i_y}$. We deduce $\sum_{i \in [R]} \epsilon_2^{(i)} = \epsilon_2$. Let L be a subset of indices $i \in [R]$ such that $\epsilon_2^{(i)} \geq \frac{\epsilon_2}{2 \cdot R}$ and note that $\sum_{i \in L} \epsilon_2^{(i)} \geq \frac{\epsilon_2}{2}$. For every value $i \in [R]$ also denote by $\epsilon_3^{(i)}$ the probability that $y_0 = y_1 := y$, $(\mathbf{u}_0 - \mathbf{u}_1) \notin T_{i_y}$ and also $i_y = i$. We deduce $\sum_{i \in [R]} \epsilon_3^{(i)} = \epsilon_3$.

We would now like to use Lemma 23, so we make sure that we satisfy its requirements. Let any $i \in L$, we know that by definition $\epsilon_2^{(i)} \geq \frac{\epsilon_2}{2 \cdot R}$ and also recall that $\epsilon_2 \geq \frac{3 \cdot \epsilon}{4}$, $R := (300 \cdot q^3) \cdot \frac{2^7 \cdot k^2}{\epsilon}$ and thus

$$\epsilon_2^{(i)} \geq \frac{\epsilon_2}{2 \cdot R} \geq \frac{3 \cdot \epsilon}{8} \cdot \frac{1}{R} \geq \Omega\left(\frac{\epsilon^2}{q^3 \cdot k^2}\right).$$

Let $s' := s - (n - r - s)$ and for any $i \in L$ let $\ell_i := \frac{k^2}{\epsilon_2^{(i)}} \leq O\left(\frac{k^4 \cdot q^3}{\epsilon^2}\right)$. Note that by our Lemma 32 statement's conditions, by Condition 2 we have (1) $\frac{(n-r-s)}{2^{s'}} \leq \frac{\epsilon_2^{(i)}}{2}$ and by Condition 1 we have (2) $\frac{q \cdot \ell_i^2 \cdot s}{\sqrt{2^{n-r-s}}} \leq \frac{1}{2}$. Since this satisfies Lemma 23, it follows that for every $i \in L$ we have $\epsilon_3^{(i)} \geq \frac{\epsilon_2^{(i)}}{16 \cdot k^2}$. It follows that

$$\epsilon_3 = \sum_{i \in [R]} \epsilon_3^{(i)} \geq \sum_{i \in L} \epsilon_3^{(i)} \geq \sum_{i \in L} \frac{\epsilon_2^{(i)}}{16 \cdot k^2} \geq \frac{\left(\frac{\epsilon_2}{2}\right)}{16 \cdot k^2} \geq \frac{3 \cdot \epsilon}{2^7 \cdot k^2}.$$

- **Hyb₄**: For every $i \in [R]$, de-randomizing T_i and defining it as the column span of $\mathbf{A}_i^{(1)} \in \mathbb{Z}_2^{k \times (n-(r+s))}$, the last $n - (r + s)$ columns of the matrix $\mathbf{A}_i$, by using the random permutation Π and random function F .

This hybrid is the same as the previous, with one change: For every $i \in [R]$, after sampling the coset $(\mathbf{A}_i, \mathbf{b}_i)$, we will not continue to randomly sample $T_i^\perp$ (which, previously, was a uniformly random $((k - (n - r)) + s)$ -dimensional superspace of $\text{ColSpan}(\mathbf{A}_i)^\perp$) and simply define $T_i := \text{ColSpan}(\mathbf{A}_i^{(1)})$ such that $\mathbf{A}_i^{(1)} \in \mathbb{Z}_2^{k \times (n-r-s)}$ is defined to be the last $n - r - s$ columns of the matrix $\mathbf{A}_i \in \mathbb{Z}_2^{k \times (n-r)}$. We will define an intermediate hybrid **Hyb_{3.1}** and then explain why **Hyb₃** $\equiv$ **Hyb_{3.1}** $\equiv$ **Hyb₄**.

- **Using the random permutation Π** . For every $i \in [R]$ consider the superspace $T_i^\perp$, which has $(k - n + r) + s$ dimensions. Accordingly, the dual T_i has $k - ((k - n + r) + s)$ dimensions, which equals $n - r - s$. For $i \in [R]$, $j \in [n - r - s]$, denote by $\mathbf{t}_{i,j} \in \mathbb{Z}_2^k$ the j -th basis vector

for the subspace T_i . Now, for every $i \in [R]$ let $M_i \in \mathbb{Z}_2^{(n-r) \times (n-r)}$ a full-rank matrix that represents the coordinates vectors of the basis vectors of T_i . Formally, M_i is such that the j -th column of M_i , denoted $M_{i,j} \in \mathbb{Z}_2^{n-r}$, satisfies $\mathbf{A}_i \cdot M_{i,j} = \mathbf{t}_{i,j} \in \mathbb{Z}_2^k$.

In $\text{Hyb}_{3,1}$, we define the permutation Γ over $\{0,1\}^n$ defined as follows: For an input $s \in \{0,1\}^n$, it takes the left r bits denoted $y \in \mathbb{Z}_2^r$, computes $i_y \in [R]$, then applies matrix multiplication by M_{i_y} to the remaining right $n-r$ bits. Observe that since M_i is full rank for all i , then Γ is indeed a permutation. The change we make from Hyb_3 to $\text{Hyb}_{3,1}$ is that in the current hybrid we apply Γ to the *output* of Π inside the execution of a query to $\mathcal{P}$, and apply Γ^{-1} to the *input* of Π^{-1} inside the execution of a query to $\mathcal{P}^{-1}$. Note that for a truly random n -bit permutation Π , concatenating any fixed permutation Γ like this is statistically equivalent to just computing Π and Π^{-1} , thus the outputs (and in particular success probabilities) between Hyb_3 and $\text{Hyb}_{3,1}$ are identical.

- **Using the random function F .** In Hyb_4 , we stop applying the permutation Γ to the output of Π (and likewise stop applying Γ^{-1} to the input of Π^{-1}), and also stop sampling $T_i^\perp$ and simply define it as $\text{ColSpan}(\mathbf{A}_i^{(1)})^\perp$. Note that for every choice of Π , the following two distributions over oracles are statistically equivalent. (1) for every $i \in [R]$ sample $\mathbf{A}_i$ uniformly at random, then sample the superspace $T_i^\perp$ uniformly at random, and then set Γ accordingly and concatenate it with Π as we did in the above $\text{Hyb}_{3,1}$. (2) Just sample $\mathbf{A}_i$ uniformly at random for every $i \in [R]$, and then set $T_i := \text{ColSpan}(\mathbf{A}_i^{(1)})$. It follows that due to the fact that F is a random function (or more precisely, because for every $i \in [R]$ we choose a uniformly random coset $(\mathbf{A}_i, \mathbf{b}_i)$) then the outputs of $\text{Hyb}_{3,1}$ and Hyb_4 are statistically equivalent and in particular the success probability in both cases is the same.

It follows that the success probability ϵ_4 in Hyb_4 equals the success probability ϵ_3 in Hyb_3 .

- **Hyb_5 :** Moving back to using an exponential number of cosets, by using small-range distribution again.

We rewind the process of sampling an R -small range distribution version of F , and use F as a standard random function. By the same argument for the indistinguishability between Hyb_0 and Hyb_1 , the output of the current process has statistical distance bounded by $\frac{300 \cdot q^3}{R} = \frac{\epsilon}{2^7 \cdot k^2}$, which means in particular that the outputs of this hybrid and the previous hybrid has statistical distance bounded by $\frac{\epsilon}{2^7 \cdot k^2}$. It follows that the success probability of the current hybrid is

$$:= \epsilon_5 \geq \epsilon_4 - \frac{\epsilon}{2^7 \cdot k^2} \geq \frac{3 \cdot \epsilon}{2^7 \cdot k^2} - \frac{\epsilon}{2^7 \cdot k^2} = \frac{\epsilon}{2^6 \cdot k^2} .$$

To conclude, note that the process Hyb_5 is exactly the process where $\mathcal{A}$ executes on input oracle sampled from $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \mathcal{O}'_{n,r,k,s}$. This finishes our proof. $\square$

4.2 Simulating the Dual

In this section we prove the following lemma.

Lemma 33. Suppose there is an oracle aided q -query quantum algorithm $\mathcal{A}$ such that

$$\Pr \left[\begin{array}{l} y_0 = y_1 := y, \\ (\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}(y)^{(1)}) \end{array} : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \mathcal{O}'_{n,r,k,s} \\ (x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'} \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \right] \geq \epsilon .$$

Then, there is an oracle aided q -query quantum algorithm $\mathcal{B}$ such that

$$\Pr \left[\begin{array}{l} (\bar{y}_0 = \bar{y}_1) \wedge (\bar{x}_0 \neq \bar{x}_1) : \\ (\bar{x}_0, \bar{x}_1) \leftarrow \mathcal{B}^{\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}} \\ (\bar{y}_b, \bar{\mathbf{u}}_b) \leftarrow \bar{\mathcal{P}}(\bar{x}_b) \end{array} \right] \geq \epsilon .$$

Proof. We first describe the actions of the algorithm $\mathcal{B}$ (which will use the code of $\mathcal{A}$ as part of its machinery) and then argue why it breaks collision resistance with the appropriate probability. Given oracle access to $\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}$ which comes from $(\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}, \bar{\mathcal{D}}) \leftarrow \mathcal{O}_{r+s, r, k-(n-r-s)}$, the algorithm $\mathcal{B}$ does the following:

- Sample a random function $F_{\mathbf{C}}$ that outputs some sufficient (polynomial) amount of random bits on an r -bit input, and sample a random n -bit permutation Γ . Define the following oracles.
- $(y \in \mathbb{Z}_2^r, \mathbf{u} \in \mathbb{Z}_2^k) \leftarrow \mathcal{P}(x \in \mathbb{Z}_2^n)$:
 - $(\bar{x} \in \mathbb{Z}_2^{r+s}, \tilde{x} \in \mathbb{Z}_2^{n-r-s}) \leftarrow \Gamma(x)$.
 - $(y \in \mathbb{Z}_2^r, \bar{\mathbf{u}} \in \mathbb{Z}_2^{k-(n-r-s)}) \leftarrow \bar{\mathcal{P}}(\bar{x})$.
 - $(\mathbf{C}(y) \in \mathbb{Z}_2^{k \times k}, \mathbf{d}(y) \in \mathbb{Z}_2^{n-r-s}) \leftarrow F_{\mathbf{C}}(y)$.
 - $\mathbf{u} \leftarrow \mathbf{C}(y) \cdot \begin{pmatrix} \bar{\mathbf{u}} \\ \tilde{x} + \mathbf{d}(y) \end{pmatrix}$.
- $(x \in \mathbb{Z}_2^n) \leftarrow \mathcal{P}^{-1}(y \in \mathbb{Z}_2^r, \mathbf{u} \in \mathbb{Z}_2^k)$:
 - $(\mathbf{C}(y) \in \mathbb{Z}_2^{k \times k}, \mathbf{d}(y) \in \mathbb{Z}_2^{n-r-s}) \leftarrow F_{\mathbf{C}}(y)$.
 - $\begin{pmatrix} \bar{\mathbf{u}} \\ \tilde{x} \end{pmatrix} \leftarrow \mathbf{C}(y)^{-1} \cdot \mathbf{u} - \begin{pmatrix} 0^{k-(n-r-s)} \\ \mathbf{d}(y) \end{pmatrix}$.
 - $(\bar{x} \in \mathbb{Z}_2^{r+s}) \leftarrow \bar{\mathcal{P}}^{-1}(y, \bar{\mathbf{u}})$.
 - $x \leftarrow \Gamma^{-1}(\bar{x}, \tilde{x})$.
- $\mathcal{D}'(y \in \mathbb{Z}_2^r, \mathbf{v} \in \mathbb{Z}_2^k) \in \{0, 1\}$:
 - $(\mathbf{C}(y) \in \mathbb{Z}_2^{k \times k}, \mathbf{d}(y) \in \mathbb{Z}_2^{n-r-s}) \leftarrow F_{\mathbf{C}}(y)$.
 - $\mathbf{A}^{(1)}(y) := \text{last } n - r - s \text{ columns of } \mathbf{C}(y)$.
 - Output 1 iff $\mathbf{v}^T \cdot \mathbf{A}^{(1)}(y) = \mathbf{0}^{n-r-s}$.

The remainder of the reduction is simple: $\mathcal{B}$ executes $(x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'}$ and then $(\bar{x}_b, \tilde{x}_b) \leftarrow \Gamma(x_b)$ and outputs $(\bar{x}_0, \bar{x}_1)$. Assume that the output of $\mathcal{A}$ satisfies $y_0 = y_1 := y$ and also $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}(y)^{(1)})$, and recall that $\mathbf{A}(y)^{(1)} \in \mathbb{Z}_2^{k \times (n-r-s)}$ are the last $n-r-s$ columns of the matrix $\mathbf{A}(y) \in \mathbb{Z}_2^{k \times (n-r)}$, which is generated by the reduction. We explain why it is necessarily the case that $\bar{x}_0 \neq \bar{x}_1$.

First note that due to how we defined the reduction, $\mathbf{A}(y) := \mathbf{C}(y) \cdot \begin{pmatrix} \bar{\mathbf{A}}(y) & \\ & \mathbf{I}_{n-r-s} \end{pmatrix}$, where $\bar{\mathbf{A}}(y) \in \mathbb{Z}_2^{(k-(n-r-s)) \times s}$ is the matrix arising from the oracles $\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}$ and $\mathbf{I}_{n-r-s} \in \mathbb{Z}_2^{(n-r-s) \times (n-r-s)}$ is the identity matrix of dimension $n-r-s$. Also note that because $\mathbf{C}(y), \bar{\mathbf{A}}(y)$ are full rank then $\mathbf{A}(y)$ is full rank. Now, since $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}(y)^{(1)})$ and since $\mathbf{A}(y)^{(1)}$ are the last $n-r-s$ columns of $\mathbf{A}(y)$, it follows that if we consider the coordinates vector $\mathbf{x} \in \mathbb{Z}_2^{n-r}$ of $(\mathbf{u}_0 - \mathbf{u}_1)$ with respect to $\mathbf{A}(y)$, the first s elements are not 0's. By linearity of matrix multiplication it follows that if we look at each of the two coordinates vectors $\mathbf{x}_0, \mathbf{x}_1$ (each has $n-r$ bits) for $\mathbf{u}_0, \mathbf{u}_1$, respectively, somewhere in the first s bits, they differ. Now, recall how we obtain the first s bits of $\mathbf{x}_b$ – this is exactly by applying $\bar{\Pi}$ (the permutation on $\{0, 1\}^{r+s}$ arising from the oracles $\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}$) to $\bar{x}_b$ and taking the last s bits of the output. Since these bits differ in the output of the permutation, then the preimages have to differ, i.e., $\bar{x}_0 \neq \bar{x}_1$.

Define $\epsilon_{\mathcal{B}}$ as the probability that the output of $\mathcal{A}$ indeed satisfies $y_0 = y_1 := y$ and also $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}^{(1)}(y))$, and it remains to give a lower bound for the probability $\epsilon_{\mathcal{B}}$. We do this by a sequence of hybrids, eventually showing that the oracle which $\mathcal{B}$ simulates to $\mathcal{A}$ is indistinguishable from an oracle sampled from $\mathcal{O}'_{n,r,k,s}$. More precisely, each hybrid describes a process, it has an output, and a success predicate on the output.

- **Hyb₀**: The above distribution $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \mathcal{B}^{\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}}$, simulated to the algorithm $\mathcal{A}$.

The first hybrid is where $\mathcal{B}$ executes $\mathcal{A}$ by the simulation described above. The output of the process is the output (x_0, x_1) of $\mathcal{A}$. The process execution is considered as successful if $y_0 = y_1 := y$ and $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}(y)^{(1)})$.

- **Hyb₁**: Not applying the inner permutation $\bar{\Pi}$ (which comes from the oracles $\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}$), by using the random permutation Γ .

Let $\bar{\Pi}$ the permutation on $\{0, 1\}^{r+s}$ that's inside $\bar{\mathcal{P}}$. In the previous hybrid we apply the n -bit permutation Γ to the input $x \in \mathbb{Z}_2^n$ and then proceed to apply the inner permutation $\bar{\Pi}$ to the first (i.e. leftmost) $r+s$ output bits of the first permutation Γ (we also apply Γ^{-1} to the output of the inverse of the inner permutation, in the inverse oracle $\mathcal{P}^{-1}$). The change we make to the current hybrid is that we simply apply only Γ and discard the inner permutation and its inverse. Since a random permutation concatenated with any permutation distributes identically to a random permutation, the current hybrid is statistically equivalent to the previous and in particular the output of this process distributes identically to the output of the previous, and so does the success probability.

- **Hyb₂**: For every $y \in \mathbb{Z}_2^r$, taking $\mathbf{A}(y)$ to be the direct output of F , by using the randomness of the random function.

In order to describe the change between the current and previous hybrid we first recall the structure of the oracles from the previous hybrid: Observe that in the previous hybrid, for every $y \in \mathbb{Z}_2^r$ we defined $\mathbf{A}(y) := \mathbf{C}(y) \cdot \begin{pmatrix} \overline{\mathbf{A}}(y) \\ \mathbf{I}_{n-r-s} \end{pmatrix}$, where $\mathbf{C}(y) \in \mathbb{Z}_2^{k \times k}$ is the output of $F_{\mathbf{C}}(y)$ and $\overline{\mathbf{A}}(y) \in \mathbb{Z}_2^{(k-(n-r-s)) \times s}$ is the output of the inner random function $\overline{F}$ (which comes from the inside of the oracles $(\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1})$). In the current hybrid we are going to ignore the inner random function $\overline{F}$, its generated matrix $\overline{\mathbf{A}}(y)$ and also the pair $\mathbf{C}(y), \mathbf{d}(y)$, sample a fresh random function $F_{\mathbf{A}}$ at the beginning of the process, and on query y generate $(\mathbf{A}(y), \mathbf{b}(y)) \leftarrow F_{\mathbf{A}}(y)$, for $\mathbf{A}(y) \in \mathbb{Z}_2^{k \times (n-r)}$, $\mathbf{b}(y) \in \mathbb{Z}_2^k$.

To see why the two distributions are indistinguishable, note that the following two ways to sample $\mathbf{A}(y)$, are statistically equivalent: (1) For every $y \in \mathbb{Z}_2^r$, the matrix $\mathbf{A}(y)$ is generated by sampling a random full-rank matrix $\mathbf{C}(y) \in \mathbb{Z}_2^{k \times k}$ and letting $\mathbf{A}(y)$ be $\mathbf{C}(y) \cdot \begin{pmatrix} \overline{\mathbf{A}}(y) \\ \mathbf{I}_{n-r-s} \end{pmatrix}$. (2) For every $y \in \mathbb{Z}_2^r$ just sample a full-rank matrix $\mathbf{A}(y) \in \mathbb{Z}_2^{k \times (n-r)}$. Since we are using random functions and in the previous hybrid we are sampling $\mathbf{A}(y)$ according to (1) and in the current hybrid we are sampling $\mathbf{A}(y)$ according to (2), the outputs of the two hybrids distribute identically.

Finalizing the reduction. Observe that the distribution generated in the above Hyb_2 is exactly an oracle sampled from $\mathcal{O}'_{n,r,k,s}$. From the lemma's assumptions, the success probability for Hyb_2 is thus ϵ . Since we also showed that the hybrids have identical success probabilities, it follows that $\epsilon_{\mathcal{B}} = \epsilon$, which finishes our proof. $\square$

We conclude this section by stating the following Theorem, which is obtained as a direct corollary from Lemmas 32 and 33.

Theorem 34. *Suppose there is an oracle aided q -query quantum algorithm $\mathcal{A}$ such that*

$$\Pr \left[x_0 \neq x_1 \wedge H(x_0) = H(x_1) : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \mathcal{O}_{n,r,k} \\ (x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}} \end{array} \right] \geq \epsilon .$$

Also, let $s \leq n - r$, $s - (n - r - s) := s'$ such that,

1. $\frac{k^8 \cdot q^7 \cdot s}{\sqrt{2^{n-r-s}}} \leq o(\epsilon^4),$
2. $\frac{k^2 \cdot q^3 \cdot (n-r-s)}{2^{s'}} \leq o(\epsilon^2).$

Then, there is an oracle aided q -query quantum algorithm $\mathcal{B}$ such that

$$\Pr \left[x_0 \neq x_1 \wedge H(x_0) = H(x_1) : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \mathcal{O}_{r+s, r, k-(n-r-s)} \\ (x_0, x_1) \leftarrow \mathcal{B}^{\mathcal{P}, \mathcal{P}^{-1}} \end{array} \right] \geq \frac{\epsilon}{2^6 \cdot k^2} .$$

4.3 Hardness of the Dual-free Case from 2-to-1 Collision-Resistance

We start with defining coset partition functions, which are an object we will use in order to show that collision finding in the dual-free case is at least as hard as finding collisions in 2-to-1 random functions.

Definition 35 (Coset Partition Functions). For $n, \ell \in \mathbb{N}$ such that $\ell \leq n$ we say a function $Q : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is a (n, m, ℓ) -coset partition function if, for each y in the image of Q , the pre-image set $Q^{-1}(y)$ has size 2^ℓ and is a coset of a linear space of dimension ℓ . We allow different pre-image sets to be cosets of different linear spaces.

From Dual-free to Coset Partition Functions. We next show that finding collisions in the dual-free case is no easier than finding collisions in *worst-case* coset partition functions.

Theorem 36. Let $k \geq n \geq r$. Suppose there is an oracle aided q -query quantum algorithm $\mathcal{A}$ such that

$$\Pr \left[(y_0 = y_1) \wedge (x_0 \neq x_1) : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \mathcal{O}_{n,r,k} \\ (x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}} \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \right] \geq \epsilon .$$

Then there is an oracle aided q -query quantum algorithm $\mathcal{B}$ that given any $(n, r, n-r)$ -coset partition function Q , satisfies

$$\Pr [(Q(w_0) = Q(w_1)) \wedge (w_0 \neq w_1) : (w_0, w_1) \leftarrow \mathcal{B}^Q] \geq \epsilon .$$

Proof. $\mathcal{B}$ works as follows. Given oracle access to some $(n, r, n-r)$ -coset partition function Q , it chooses a random permutation $\Gamma : \{0, 1\}^n \rightarrow \{0, 1\}^n$, and for each y , it chooses a random full-column-rank matrix $\mathbf{C}_y \in \mathbb{Z}_2^{k \times n}$ (which is possible since we assume $k \geq n$) and random vector $\mathbf{d}_y \in \mathbb{Z}_2^k$. It then runs $\mathcal{A}$, simulating the oracles $\mathcal{P}, \mathcal{P}^{-1}$ as follows:

The oracle $\mathcal{P}(x)$:

- $y \leftarrow Q(\Gamma(x))$.
- Output $(y, \mathbf{C}_y \cdot \Gamma(x) + \mathbf{d}_y)$.

We now claim that $\mathcal{P}$ is correctly distributed. To do so, we will first define an augmented function $Q' : \{0, 1\}^n \rightarrow \{0, 1\}^n$. On input $\mathbf{z}$, the n -bit output of $Q'(\mathbf{z})$ consists of two parts. The first r bits are set to $y = Q(\mathbf{z})$. The preimage set $Q^{-1}(y)$ is then a coset, which can be described as the set $\{\overline{\mathbf{A}}_y \cdot \mathbf{r} + \overline{\mathbf{b}}_y\}$ as $\mathbf{r}$ ranges over $\mathbb{Z}_2^{n-r}$ (where $\overline{\mathbf{A}}_y, \overline{\mathbf{b}}_y$ are both unknown to the reduction algorithm $\mathcal{B}$). Here, $\overline{\mathbf{A}}_y \in \mathbb{Z}_2^{n \times (n-r)}$ has full column-rank and $\overline{\mathbf{b}} \in \mathbb{Z}_2^n$. Define the function $\overline{J}(\mathbf{z})$ that outputs the unique vector in $\mathbb{Z}_2^{n-r}$ such that $\mathbf{z} = \overline{\mathbf{A}}_y \cdot \overline{J}(\mathbf{z}) + \overline{\mathbf{b}}_y$. Then define $Q'(\mathbf{z}) = (Q(\mathbf{z}), \overline{J}(\mathbf{z}))$. Note that Q' is not efficiently computable without knowing $\overline{\mathbf{A}}_y, \overline{\mathbf{b}}_y$, but here we will not need it to be. Intuitively, the reason that we do not need $\overline{J}(\cdot)$ to be efficiently computable is because whenever we simulate the oracles $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D})$, the value $J(\cdot)$ is never output, it is only the connector between preimages x and vectors $\mathbf{u}_x$ in the coset $\text{ColSpan}(\mathbf{A}(y)) + \mathbf{b}(y)$. Notice that Q' is a function from $\mathbb{Z}_2^n$ to $\mathbb{Z}_2^n$, and it is moreover a permutation with $(Q')^{-1}(y, \mathbf{r}) = \overline{\mathbf{A}}_y \cdot \mathbf{r} + \overline{\mathbf{b}}_y$.

Observe that $\mathcal{B}$'s simulation of $\mathcal{P}$ is implicitly setting the following parameters

$$\begin{aligned} \Pi(x) &= Q'(\Gamma(x)) , & H(x) &= Q(\Gamma(x)) , & J(x) &= \overline{J}(\Gamma(x)) , \\ \mathbf{A}_y &= \mathbf{C}_y \cdot \overline{\mathbf{A}}_y , & \mathbf{b}_y &= \mathbf{C}_y \cdot \overline{\mathbf{b}}_y + \mathbf{d}_y . \end{aligned}$$

Thus, we must check that these quantities have the correct distribution. Indeed, for every Q , which in turn defines $(\overline{\mathbf{A}}_y, \overline{\mathbf{b}}_y)_{y \in \{0, 1\}^r}$, the function $Q'(x)$ is a permutation: Given $y \in \{0, 1\}^r$, $\mathbf{r} \in \mathbb{Z}_2^{n-r}$,

one can recover $\mathbf{z} \in \{0, 1\}^n$ as $\mathbf{z} = \overline{\mathbf{A}}_y \cdot \mathbf{r} + \overline{\mathbf{b}}_y$. Hence Π is a permutation since it is the composition of two permutations. Moreover, since one of the two permutations (Γ) is uniformly random, so is Π .

Now we look at the distribution of $\mathbf{A}_y, \mathbf{b}_y$. Recall that $\overline{\mathbf{A}}_y \in \mathbb{Z}_2^{n \times (n-r)}$ is a full-column-rank matrix, and $\mathbf{C}_y \in \mathbb{Z}_2^{k \times n}$ is a *random* full-column-rank matrix. Thus, $\mathbf{A}_y = \mathbf{C}_y \cdot \overline{\mathbf{A}}_y \in \mathbb{Z}_2^{k \times (n-r)}$ is also a random full-column-rank matrix.

Then we have that $\mathbf{b}_y = \mathbf{C}_y \cdot \overline{\mathbf{b}}_y + \mathbf{d}_y$ where $\mathbf{d}_y$ is random, meaning $\mathbf{b}_y$ is random. Thus, $\mathcal{P}$ has an identical distribution to that arising from $\mathcal{O}_{n,r,k}$.

The oracle $\mathcal{P}^{-1}(y, \mathbf{u})$:

- $x \leftarrow \begin{cases} \Gamma^{-1}(\mathbf{w}) & \exists \mathbf{w} \in \mathbb{Z}_2^n \text{ such that } \mathbf{C}_y \cdot \mathbf{w} + \mathbf{d}_y = \mathbf{u} \\ \perp & \text{if no such } \mathbf{w} \text{ exists} \end{cases}$
- Output $\begin{cases} x & \text{if } x \neq \perp \text{ and } Q(\Gamma(x)) = y \\ \perp & \text{if } x = \perp \text{ or } Q(\Gamma(x)) \neq y \end{cases}$

Observe that $\mathcal{P}^{-1}(\mathcal{P}(x)) = x$, and for all pairs $(y \in \{0, 1\}^r, \mathbf{u} \in \mathbb{Z}_2^k)$ that are not in the image of $\mathcal{P}$, we have $\mathcal{P}^{-1}(y, \mathbf{u}) = \perp$. Thus, $\mathcal{P}^{-1}$ is the uniquely-defined inverse of $\mathcal{P}$. Thus, since the distribution of $\mathcal{P}$ simulated by $\mathcal{B}$ exactly matches the distribution arising from $\mathcal{O}_{n,r,k}$, the same is true of the pairs $(\mathcal{P}, \mathcal{P}^{-1})$.

Finishing touches. Thus we saw that for every input $(n, r, n-r)$ -coset partition function Q , the algorithm $\mathcal{B}$ perfectly simulates the view of $\mathcal{A}$, which consists of the pair of oracles $\mathcal{P}, \mathcal{P}^{-1}$ that distribute according to $\mathcal{O}_{n,r,k}$. Hence, with probability ϵ , the algorithm $\mathcal{A}$ will produce a collision $x_0 \neq x_1$ such that $H(x_0) = H(x_1)$. It remains to explain what $\mathcal{B}$ does in order to obtain a collision in Q , for every collision in the simulated H . Given (x_0, x_1) , the reduction $\mathcal{B}$ will then compute and output $(w_0 = \Gamma(x_0), w_1 = \Gamma(x_1))$. Observe that if $x_0 \neq x_1$, then $w_0 \neq w_1$ since Γ is a permutation. Moreover, if $H(x_0) = H(x_1)$, then

$$Q(w_0) = Q(\Gamma(x_0)) = H(x_0) = H(x_1) = Q(\Gamma(x_1)) = Q(w_1) .$$

Hence, with probability at least ϵ , $\mathcal{B}$ will output a collision for Q . Notice that for each query that $\mathcal{A}$ makes to $\mathcal{P}(\cdot)$, the reduction $\mathcal{B}$ needs to make exactly one query to Q and the same goes for the inverse $\mathcal{P}^{-1}(\cdot)$. Thus $\mathcal{B}$ makes exactly q queries to Q . This completes the proof. $\square$

Collision-resistant Coset Partition Functions. We now show how to construct a collision resistant coset partition functions relative to an oracle. Our main theorem for this subsection is the following.

Theorem 37. *For any $n, \ell \in \mathbb{N}$ such that $\ell \mid n$, there exists a distribution over $(n, n-\ell, \ell)$ -coset partition functions H , such that any algorithm making q queries to H can only find collisions in H with probability at most $O\left(\frac{\ell^3 \cdot q^3}{2^{\frac{n}{\ell}}}\right)$.*

Proof. We will start with a much weaker goal of constructing distributions of collision-resistant 2-to-1 functions, that are shrinking by a single bit.

Lemma 38. *A random 2-to-1 function $H : \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}$ is collision resistant given quantum queries to H . In particular, any quantum algorithm making q queries has a $O(q^3/2^n)$ probability of producing a collision.*

Proof. The original work to lower-bound the quantum query complexity of collision resistance was [AS04]. That work proves that random 2-to-1 functions are collision-resistant, provided that the range is at least as large as the domain. They need the large range since their proof works via showing that the function is indistinguishable from a 1-to-1 function. But of course, a 1-to-1 function must have a range at least as large as domain. This is not quite good enough for us, as we insist on our H “losing” one bit.

Instead, we first point out that H can be extended into a permutation as follows. First let $J : \{0, 1\}^n \rightarrow \{0, 1\}$ be an arbitrary function which for any collision (x_0, x_1) of H , assigned random but distinct values to $J(x_0)$ and $J(x_1)$. Since H is 2-to-1, such a J only needs a 1-bit range. Then $\Pi(x) = (H(x), J(x))$ is a permutation, and if H is a random 2-to-1 function, then Π is a random permutation.

We will argue that H is collision-resistant *even given queries to Π* (but not Π^{-1}). Note that this is potentially stronger than giving access to H , as an algorithm can always ignore J . Suppose there is an oracle aided algorithm $\mathcal{A}$ which given oracle access to Π , finds a collision in H (that is, a collision in the first $n - 1$ output bits of Π) with probability ϵ .

To argue for the collision resistance given Π , we recall that for every q -query quantum algorithm having access to Π (but not its inverse), Π being a random permutation is $O\left(\frac{q^3}{2^n}\right)$ -indistinguishable from Π being a random function [Zha15]. So now let H be the first $n - 1$ bits of a random function $\Pi : \{0, 1\}^n \rightarrow \{0, 1\}^n$, and J be the last bit. Then we have that $\mathcal{A}^\Pi$ still finds a collision in this H with probability at least $\epsilon - O\left(\frac{q^3}{2^n}\right)$.

But now we observe that H and J are simply just independent random functions, and J can be simulated without making any queries to H (this can even be made efficient, but that is irrelevant since we only care about query counts). Thus, we obtain an algorithm $\mathcal{B}^H$ which finds a collision in a random function $H : \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}$ with probability at least $\epsilon - O\left(\frac{q^3}{2^n}\right)$. But we know that random functions are collision resistant, regardless of the relationship between domain and range. In particular, the probability of finding a collision in such random H is at most $O\left(\frac{q^3}{2^{n-1}}\right) = O\left(\frac{q^3}{2^n}\right)$ by [Zha15]. Thus, we can bound $\epsilon \leq O\left(\frac{q^3}{2^n}\right)$. $\square$

Extending to coset partition functions. Observe that a 2-to-1 function is trivially a coset partition function. Indeed, since the pre-image set always is a pair $\{x_0, x_1\}$, which is a coset of the 1-dimensional linear space $\{0, x_0 \oplus x_1\} \subseteq \mathbb{Z}_2^n$ over the field $\mathbb{Z}_2$. To conclude what we saw so far, a random 2-to-1 function $H : \{0, 1\}^n \rightarrow \{0, 1\}^{n-1}$ (that is shrinking by one bit) is collision resistant by Lemma 38, and is also a $(n, n - 1, 1)$ coset partition function, which proves Theorem 37 for the case $\ell = 1$.

We extend to (an almost) general ℓ as follows. We can let $H^\ell : \{0, 1\}^{n \cdot \ell} \rightarrow \{0, 1\}^{(n-1) \cdot \ell}$ be the function such that

$$H^\ell(x_1, \dots, x_\ell) := (H(x_1), \dots, H(x_\ell)) \quad .$$

It is straightforward that any collision for H^ℓ immediately gives a collision for H . Moreover, we can simulate any query to H^ℓ given ℓ queries to H . Thus, any algorithm making q queries to H^ℓ

can only find collisions in H^ℓ with probability at most $O\left(\frac{\ell^3 \cdot q^3}{2^n}\right)$.

Lastly, parallel repetition preserves coset partitions, since the pre-image sets of H^ℓ are just the direct sums of ℓ of the pre-image sets of H . Thus, we obtain the theorem by replacing $n \cdot \ell$ with n . $\square$

5 Permutable PRPs and Applications

In this section we develop a new concept of pseudo-random permutations, that will be useful for our obfuscation-based construction and may also be of independent interest. Quite roughly, our notion of a permutation will allow us to compose the permutation with another (fixed) permutation, while maintaining indistinguishability between the cases for whether the permutation was applied or not.

Definition 39. Let $G = \{G_N\}_{N \in \mathbb{N}}$ be a collection where each G_N is a set of permutations over $[N]$. An output-permutable PRP (OP-PRP) for G is a tuple of algorithms $(\Pi, \Pi^{-1}, \text{Permute}, \text{Eval}, \text{Eval}^{-1})$ with the following properties.

- **Efficient Permutations:** For any key $k \in \{0, 1\}^\lambda$ and any desired “block size” N , $\Pi(k, \cdot)$ is an efficiently computable permutation on $[N]$ with $\Pi^{-1}(k, \cdot)$ being its efficiently computable inverse.
- **Output Permuted:** $\text{Permute}(k, \Gamma, c)$ is a deterministic polynomial-time procedure which takes as input a key $k \in \{0, 1\}^\lambda$, the circuit description of a permutation Γ in G_N , and a bit c . It outputs a permuted key $k^{\Gamma, c}$.
- **Output Permuted Correctness:** For all $\lambda \in \mathbb{N}$, $k \in \{0, 1\}^\lambda$, $c \in \{0, 1\}$, $\Gamma \in G_N$ and $x, z \in [N]$,

$$\begin{aligned} \text{Eval}(k^{\Gamma, c}, x) &= \begin{cases} \Pi(k, x) & \text{if } c = 0 \\ \Gamma(\Pi(k, x)) & \text{if } c = 1 \end{cases} \\ \text{Eval}^{-1}(k^{\Gamma, c}, z) &= \begin{cases} \Pi^{-1}(k, z) & \text{if } c = 0 \\ \Pi^{-1}(k, \Gamma^{-1}(z)) & \text{if } c = 1 \end{cases} \end{aligned}$$

We call $k^{\Gamma, c}$ a permuted key.

- **Security:** For any interactive quantum polynomial-time adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\lambda)$ such that the following experiment with $\mathcal{A}$ outputs 1 with probability at most $\frac{1}{2} + \epsilon(\lambda)$:
 - $\mathcal{A}(1^\lambda)$ produces a block size N (in binary) and the description of a permutation $\Gamma \in G_N$.
 - The experiment chooses a random $k \leftarrow \{0, 1\}^\lambda$ and a random bit $c \in \{0, 1\}$. It gives $k^{\Gamma, c} \leftarrow \text{Permute}(k, \Gamma, c)$ to $\mathcal{A}$.
 - $\mathcal{A}$ produces a guess c' for c . The experiment outputs 1 if $c' = c$.

On the relations between parameters. Observe that $\lambda, N \in \mathbb{N}$ can be arbitrary, which means in particular that we can fix the block size N and vary the security parameter λ , or alternatively fix λ and vary N . This lets us construct permutable PRPs for small domains with high security, for example. We will often overload notation, and use the same symbol for both Π and Eval (and corresponding symbols for Π^{-1} and Eval^{-1}), when clear from context whether we are using a permuted or normal key. In this case, an OP-PRP would be a triplet $(\Pi, \Pi^{-1}, \text{Permute})$.

Sub-exponential Security. For arbitrary functions $f_0, f_1 : \mathbb{N} \rightarrow \mathbb{N}$, we say that an OP-PRP is $(f_0, \frac{1}{f_1})$ -secure if in the above the security part of the definition, we ask that the indistinguishability holds for every adversary of size $\leq f_0(\lambda)$ and we swap $\epsilon(\lambda)$ with $\frac{1}{f_1(\lambda)}$. Concretely, a sub-exponentially secure OP-PRP scheme would be one such that there exists a positive real constant $\delta > 0$ such that the scheme is $(2^{\lambda^\delta}, \frac{1}{2^{\lambda^\delta}})$ -secure.

From output-permutable to arbitrarily permutable. We say that a PRP is an *input permutable* (IP-) PRP if we apply Γ to the inputs rather than the outputs of Π (and accordingly apply Γ^{-1} to the output, and not the input, of Π^{-1}). Note that by exchanging the roles of Π and Π^{-1} (and likewise Eval and Eval^{-1}), we can turn any OP-PRP into an IP-PRP and vice versa. Also, for a PRP of the form $\Pi_{\text{out}}(k_{\text{out}}, \Pi_{\text{in}}(k_{\text{in}}, \cdot))$, that's the composition of an IP-PRP Π_{in} and an OP-PRP Π_{out} , both properties are simultaneously satisfied. We simply call such a PRP a permutable PRP (P-PRP).

Decomposable Permutations. In this work the class G of permutations that we will be interested in, is the class of *decomposable permutations*, defined next.

Definition 40 (Decomposable Permutations). *Let $N, T, s \in \mathbb{N}$, let Γ a permutation on $[N]$. We say that Γ is (T, s) -decomposable if there exists a sequence of permutations $\Gamma_0, \Gamma_1, \dots, \Gamma_T$ such that:*

- Γ_0 is the identity.
- $\Gamma_T = \Gamma$.
- For each $i \in [T]$, there exist circuits C_i, C_i^{-1} , each of size at most s , that compute the permutations Γ_i, Γ_i^{-1} , respectively.
- For each $i \in [T]$, either (1) $\Gamma_i = \Gamma_{i-1}$ or (2) there exists a $z_i \in [N]$ such that $\Gamma_i = \Gamma_{i-1} \circ (z_i \ z_i + 1)$. Here $(z_i \ z_i + 1)$ is the neighbor-swap permutation for z_i , which swaps between z_i and $(z_i + 1 \bmod N)$, and acts as the identity on all other elements in $[N]$.

In the uniform setting, we will additionally ask that there is a uniform polynomial-time (quantum) algorithm which given the description of Γ and i constructs both z_i and the circuits for Γ_i, Γ_i^{-1} .

Examples of Decomposable Families of Permutations. Whenever we ask that the circuit size parameter $s(N)$ is polynomial (in $\log(N)$), we do not know whether any efficiently computable (in both directions) permutation Π, Π^{-1} is decomposable. Intuitively, the reason is that while any permutation can be written as a concatenation of neighbor swaps, it may very well be the case that somewhere along the (likely exponentially-long) sequence of permutations Γ_i, Γ_i^{-1} , some of

them will not have an efficient circuit implementation. This leads to our Question 8 regarding the ability to efficiently decompose permutations. Despite our lack of complete understanding of the decomposability of permutations, we mention some examples of decomposable families of permutations.

- **Naive composition of decomposable permutations.** Let Γ that can be decomposed into a polynomial-length sequence $\Gamma = \Gamma^1 \circ \dots \circ \Gamma^r$ such that for every $i \in [r]$, the permutation Γ^i is (T, s) -decomposable. Then Γ is (rT, rs) -decomposable.
- **Linear cycles.** Linear cycles $(j \ j+1 \ j+2 \ \dots \ \ell-1 \ \ell)$ swapping j and $j+1$, then $j+1$ and $j+2$ and eventually $\ell-1$ and ℓ . Equivalently, j goes to position ℓ and all other elements in the range are subtracted by 1. Linear cycles as well as their inverses, are $(N, \text{polylog}(N))$ -decomposable. To see this one can consider a straightforward decomposition of the cycle into neighbor swaps, and furthermore the efficient implementation of each of the intermediate permutations is given by a circuit that simply check if z is in the range of the cycle, if so, decrements by 1 (mod N) or if the element is j , sends it to position $j+i$ (where i denotes the step in the decomposition).
- **Transpositions.** Transpositions $(j \ \ell)$, or (non-neighboring) swaps, are $(O(N), \text{polylog}(N))$ -decomposable using the decomposition

$$\begin{aligned} (j \ \ell) &= (j \ j+1 \ j+2 \ \dots \ \ell-1) (\ell-1 \ \ell) (j \ j+1 \ j+2 \ \dots \ \ell-1)^{-1} \\ &= (j \ j+1 \ j+2 \ \dots \ \ell-1 \ \ell) (j \ j+1 \ j+2 \ \dots \ \ell-1)^{-1} , \end{aligned}$$

To see the $(O(N), \text{polylog}(N))$ decomposition, we use the above rule of composition of decomposable permutations, for the two decomposable permutations $(j \ j+1 \ j+2 \ \dots \ \ell-1 \ \ell)$ and $(j \ j+1 \ j+2 \ \dots \ \ell-1)^{-1}$.

- **Permutations that are decomposable to transpositions, rather than neighbor swaps.** Let Γ a permutation on $[N]$ that's (T, s) -decomposable, but to transpositions rather than neighbor swaps. Specifically, in the T -length sequence of permutations that Γ decomposes to (and each of such permutation Γ_i, Γ_i^{-1} has implementation of complexity $\leq s$), each consecutive pair is either identical or differs in one transposition. Then, Γ is $(T \cdot O(N), s + \text{polylog}(N))$ -decomposable into neighbor swaps.
- **Involutions.** Involutions are permutations where $\Gamma \circ \Gamma$ is the identity. Involutions that are computable by circuits of size s are $(O(N^2), s + \text{polylog}(N))$ -decomposable. We will decompose Γ into a (N, s) -decomposition of transpositions, which will imply our wanted decomposition to neighbor swaps. Intuitively, we will visualize Γ as the applications of disjoint transpositions, and then the way we are going to decompose Γ is by adding each of the (possibly exponentially many) transpositions, only when both of its elements are smaller than some index. Formally,

$$\Gamma_i(x) := \begin{cases} \Gamma(x) & \text{if } x \leq i \text{ and } \Gamma(x) \leq i \\ x & \text{otherwise} \end{cases} .$$

Each permutation in the sequence is of complexity $\leq s + \text{polylog}(N)$, we have Γ_0 is the identity and $\Gamma_N := \Gamma$. Also, $\Gamma_i = \Gamma_{i-1}$ if $\Gamma(i) \geq i$ and otherwise $\Gamma_i(x) = \Gamma_{i-1} \circ (i \ \Gamma(i))$.

- **Conjugations.** If $\Gamma = \Lambda^{-1} \circ \Gamma' \circ \Lambda$ where Λ, Λ^{-1} are permutations with circuits of size U and Γ' is (T, S) -decomposable, then Γ is $(T, O(S + U))$ -decomposable, by simply conjugating the decomposition of Γ by Λ . Note that Λ does *not* need to be decomposable.
- **Applying a decomposable permutation to a subset of the bits.** If $N = N_0 \cdot N_1$, and Γ_0 is a (T, s) -decomposable permutation on domain $[N_0]$, we can extend it to a permutation Γ with domain $[N] \cong [N_0] \times [N_1]$, that applies Γ_0 to $[N_0]$ and the identity to $[N_1]$. Then Γ is also (T, s) -decomposable.
- **Applying a conditional decomposable permutation.** If $N = N_0 \cdot N_1$, and Γ_0 is a (T, s) -decomposable permutation on domain $[N_0]$, we can extend it to a permutation Γ with domain $[N] \cong [N_0] \times [N_1]$ where Γ applies Γ_0 to $[N_0]$ conditioned on some target value $v \in [N_1]$, and the identity otherwise. Then Γ is $(T, s + \text{polylog}(N))$ -decomposable.
- **Applying a controlled decomposable permutation.** If $N = N_0 \cdot N_1$, and for every $v \in [N_1]$, Γ_v is a (T, s) -decomposable permutation on domain $[N_0]$, we can extend to a permutation Γ with domain $[N] \cong [N_0] \times [N_1]$ where Γ applies Γ_v to $[N_0]$ conditioned on the element in $[N_1]$ being v . Then Γ is $(T \cdot N_1, s + \text{polylog}(N))$ -decomposable.
- **Scalar addition.** $x \mapsto x + s \bmod N$ is $(N, \text{polylog}(N))$ -decomposable, as adding s is decomposable into a linear chain of transpositions:

$$(0 \ s) \circ (1 \ 1 + s) \circ \cdots \circ (N - 2 - s \ N - 2) \circ (N - 1 - s \ N - 1) \ ,$$

where each point in the chain is efficiently computable (and invertible), because it effectively moves all elements in some (efficiently computable) domain s steps forward, and the largest s elements in the domain become the smallest s elements in the domain, in an order-preserving manner. Thus scalar addition is $(N, \text{polylog}(N))$ -decomposable. Similarly, vector addition $\mathbf{x} \mapsto \mathbf{x} + \mathbf{s} \bmod \mathbb{Z}_N^n$ is thus $(N \cdot n, \text{polylog}(N) \cdot n)$ -decomposable, by performing the addition per coordinate, and deducing decomposability by naive composition of decomposable permutations.

- **Scalar multiplication.** $x \mapsto ax \bmod N$ for any polynomial a which has an inverse in N is $(N, \text{polylog}(N))$ -decomposable, though this seems to require a bit of work. Under the Extended Riemann Hypothesis (ERH), all a are $(N^2, \text{polylog}(N))$ -decomposable. If discrete logarithms mod N had small circuits, we could use the conjugation example above to reduce multiplication to addition, decomposing multiplication by any a . However, since discrete logarithms are presumably classically hard, we have to do something else. In Remark 49 following the proof of Theorem 48, we explain how to decompose multiplications by small a , or more generally any $a \in \mathbb{Z}_N^*$ that is generated by small integers. Assuming ERH, all a are generated by small integers [Bac90], giving a conditional decomposability for all a .
- **Affine transformations.** Affine permutations $\mathbf{x} \mapsto \mathbf{A} \cdot \mathbf{x} + \mathbf{b} \bmod r$ where $\mathbf{A} \in \mathbb{Z}_r^{n \times n}$ is invertible and $\mathbf{b} \in \mathbb{Z}_r^n$, are $(O(r^n \times n^2), \text{poly}(r, n))$ -decomposable. Note that the circuit size depends polynomially on r ; we can improve this $(O(r^{2n} \times n^2), \text{poly}(\log r, n))$ -decomposability under ERH. We leave it as an interesting open question to handle general r unconditionally. We observe that we can handle addition by $\mathbf{b}$ as above. To multiply by $\mathbf{A}$, we decompose $\mathbf{A}$ into $O(n^2)$ elementary row operations. Row Swaps are involutions and Row Sums are

controlled additions, which are both decomposable by the above results. Finally, since r is small, we can handle Scalar Multiplications by the above.

- **Injective functions (and permutations) with an ancilla.** We do not know how to generically decompose an arbitrary permutation Γ , though we can do it if we are willing to use ancilla bits. Slightly more generally than permutations with an ancilla, we will show how to compose with an efficiently computable and invertible injective function $\Gamma : [N] \rightarrow [M]$ that computes with an ancilla (this in particular implies for efficiently computable and invertible permutations).

Suppose $\Gamma : [N] \rightarrow [M]$ is an injective function such that (1) Γ is computable in size s and (2) $\Gamma^{-1} : [M] \rightarrow [N]$ is computable in size s (and on elements in $[M]$ that are not in the image of Γ the output can be arbitrary in $[N]$). We construct a permutation Γ' on $([N] \times [M]) \cong [N \cdot M]$ that is $\left(O\left((N \cdot M)^2\right), s \times \text{polylog}(N \cdot M)\right)$ -decomposable, such that

$$\forall x \in [N] : \Gamma'(x, 0_{[M]}) = (\Gamma(x) \in [M], 0_{[N]}) ,$$

and for cases where the second input is not $0_{[M]}$, the permutation Γ' may act arbitrarily.

We show how to decompose Γ' into 2 controlled scalar addition permutations, and 1 involution (which are all decomposable, as we know by now), which will make Γ' also appropriately decomposable. Let

$$\begin{aligned} \Gamma_1(x, y) &:= (x, y + \Gamma(x) \bmod M) , \\ \Gamma_2(x, y) &:= (x - \Gamma^{-1}(y) \bmod N, y) , \\ \Gamma_3(x, y) &:= (y, x) , \end{aligned}$$

Then, let $\Gamma' = \Gamma_3 \circ \Gamma_2 \circ \Gamma_1$. Since each of $\Gamma_1, \Gamma_2, \Gamma_3$ are involutions (since our arithmetic is over $\mathbb{Z}_2$) then the composed Γ' is accordingly decomposable using the involution case.

Our main theorem of this section is the following:

Theorem 41. *Let T be any exponential function and p any polynomial. Assuming the existence of sub-exponentially-secure one-way functions and sub-exponentially-secure iO , there exists an OP-PRP for the class of (T, p) -decomposable permutations. Moreover, the OP-PRP is itself (T, p) decomposable.*

This theorem will be proved in Sections 5.2, 5.3, and 5.4. Before proving it, however, we will give some example applications.

5.1 How to use OP-PRPs with Indistinguishability Obfuscation

Here, we explain how OP-PRPs are useful for constructions involving indistinguishability obfuscation.

Composing with fixed permutations. Consider a program $P^{O, O^{-1}}$ which makes queries to an oracle O . We show the following:

Lemma 42. *Let Γ be a permutation, and let $(\Pi, \Pi^{-1}, \text{Permute})$ be an OP-PRP for a class of permutations which includes Γ . Then for a sufficiently large polynomial s , $\text{iO}(1^\lambda, 1^s, P^{\Pi(k, \cdot), \Pi^{-1}(k, \cdot)})$ is computationally indistinguishable from $\text{iO}(1^\lambda, 1^s, P^{\Gamma(\Pi(k, \cdot)), \Pi^{-1}(k, \Gamma^{-1}(\cdot))})$, where $k \leftarrow \{0, 1\}^\lambda$ is uniformly random.*

In other words, we can compose $\Pi(k, \cdot)$ with any fixed permutation Γ applied to the output of Π . This is analogous to the oracle case, where composing a random permutation with any fixed permutation gives a random permutation.

Proof. We prove security through a sequence of hybrids:

Hyb₀: Here, the adversary is given $\text{iO}(1^\lambda, 1^s, P^{O, O^{-1}})$ where $O(\cdot) = \Pi(k, \cdot)$ and $O^{-1}(\cdot) = \Pi^{-1}(k, \cdot)$.

Hyb₁: Now we sample $k^{\Gamma, 0} \leftarrow \text{Permute}(k, \Gamma, 0)$ and switch to $O(\cdot) = \Pi(k^{\Gamma, 0}, \cdot)$ and $O^{-1}(\cdot) = \Pi^{-1}(k^{\Gamma, 0}, \cdot)$. By the correctness of the permuted key $k^{\Gamma, 0}$, O, O^{-1} , and hence $P^{O, O^{-1}}$, is unchanged by this modification. Therefore, as long as s is larger than the maximum size of $P^{O, O^{-1}}$ in Hybrids 0 and 1, by iO security the two hybrids are indistinguishable.

Hyb₂: Now we switch to $k^{\Gamma, 1} \leftarrow \text{Permute}(k, \Gamma, 1)$ and set $O(\cdot) = \Pi(k^{\Gamma, 1}, \cdot)$ and $O^{-1}(\cdot) = \Pi^{-1}(k^{\Gamma, 1}, \cdot)$. Indistinguishability from Hybrid 1 follows from OP-PRP security.

Hyb₃: Now we move to $O(\cdot) = \Gamma(\Pi(k, \cdot))$ and $O^{-1}(\cdot) = \Pi^{-1}(k, \Gamma^{-1}(\cdot))$. Observe that these oracles O, O^{-1} are functionally identical to those in Hybrid 2, and therefore so is the program P . Thus, indistinguishability from Hybrid 2 follows from iO security.

Thus, we have that Hybrids 0 and 3 are indistinguishable, proving Lemma 42. $\square$

Trapdoor permutations from iO and one-way functions. Here, we show that obfuscating an OP-PRP gives a trapdoor permutation.

Construction 43. *Let (Π, Π^{-1}) be a PRP, and iO an indistinguishability obfuscator. Then define the trapdoor permutation (Gen, F, F^{-1}) as:*

- **Gen** (1^λ) : Sample $k \leftarrow \{0, 1\}^\lambda$. Let $P : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ defined as $P(x) := \Pi(k, x)$, setting the block size $n = \lambda$. Let s be a sufficiently large function of λ . Output $\text{pk} = \hat{P} \leftarrow \text{iO}(1^\lambda, 1^s, P)$ and $\text{sk} = k$.
- **F** (pk, x) : Interpret pk as a program $\hat{P}$, and output $\hat{P}(x)$.
- **F⁻¹** (sk, y) : Interpret sk as a key k , and output $\Pi^{-1}(k, y)$.

Theorem 44 (Trapdoor One-Way Permutations from iO and One-Way Functions). *Assume the existence of one-way functions. Assume $(\Pi, \Pi^{-1}, \text{Permute})$ is an OP-PRP for some class that includes all transpositions, and iO is a secure iO. Then, if we instantiate Construction 43 with $(\Pi, \Pi^{-1}, \text{Permute})$ as the PRP, we get a secure trapdoor permutation.*

Proof. We need to show that there is no algorithm $\mathcal{A}$ which, given $\hat{P} \leftarrow \text{iO}(1^\lambda, 1^s, P)$ and a random $y^* \leftarrow \{0, 1\}^\lambda$, outputs x^* such that $\hat{P}(x^*) = y^*$. Assume toward contradiction that there is such an $\mathcal{A}$ with success probability ϵ . We will show that ϵ is negligible through a sequence of hybrid experiments.

Hyb₀: Here, $\mathcal{A}$ is given $\hat{P} \leftarrow \text{iO}(1^\lambda, 1^s, P)$ and y^* , where $y^* \leftarrow \{0, 1\}^\lambda$ and $P(\cdot) = \Pi(k, \cdot)$ for a random key $k \leftarrow \{0, 1\}^\lambda$. $\mathcal{A}$ wins if it outputs x such that $\Pi(k, x) = y^*$, which by assumption is with non-negligible probability.

Hyb₁: Now we additionally sample a random $y' \leftarrow \{0, 1\}^\lambda$, and switch to $\hat{P} \leftarrow \text{iO}(1^\lambda, 1^s, P_1)$ where

$$P_1(x) = \begin{cases} y^* & \text{if } \Pi(k, x) = y' \\ \Pi(k, x) & \text{otherwise} \end{cases}.$$

$\mathcal{A}$ still wins if it outputs x such that $\Pi(k, x) = y^*$. Indistinguishability from Hybrid 0 follows from Lemma 18.

Hyb₂: Now we sample $k^* \leftarrow k^{(y^* \ y'), 0} \leftarrow \text{Permute}(k, (y^* \ y'), 0)$, change $\hat{P} \leftarrow \text{iO}(1^\lambda, 1^s, P_2)$, where

$$P_2(x) = \begin{cases} y^* & \text{if } \Pi(k^*, x) = y' \\ \Pi(k^*, x) & \text{otherwise} \end{cases}.$$

Now, we switch to $\mathcal{A}$ winning if it produces an x such that $\Pi(k^*, x) = y^*$. Since the programs $\Pi(k, \cdot)$ and $\Pi(k^{(y^* \ y'), 0}, \cdot) := \Pi(k^*, \cdot)$ are functionally equivalent, the obfuscations of the programs P_1 and P_2 are indistinguishable by the security of the outer iO . Also, the success condition of the adversary is functionally equivalent. Overall, indistinguishability from Hybrid 1 follows from iO security.

Hyb₃: Now we switch to sampling $k^* = k^{(y^* \ y'), 1} \leftarrow \text{Permute}(k, (y^* \ y'), 1)$ and use this key in the program P_2 and also for the win condition. Indistinguishability from Hybrid 2 follows from OP-PRP security. Now observe that the win condition is $\Pi(k^{(y^* \ y'), 1}, x) = y^*$, which is equivalent to $\Pi(k, x) = y'$.

Hyb₄: Now we switch back to giving $\mathcal{A}$ the program $\hat{P} \leftarrow \text{iO}(1^\lambda, 1^s, P_1)$, but keeping the winning condition as $\Pi(k, x) = y'$. Observe that switching from $k^* = k^{(y^* \ y'), 0}$ to $k^* = k^{(y^* \ y'), 1}$ in P_2 actually did not change the functionality at all: the change permuted the values of y^* and y' in the output of Π , but since both values cause P_2 to output y^* , swapping them does not change the functionality. Therefore, the functionality of P_2 using either permuted key remains equivalent to P_1 . Thus, $\mathcal{A}$ outputs x such that $\Pi(k, x) = y'$ with non-negligible probability.

Now, observe that if we additionally give the adversary k , it can compute $y' = \Pi(k, x)$ for itself, with non-negligible probability. Thus, we obtain an adversary $\mathcal{B}$ which is given the description of P (namely, the key k) and $\text{iO}(1^\lambda, 1^s, P_1)$, and guesses y' with non-negligible probability. But this contradicts the computational unpredictability guarantee from Lemma 18. Hence the original advantage of $\mathcal{A}$ in inverting the one-way permutation must be negligible. $\square$

Fixed sparse triggers. Lemma 18 allows for “puncturing” a program if a certain random trigger is hit, in which case the program may behave completely differently from the original program. Here, we show that the trigger can even be *fixed*, as long as it is appropriately scrambled by OP-PRPs. Concretely, consider the program $P(x)$ with hard-coded OP-PRP keys k_0, k_1 that works as follows (and is also described in Figure 7):

1. Apply some polynomial-sized circuit $P_0(x)$, obtaining a pair x_1, w_1 .
2. Run $\Pi(k_0, x_1)$, and parse the output as (x_2, w_2) .

3. Apply some polynomial-sized circuit $P_1(w_1, w_2)$, obtaining w_3, w_4 .
4. Then run $\Pi^{-1}(k_1, (x_2, w_3))$, obtaining x_3 .
5. Finally feed x_3, w_4 into some polynomial-sized circuit P_2 , and output the result.

The key structural property of the program P is that x_2 is the output of $\Pi(k_0, \cdot)$ and is fed into $\Pi^{-1}(k_1, \cdot)$ without modification and without affecting any other part of the circuit.

Now consider a different program $P'(x)$, which is identical except that we modify Step 3 into Step 3' by embedding a trigger:

- 3'. Apply some polynomial-sized circuit $R(x_2)$ with single-bit outputs. If $R(x_2) = 0$, then let $(w_3, w_4) \leftarrow P_1(w_1, w_2)$ as in program P . However, if $R(x_2) = 1$, we instead let $(w_3, w_4) \leftarrow P'_1(w_1, w_2)$, for some different polynomial-sized circuit P'_1 .

Extending the above template to a pair of circuits instead of a single circuit. In the above, the program P is defined as a function of the programs P_0, P_1, P_2 and the program P' is defined as a function of the programs P_0, P_1, P_2 in addition to the programs R, P'_1 .

We next extend the above template in a way that the single program P is extended to a pair of programs P, Q , and the single program P' is extended to a pair of programs P', Q' , in the following way:

- Let $Q_0, Q_1, Q_2, P_0, P_1, P_2$ arbitrary programs. The program Q is defined as a function of Q_0, Q_1, Q_2 in the exact same way that P is defined as a function of P_0, P_1, P_2 . The programs P, Q are sampled for the *same* pair of keys k_0, k_1 , with the only change that the roles of the keys are reversed between P and Q .
- Let R, Q'_1, P'_1 some arbitrary programs, and let $Q_0, Q_1, Q_2, P_0, P_1, P_2$ the programs defining the above P, Q . The program Q' is defined as a function of Q_0, Q_1, Q_2, R, Q'_1 in the same way that the program P' is defined as a function of the programs P_0, P_1, P_2, R, P'_1 .

Thus, we can consider a sampling mechanism that either samples the pair P, Q together (by having the programs $P_0, P_1, P_2, Q_0, Q_1, Q_2$ fixed and sampling the keys k_0, k_1) or samples the pair P', Q' together (by having the programs $P_0, P_1, P_2, Q_0, Q_1, Q_2$ fixed and also the programs R, P'_1, Q'_1 fixed, and sampling the keys k_0, k_1). We will show that obfuscating the un-primed and primed versions give computationally indistinguishable programs, for certain choices of R . In the following, we will interpret x_2 as an element in $[0, N)$ for some $N \in \mathbb{N}$.

Lemma 45 (Permuted Fixed Sparse Triggers). *Let $P_0, P_1, P_2, Q_0, Q_1, Q_2, R, P'_1, Q'_1$ arbitrary programs and let P, Q (or P', Q') the derived pair of programs, as the template above. Assume that $R(x_2)$ outputs 1 if and only if $x_2 \in [a, b)$, for some integers $0 \leq a < b < N$ such that the ratio $\frac{N}{b-a}$ is exponential in $\lambda \in \mathbb{N}$, the security parameter. Assuming $(\Pi, \Pi^{-1}, \text{Permute})$ the permutation used inside the programs, is an OP-PRP for any class that includes all involutions and iO is a secure indistinguishability obfuscator, for random choices of the keys k_0, k_1 , for a sufficiently large polynomial s , we have that the obfuscated pair $\text{iO}(1^\lambda, 1^s, P), \text{iO}(1^\lambda, 1^s, Q)$ is computationally indistinguishable from the obfuscated pair $\text{iO}(1^\lambda, 1^s, P'), \text{iO}(1^\lambda, 1^s, Q')$.*

Proof. We prove indistinguishability through a sequence of hybrids.

Hyb₀: Here, we obfuscate P, Q .

Hyb₁: Here, we choose a random $y \in [0, \lceil \frac{N}{b-a} \rceil]$, and obfuscate the programs P', Q' , but where we replace the relation R in both programs with the relation R' where $R'(x_2)$ outputs 1 if and only if $x_2 \in [y(b-a), (y+1)(b-a))$. This is the same as saying that $(x_2 - [x_2 \bmod (b-a)]) / (b-a) = y$. Observe that the range of $(x_2 - [x_2 \bmod (b-a)]) / (b-a)$ is contained in $[N/(b-a)]$. Indistinguishability of Hybrid 0 and Hybrid 1 follows from the indistinguishability guarantee of Lemma 18 and the fact that y is uniform in an exponential-sized domain.

Hyb₂: Here, we switch to a random $y \in [0, \lfloor N/(b-a) \rfloor]$. Since $N/(b-a)$ is exponential, this is a negligible change in the distribution of y , hence Hybrid 1 and Hybrid 2 are indistinguishable.

Hyb₃: Let π be the involution on x_2 , which exchanges the ranges $[a, b)$ and $[y(b-a), (y+1)(b-a))$. Notice that these intervals have the same size, and the latter interval is contained in $[0, N)$ since $y \leq N/(b-a) - 1$. We can easily extend π to be an involution mapping $(x_2, w_2) \mapsto (\pi(x_2), w_2)$ or $(x_2, w_3) \mapsto (\pi(x_2), w_3)$.

Now instead of obfuscating the program P' , we switch to obfuscating the program P'' which replaces $\Pi(k_0, \cdot)$ with $\pi(\Pi(k_0, \cdot))$ and $\Pi^{-1}(k_1, \cdot)$ with $\Pi^{-1}(k_1, \pi^{-1}(\cdot))$ (still using the relation R'). We likewise switch from Q' to the analogous program Q'' . Since this is just composing the PRP applications with the fixed involution π , Hybrids 1 and 2 are indistinguishable by Lemma 42.

Observe that in P'' we now we apply π to the output of $\Pi(k_0, \cdot)$ and to the input to $\Pi^{-1}(k_1, \cdot)$ (and the analogous statements for Q''). Thus, the two applications of π cancel out, *except* that the trigger is checked *between* applications of π . Since π exchanges the roles of $[a, b)$ and $[y(b-a), (y+1)(b-a))$, if we were to test the output of $\Pi(k_0, \cdot)$ itself, the trigger value would in fact be the interval $[a, b)$.

Hyb₄: Now we switch to obfuscating P', Q' without π but with the correct relation R . This is functionally equivalent to our modified P'', Q'' , since the permuted keys changes the trigger (when interpreted as an output of $\Pi(k_0, \cdot)$ or $\Pi(k_1, \cdot)$) to be $[a, b)$. Thus, by iO security, Hybrid 2 and Hybrid 3 are computationally indistinguishable. This completes the proof of Lemma 45. $\square$

5.2 ONS-Merges

We now gradually build up to our proof of Theorem 41. Here, we start from a seemingly much weaker object called an Output Neighbor Swap Merge (ONS-Merge).

A neighbor swap is a permutation which exchanges some j with $j+1$ and otherwise is the identity. In cycles notation, a neighbor swap would be written as $(j \ j+1)$.

A Merge is a permutation with the added correctness requirement. The domain $[N]$ is interpreted as pairs (b, x) for $b \in \{0, 1\}$ and $x \in N_b$, where $N_0 + N_1 = N$. The range $[N]$ remains $[N]$. We let $L = \{(0, x)\} \cong [N_0]$ and $R = \{(1, x)\} \cong [N_1]$. A Merge is then a permutation preserves the ordering of elements in L , and also preserves the ordering of elements in R . An Output Neighbor Swap Merge can be thought of as an OP-PRP for the simple class of neighbor swaps. However, many such swaps will actually break the strong order-preserving property of the merge, and hence will be illegal. We therefore need to modify the definition (both correctness and security) to account for this.

Definition 46. *An Output Neighbor Swap (ONS-) Merge is a tuple of five algorithms $(M, M^{-1}, \text{Permute}, \text{Eval}, \text{Eval}^{-1})$ with the following properties:*

- **Efficient Permutations:** For any key $k \in \{0, 1\}^\lambda$, any desired block-sizes N_0, N_1 , $M(k, \cdot)$ is an efficiently computable permutation on $[N = N_0 + N_1]$ with $M^{-1}(k, \cdot)$ being its efficiently computable inverse.
- **Order-Preserving:** For any key $k \in \{0, 1\}^\lambda$, any block sizes N_0, N_1 , and any two inputs $x_0 < x_1 \in [N_0]$ (resp. $x_0 < x_1 \in [N_1]$), then $M(k, (0, x_0)) < M(k, (0, x_1))$ (resp. $M(k, (1, x_0)) < M(k, (1, x_1))$). If $b_0 \neq b_1$, there is no restriction on the ordering of $M(k, (b_0, x_0))$ and $M(k, (b_1, x_1))$.
- **Output Neighbor Swapping:** $\text{Permute}(k, \Gamma, c)$ is a deterministic polynomial-time procedure which takes as input a key $k \in \{0, 1\}^\lambda$, a neighbor swap $(z \ z+1)$, and a bit c . If $M^{-1}(k, z) = (b_0, x_0)$ and $M^{-1}(k, z+1) = (b_1, x_1)$ with $b_0 \neq b_1$, it outputs a swapped key $k^{(z \ z+1), c}$. Otherwise if $b_0 = b_1$, the input is considered illegal and the output is $\perp$.
- **Output Swapping Correctness:** For all $\lambda \in \mathbb{Z}$, $k \in \{0, 1\}^\lambda$, all legal neighbor swaps $(z \ z+1)$, and all $x, z' \in [N]$,

$$\begin{aligned} \text{Eval}(k^{(z \ z+1), c}, x) &= \begin{cases} \Pi(k, x) & \text{if } c = 0 \\ (z \ z+1) \circ (\Pi(k, x)) & \text{if } c = 1 \end{cases} \\ \text{Eval}^{-1}(k^{(z \ z+1), c}, z') &= \begin{cases} \Pi^{-1}(k, z') & \text{if } c = 0 \\ \Pi^{-1}(k, (z \ z+1)(z')) & \text{if } c = 1 \end{cases} \end{aligned}$$

- **Security:** For any interactive QPT adversary $\mathcal{A}$, there exists a negligible function $\epsilon(\lambda)$ such that the following experiment with $\mathcal{A}$ outputs 1 with probability at most $1/2 + \epsilon(\lambda)$:
 - $\mathcal{A}(1^\lambda)$ chooses a neighbor swap $(z \ z+1)$ for $z \in [N-1]$.
 - The experiment chooses a random $k \leftarrow \{0, 1\}^\lambda$ and a random bit $c \in \{0, 1\}$. It computes $(b_0, x_0) \leftarrow M^{-1}(k, z)$ and $(b_1, x_1) \leftarrow M^{-1}(k, z+1)$. It checks that $b_0 \neq b_1$; if $b_0 = b_1$ the experiment immediately aborts and returns a random bit. If $b_0 \neq b_1$, it returns $k^{(z \ z+1), b} \leftarrow \text{Permute}(k, (z \ z+1), c)$ to $\mathcal{A}$.
 - $\mathcal{A}$ produces a guess c' for c . The experiment outputs 1 if $c' = c$.

Note that the abort condition is necessary: in these cases, permuting the output by $(z \ z+1)$ actually reverses the order of two strings belonging to the same set L or R . But this breaks the order-preserving property of the permuted key, which allows for easy distinguishing. However, if $M^{-1}(k, z)$ and $M^{-1}(k, z+1)$ have are in different sets L and R , then the order between them is arbitrary, and so we can hope that the permuted keys are indistinguishable.

Hypergeometric Distribution. Let $D_{N,t,s}$ the following distribution: let U be an arbitrary subset of $[N]$ of size s . Choose a random set V of $[N]$ of size exactly t , and output the number of elements in $V \cap U$. This distribution is known as the hypergeometric distribution and can be efficiently sampled. More specifically, there is a sequence of functions $D_{N,t,s}^\kappa$ with domain $\{0, 1\}^\kappa$ such that $D_{N,t,s}^\kappa(r)$ for random coins r approximates a distribution that is $O(N \times 2^{-\kappa})$ -close to $D_{N,t,s}$.

Tally Trees. In order to describe our construction, we introduce the notion of a tally tree. Consider a merge M . Assign to each range element z a bit b indicating the first bit of the pre-image of z . Thus, we obtain a sequence V of N bits, which determines the images of the sets L and R . The number of 0's is exactly N_0 and the number of 1's is exactly N_1 . Observe that V is in bijection with the merge M . This is because once you choose the set of images of L (resp. R), the actual mapping from L (resp. R) to those images is fixed by the ordering.

Now, notice that V alone does not actually allow for *efficient* computation of M (nor M^{-1}), since to determine the image of, say, $(0, x)$, you would need to find the position in V of the x -th 0. But this presumably requires scanning exponentially-many bits in V to find the right location.

We can, however, speed this up by supplying more information, which is exactly the *tally tree*. A tally tree T is a binary tree with N leaves, one for each element of the range N . We will think of T as having a fixed topology that depends on N , with the goal of making T shallow. We will associate two quantities to each node z . The first is $s(z)$, which is the number of leaves of the subtree rooted at z . $s(z)$ is solely a function of the topology of T and will just be used for notational convenience.

The second value is $v(z)$, which is the total of all V_u values for all leaves u in the subtree rooted at z . Equivalently, $v(z) = V_z$ for all leaves, and $v(p)$ for any internal node is equal to the sum $v(p) = v(u) + v(w)$ where u, w are the left and right children of p . Observe that for the root ε , $v(\varepsilon) = N_1$.

Observe that we can equivalently sample a tally tree in reverse, starting from the root. We start by setting $v(\varepsilon) = N_1$. Then suppose we have set $v(p) = t$ for a node p with left child u and right child w . Setting $v(p) = t$ stipulates that among the $s(p)$ leaves of the tree rooted at p , t of them are set to 1. But as we haven't set any of the descendants of p yet, just the total of them, the distribution over the positions of those t 1's in the subtree rooted at p is uniformly random. We then have that $v(u)$ is exactly distributed according to $D_{s(p), t, s(u)}$. We then set $v(w) = v(p) - v(u)$.

Notice that by sampling from appropriate hypergeometric distributions, we can sample the nodes of T in basically any order. For example, let C be a *cover* of T , meaning a set of nodes whose subtrees are disjoint and jointly include all of the leaves of T . We can sample $v(u)$ for all $u \in C$ in any order as follows. Let N' denote the portion of the domain yet to be determined (initially $N' = N$) and S the number of 1's remaining to allocate (initially $S = N_1$). Then in any order, we choose an element $u \in C$, sample $v(u) \leftarrow D_{N', s(u), S}$, and then update $N' \mapsto N' - s(u)$, $S \mapsto S - v(u)$. Notice that setting $v(u)$ for $u \in C$ determines $v(u')$ for all u' that are "above" the cover. Observe that through this process, the distribution of $v(u)$ for any $u \in C$ depends on the total $\sum_{u'} v(u')$ of all $v(u')$ sampled so far, but is otherwise independent of the actual values $v(u')$.

Given a tally tree T , we evaluate $(b, x) \leftarrow M^{-1}(z)$ as follows. First set $b = V_z$ by looking the value stored at the leaf labeled z . Now by the ordering property of a merge, x is just a count of the number of $z' < z$ with $M^{-1}(z)$ having the first bit b . We cannot directly count such z' in V (since there will be exponentially-many), but we can instead use the internal nodes of the tree T . Namely, let $U_L = \{u\}$ be the set of nodes that are *left* siblings of nodes on the path from root to z . Then $x = \sum_{u \in U_L} v(u)$. Observe that this process only visits a single path from root to leaf and its siblings, and therefore only $O(d)$ nodes where d is the depth of the tree.

To evaluate $M(x)$ given T , we simply do a binary search, exploiting the ordered property and our ability to compute $M^{-1}(x)$.

Our Construction. To give our construction, we will show how to implicitly generate a tally tree, which will then generate a merge. Rather than build the tally tree from the leaves toward the

roots, our construction use the top-down generation, but pseudorandomly generate the values in the tree.

Construction 47. Let $F : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$, and suppose it has a puncturing algorithm **Punc**. Given a key $k \in \{0, 1\}^\lambda$, the tally tree T is implicitly defined as follows. We deterministically choose a topology that has depth $O(\log N)$, which determines $s(z)$ for all nodes z . We set $v(\varepsilon) = N_1$. Then we recursively internal nodes as follows. For a node u that is a left child of some node p (which already has $s(p)$ defined), define $v(u) = D_{s(p), v(p), s(u)}^\kappa(F(k, p))$. We then let $v(w)$ for the right child w to be $v(p) - v(u)$. Note that these values are not explicitly computed, but rather left implicitly determined by k .

To permute a key k according to a legal swap $(z \ z+1)$ and bit c , collect into a set H all nodes in the paths from root to z or $z+1$, together with all the siblings of nodes on this path. Here, H stands for “hard-coded”. Let S denote the nodes just on the paths, excluding $z, z+1$. Here, S stands for “punctured” nodes. Assume without loss of generality that $v(z) = 0$ and $v(z+1) = 1$. Let $\bar{k}^S \leftarrow \text{Punc}(k, S)$. Then do the following:

- If $c = 0$, output $k^{(z \ z+1), 0} = (\bar{k}^S, \{(u, v(u))\}_{u \in H})$.
- If $c = 1$, output $k^{(z \ z+1), 1} = (\bar{k}^S, \{(u, v'(u))\}_{u \in H})$ where

$$v'(u) = \begin{cases} v(u) + 1 & \text{if } u \text{ is an ancestor of } z \text{ but not } z+1 \\ v(u) - 1 & \text{if } u \text{ is an ancestor of } z+1 \text{ but not } z \\ v(u) & \text{otherwise} \end{cases}$$

To evaluate $v(u)$ for some node u , we first look up if there is a pair $(u, v) \in H$, and if so produce the value v . Otherwise, we can use $\bar{k}^S$ to compute the remaining nodes.

Theorem 48. Suppose (F, Punc) is a secure puncturable PRF. Assume $\kappa \geq \lambda + \log N$. Then the protocol given in Construction 47 is an ONS Merge. In particular, if (F, Punc) is ϵ -secure, then Construction 47 is $O(\epsilon + N^2 \times 2^{-\kappa})$ -secure. Moreover, the permutations $M(k, \cdot), M^{-1}(k, \cdot)$ are $(O(N^2), \text{polylog}(N))$ -decomposable.

Proof. We first argue decomposability. Consider a tally tree T . The “identity” tally tree T_0 simply has all the leaves of value 1 as the rightmost leaves. Define $T_{r,i}$ as the tree which has the first (left-most) $r-1$ 1’s in the leaves in the correct position, the right-most $N_1 - r$ pushed all the way to the right, and the r -th 1 at position $N - (N_1 - r) - i$. Let i_r be such that the $N - (N_1 - r) - i_r$ is the correct position of the r th 1. The tally trees $T_{r,i}$ can all be succinctly represented: let C_r be the cover of the leaves containing the left-most $r-1$ 1’s. Then $v(u)$ for u descendant from C_r is determined exactly as in T . Let C'_r be a cover for the $N_1 - r$ leaves that have all the right-most 1’s. Then any u descendant from C'_r has $v(u) = s(u)$. Finally, let C''_r be a cover for the remaining nodes. Since there is at most 1 leaf descendant from C''_r that contains a 1, we can easily compute $v(u)$ for any u descendant from C''_r , by simply deciding whether that 1 is a descendant of u .

Then $T_0 = T_{1,0}$, $T_{r,i_r} = T_{r+1,0}$, and $T_{N_1+1,0} = T$. Going from $T_{r,i}$ to $T_{r,i+1}$ corresponds to a neighbor swap. This gives a sequence of length $O(N^2)$ connecting T_0 to T , showing that T is $(O(N^2), \text{polylog}(N))$ -decomposable.

We now prove security through a sequence of hybrids.

Hyb₀: Here, the adversary sees $k^{(z, z+1), 0} = (\bar{k}^S, \{(u, v_0(u))\}_{u \in S})$ where $v_0(u)$ is defined as $v_0(u) = v(u)$, which for the left child u of a parent p is equal to $D_{s(p), v(p), s(u)}^\kappa(F(k, p))$. Observe that we only need to consider the left children, as the right children are determined by their parent and sibling. The hybrid outputs a random bit and rejects if $v_0(z) = v_0(z+1)$.

Hyb₁: Here, we replace $k^{(z, z+1), 0}$ with $(\bar{k}^S, \{(u, v_1(u))\}_{u \in S})$ where for left-children u , $v_1(u)$ is defined as $D_{s(p), v(p), s(u)}^\kappa(r_u)$ for independent random coins $r_u \leftarrow \{0, 1\}^\kappa$. The hybrid outputs a random bit and rejects if $v_1(z) = v_1(z+1)$. By a straightforward reduction to punctured PRF security, Hybrid's 0 and 1 are indistinguishable except with probability ϵ .

Hyb₂: We now move to $(\bar{k}^S, \{(u, v_2(u))\}_{u \in S})$, where $v_2(u)$ is sampled as a fresh random sample from $D_{s(p), v(p), s(u)}^\kappa$. The hybrid outputs a random bit and rejects if $v_2(z) = v_2(z+1)$. Each such sample is at most $O(N \times 2^\kappa)$ -close to the distribution $D_{s(p), v(p), s(u)}^\kappa$ sampled in Hybrid 1, and there are at most $O(N)$ such samples. Therefore, Hybrids 1 and 2 are at most $O(N^2 \times 2^{-\kappa})$ -close.

Hyb₃: Here, we define v_2 as in Hybrid 2, but give the adversary $(\bar{k}^S, \{(u, v_3(u))\}_{u \in S})$, where, assuming $v_2(z) = 0$ and $v_2(z+1) = 1$, we define

$$v'_2(u) = \begin{cases} v_2(u) + 1 & \text{if } u \text{ is an ancestor of } z \text{ but not } z+1 \\ v_2(u) - 1 & \text{if } u \text{ is an ancestor of } z+1 \text{ but not } z \\ v_2(u) & \text{otherwise} \end{cases}$$

We now argue that the views $\{(u, v_2(u))\}_{u \in S}$ and $\{(u, v'_2(u))\}_{u \in S}$ are actually distributed identically, even given $\bar{k}^S$. To do so, observe that an equivalent way to sample v_2 is as follows. Let C be the cover of T defined as $H \setminus P$, obtained by taking all the nodes in S that are *not* on the paths from root to z or $z+1$, and additionally include $z, z+1$ themselves. This is given by the red squares and yellow pentagons in Figure 4. Then sample $v_2(u)$ for $u \in C$ according to the algorithm for sampling covers of tally trees described above, starting with $z, z+1$ and moving to the rest of C . Then compute internal nodes “above” the cover by adding the values of the node’s children, which gives all pairs $(u, v_2(u))$ for $u \in S$. The nodes “below” the cover C are then implicitly generated by F as before. The hybrid then rejects if $v_2(z) = v_2(z+1)$.

Since the values of T for the cover C is distributed exactly as in the case of a random merge, we see that conditioned on not rejecting, $v_2(z)$ and $v_2(z+1)$ are distributed as random distinct bits.

Hybrid 3 then is identical, except that we swap the values of at z and $z+1$. But since $v(z), v(z+1)$ were random distinct values anyway, this distributions are identical. Moreover, the rest of the cover C is sampled only depending on the total $v(z) + v(z+1)$, which we know is 1 in both Hybrid 3 and Hybrid 2. Thus, the distribution of the entire cover C , and therefore the entire tree, is identical in Hybrid 2 and Hybrid 3.

Hyb₄ and Hyb₅: These are analogs of Hybrids 1,0 (respectively), except that we use the values v'_1, v'_0 , which are derived from v_1, v_0 analogous to v'_2 . Hybrids 3 and 4 are indistinguishable by an identical argument to the indistinguishability of Hybrids 1 and 2. Hybrids 4 and 5 are likewise indistinguishable by an identical argument to that of Hybrids 0 and 1.

Then we observe that Hybrid 5 uses $v'_0 = v'$, and is therefore exactly the case $k^{(z, z+1), 1}$. This finishes the proof. $\square$ $\square$

Remark 49. *The decomposition of merges actually allows us to perform scalar multiplication for small scalars. We observe that $x \mapsto 2x \bmod N$ for odd N is actually a merge: indeed, it preserves*

the ordering for $x \in [0, (N-1)/2]$ as well as for $x \in [(N+1)/2, N-1]$. Moreover, we can compute the associated tally tree rather trivially. Thus we can decompose it as in Theorem 48. We can likewise handle $x \mapsto ax \bmod N$ for N relatively prime to a , as long as a is polynomial. The idea is to generalize the concept of a merge where the input domain is partitioned into a buckets and the order-preserving property holds for each bucket. We can then moreover generate $x \mapsto ax \bmod N$ for any a which can be decomposed as the product of “small” (polynomial in $\log N$) a' . Note that if the Extended Riemann Hypothesis is true, such small a' generate a [Bac90].

5.3 From ONS-Merge to ONS-PRP

An Output Neighbor Swappable PRP is an OP-PRP for the class of neighbor swaps($z \ z+1$). We now show that an ONS-Merge can be used to construct an ONS-PRP. The construction is based on the small-domain PRP of [GP07], though (1) we will use it in the large-domain setting, and (2) we will be implicitly implementing the underlying PRF with a *puncturable* PRF.

The construction can be thought of as performing a merge-sort in reverse, where we first partition the domain into a left and right set, of sizes N_0 and N_1 respectively. Then all the left elements are recursively shuffled using a smaller PRP, and all the right elements are shuffled using an independent smaller PRP. This step is the opposite of sorting the left and right parts separately. Then the two halves are randomly inserted into their final positions using a merge, which preserves the order of each half. The keys for the smaller PRPs and the merge will be derived pseudorandomly from the overall key.

Construction 50. Let $(M, M^{-1}, \text{Permute}')$ be an ONS-Merge. Let PRG be a length-tripling PRG. Then let $(\Pi, \Pi^{-1}, \text{Permute})$ be defined as follows. For $N = 2$ (x is a single bit), we will simply have $\Pi(k, x) = k \oplus x$. The only permutation is $(0 \ 1)$, which is the same as $x \mapsto 1 \oplus x$. We can permute the key information-theoretically as $\text{Permute}(k, (0 \ 1)) = k \oplus 1$. For larger N , we let $N_0 = \lfloor N/2 \rfloor$ and $N_1 = N - N_0$. We interpret the domain as $[N] \cong \{(b, x)\}_{b \in \{0,1\}, x \in [N_b]}$. We use this interpretation both for the input to $\Pi(k, \cdot)$ as well as for the input to M . For the outputs of $\Pi(k, \cdot)$ and M , we will interpret the range as $[N]$. Then we define the algorithms as follows:

- $\Pi(k, (b, x))$: Run $(k_0, k_1, k_2) \leftarrow \text{PRG}(k)$, $y \leftarrow \Pi(k_b, x)$, and output $z \leftarrow M(k_2, (b, y))$. Here, $\Pi(k_b, x)$ is called recursively.
- $\Pi^{-1}(k, z)$: Run $(k_0, k_1, k_2) \leftarrow \text{PRG}(k)$, $(b, y) \leftarrow M^{-1}(k_2, z)$ and Compute $x \leftarrow \Pi^{-1}(k_b, y)$ and output (b, x) . Here, $\Pi^{-1}(k_b, y)$ is called recursively.
- $\text{Permute}(k, (z \ z+1), c)$: Let $(k_0, k_1, k_2) \leftarrow \text{PRG}(k)$. Compute $(b_0, x_0) \leftarrow \Pi^{-1}(k, z)$ and $(b_1, x_1) \leftarrow \Pi^{-1}(k, z+1)$. We break into two cases:
 - If $b_0 \neq b_1$ output the permuted key $k^{(z \ z+1), c} = (k_0, k_1, k_2^{(z \ z+1), c})$ where $k_2^{(z \ z+1), c} \leftarrow \text{Permute}'(k_2, (z \ z+1), c)$.
 - If $b_0 = b_1 = b$, output the permuted key $k^{(z \ z+1), c} = (\overline{k_0}, \overline{k_1}, k_2)$ where $\overline{k_{1-b}} = k_{1-b}$ and $\overline{k_b} = k_b^{(y \ y+1), c} \leftarrow \text{Permute}(k_b, (y \ y+1), c)$ where $(b, y) \leftarrow M^{-1}(k_2, z)$. Here, Permute is recursively called.
- $\Pi(k^{(z \ z+1), c}, x)$: Run $\Pi(k, x)$ exactly as above but swapping out k_0, k_1, k_2 with their permuted versions if necessary. Likewise define $\Pi^{-1}(k^{(z \ z+1), c}, x)$.

Theorem 51. *If PRG is a secure PRG and $(M, M^{-1}, \text{Punc}')$ is a secure ONS-Merge, then the tuple $(\Pi, \Pi^{-1}, \text{Punc})$ in Construction 47 is a secure ONS-PRP. In particular, if PRG is ϵ_{PRG} -secure and $(M, M^{-1}, \text{Punc}')$ is ϵ_M -secure, then $(\Pi, \Pi^{-1}, \text{Punc})$ is ϵ_Π -secure where $\epsilon_\Pi(\lambda, n) \leq O((\epsilon_{\text{PRG}}(\lambda) + \epsilon_M(\lambda, N)) \times \log N)$. Moreover, the permutations $\Pi(k, \cdot)$ and $\Pi^{-1}(k, \cdot)$ are $(O(N^4), \text{polylog}(N))$ -decomposable.*

Proof. First, we argue efficiency. A call to Π on block-size N makes a call to PRG, a call to M on block-size N , and a recursive call to Π on block-size at most $\lceil N/2 \rceil$. Solving the recurrence, Π makes $O(\log N)$ calls to PRG and $O(\log N)$ calls to M on block-sizes no more than N . Since PRG and M are polynomial time, this is efficient. We can likewise analyze the other algorithms.

Next we need to argue the correctness guarantees. In particular, we need to argue that Permute makes called to $\text{Permute}'$ on neighbor swaps where the pre-images of the swapped points have different initial bits. Moreover, we need to verify that Π using a punctured key computes the correct function.

For the case $n = 1$, Π is just XORing with the key k . the only available neighbor swap is $(0 \ 1)$, which is the same as XORing with 1, which is equivalent to XORing the key with 1. Thus, the $n = 1$ case achieves perfect security. Now we handle the $n > 1$ case.

Consider running $\text{Permute}(k, (z \ z + 1), c)$. Recall that $\text{Permute}'(k_2, (z \ z + 1), c)$ is called exactly when the first bits of $\Pi^{-1}(k, z)$ and $\Pi^{-1}(k, z + 1)$ are different. Notice that this is equivalent to the case where $M^{-1}(k_2, z)$ and $M^{-1}(k_2, z + 1)$ have different first bits, which means that this is a valid call to $\text{Permute}'$. In this case, the correctness of the permuted key follows immediately from the correctness of the underlying M .

Now consider the case where $\Pi^{-1}(k, z)$ and $\Pi^{-1}(k, z + 1)$ (or equivalently, $M^{-1}(k_2, z)$ and $M^{-1}(k_2, z + 1)$) share the same first bit. This implies that $M^{-1}(k_2, z)$ and $M^{-1}(k_2, z + 1)$ are adjacent (any supposed string between them would share the same first bit, and by order preserving, its image under $M(k_2, \cdot)$ would need to lie between $z, z + 1$, which is impossible). Thus, when $\text{Permute}(k_b, (y \ y + 1), c)$ is recursively called where $(b, y) \leftarrow M^{-1}(k_2, z)$, we have that $(b, y + 1) = M^{-1}(k_2, z + 1)$. Thus, the neighbor swap $(y \ y + 1)$ is actually transposing $M^{-1}(k_2, z)$ and $M^{-1}(k_2, z + 1)$. From this, correctness of the permuted key following immediately from the (inductively justified) correctness of the underlying Π for block-size $\lceil N/2 \rceil$.

Now we argue decomposability. To decompose, we simply decompose $M(k_2, \cdot)$, and then recursively decompose $\Pi(k_0, \cdot)$ and $\Pi(k_1, \cdot)$. Solving the recurrence gives the decomposition statement in Theorem 51.

Finally, we prove security. Let $\mathcal{A}$ be a supposed adversary for $(\Pi, \Pi^{-1}, \text{Permute})$ with winning probability $1/2 + \epsilon_\Pi(\lambda, n)$. Let $k \in \{0, 1\}^\lambda$ denote the experiments random key k , and let $k_0, k_1, k_2 \leftarrow \text{PRG}(k)$ denote the keys used to evaluate $\Pi, \Pi^{-1}, \text{Punc}$, and let z_0, z_1 denote $\mathcal{A}$'s chosen pair. We consider the following sequence of hybrids:

Hyb₀: This is the ONS-PRP security experiment. By assumption, the winning probability is at least $1/2 + \epsilon_\Pi(\lambda, N)$.

Hyb₁: This is the same experiment, except we switch to k_0, k_1, k_2 being uniformly random keys, and using them in all algorithms. By a straightforward reduction to the PRG security of PRG, we have that $\mathcal{A}$ still wins with probability at least $1/2 + \epsilon_\Pi(\lambda, N) - \epsilon_{\text{PRG}}(\lambda)$.

Hyb₂: This is the same as Hybrid 1, except that we change the win condition. Namely, if the punctured key $k^{\Gamma, c}$ contains a punctured k_2 , then the experiment aborts and outputs a random bit,

independent of $\mathcal{A}$. By a straightforward reduction to the ONS-Merge security of (M, M, Punc') , we have that $\mathcal{A}$ still wins with probability at least $1/2 + \epsilon_{\Pi}(\lambda, N) - \epsilon_{\text{PRG}}(\lambda) - \epsilon_M(\lambda, N)$.

Hyb₃: This is the same as Hybrid 1, except that we further change the win condition to always output a random bit. The only difference from Hybrid 2 is that when $k^{\Gamma, c}$ contains a punctured k_b , in Hybrid 2 the experiment outputs a bit dependent on $\mathcal{A}$'s output, whereas in Hybrid 3 it outputs a random bit. By a straightforward reduction to the ONS-Security of $(\Pi, \Pi^{-1}, \text{Punc})$ with block-size at most $\lceil N/2 \rceil$, we have that $\mathcal{A}$ still wins with probability at least $1/2 + \epsilon_{\Pi}(\lambda, N) - \epsilon_{\text{PRG}}(\lambda) - \epsilon_M(\lambda, N) - \epsilon_{\Pi}(\lambda, \lceil N/2 \rceil)$. But observe that in Hybrid 3, the winning probability is exactly $1/2$. Thus we have that

$$\epsilon_{\Pi}(\lambda, N) \leq \epsilon_{\text{PRG}}(\lambda) + \epsilon_M(\lambda, N) + \epsilon_{\Pi}(\lambda, \lceil N/2 \rceil)$$

For the base case, observe that when $N = 2$, Π is a random permutation and the punctured key contains no information, so we trivially have $\epsilon_{\Pi}(\lambda, 2) = 0$. Solving the recurrence gives the statement of the theorem. $\square$

5.4 Achieving OP-PRPs for Decomposable Permutations Using Obfuscation

Construction 52. Let $(\Pi, \Pi^{-1}, \text{Permute}')$ be an ONS-PRP (that is, an OP-PRP for the family of neighbor swaps $(z, z+1)$). Let $G = (G_n)_n$ be a family of efficiently computable permutations, and s a parameter. Then let $(\Pi, \Pi^{-1}, \text{Permute})$ be a new PRP where **Permute** does the following:

- **Permute** (k, Γ, b) : Let

$$P = \begin{cases} \text{iO}(1^\lambda, 1^s, \Pi(k, \cdot)) & \text{if } b = 0 \\ \text{iO}(1^\lambda, 1^s, \Gamma(\Pi(k, \cdot))) & \text{if } b = 1 \end{cases}$$

$$P^{-1} = \begin{cases} \text{iO}(1^\lambda, 1^s, \Pi^{-1}(k, \cdot)) & \text{if } b = 0 \\ \text{iO}(1^\lambda, 1^s, \Pi(k, \Gamma^{-1}(\cdot))) & \text{if } b = 1 \end{cases}$$

Output $k^{\Gamma, b} = (P, P^{-1})$.

Then we augment Π, Π^{-1} so that $\Pi((P, P^{-1}), x) = P(x)$ and $\Pi^{-1}((P, P^{-1}), z) = P^{-1}(z)$.

Observe that this construction preserves the decomposability of Π, Π^{-1} , since the algorithms are exactly the same.

Theorem 53. Suppose iO is ϵ -secure and $(\Pi, \Pi^{-1}, \text{Permute}')$ is δ -secure as an ONS-PRP. Suppose $G = (G_n)_n$ is (T, s') -decomposable. Let t be the maximum circuit size of Π, Π^{-1} after hardcoding the key (including permuted keys from **Permute'**). Then as long as $s \geq s' + t$, $(\Pi, \Pi^{-1}, \text{Permute})$ is $O(T(\epsilon + \delta))$ -secure as an OP-PRP for G .

Proof. Let $\Gamma_0, \dots, \Gamma_T = \Gamma$ be the sequence of permutations where $\Gamma_i = \Gamma_{i-1} \circ (z_i, z_i + 1)$ and Γ_0 is the identity. Our goal is to show that the pair $\text{iO}(1^\lambda, \Pi(k, \cdot)), \text{iO}(1^\lambda, \Pi^{-1}(k, \cdot))$ is indistinguishable from $\text{iO}(1^\lambda, \Gamma(\Pi(k, \cdot))), \text{iO}(1^\lambda, \Pi^{-1}(k, \Gamma^{-1}(\cdot)))$. To do so, we will introduce a sequence of hybrids:

Hyb_i: $\text{iO}(1^\lambda, 1^s \Gamma_i(\Pi(k, \cdot))), \text{iO}(1^\lambda, \Pi^{-1}(k, \Gamma_i^{-1}(\cdot)))$. In particular, Hybrid 0 is the case where we have $\text{iO}(1^\lambda, \Pi(k, \cdot)), \text{iO}(1^\lambda, \Pi^{-1}(k, \cdot))$ and Hybrid T is the case $\text{iO}(1^\lambda, \Gamma(\Pi(k, \cdot))), \text{iO}(1^\lambda, \Pi^{-1}(k, \Gamma^{-1}(\cdot)))$.

Hyb_{(i-1).1}: $\text{iO}(1^\lambda, 1^s, \Gamma_i(\Pi(k^{\{(z_i, z_i+1), 0\}}, \cdot))), \text{iO}(1^\lambda, 1^s, \Pi^{-1}(k^{\{(z_i, z_i+1), 0\}}, \Gamma_i^{-1}(\cdot)))$. Here, we sample $k^{\{(z_i, z_i+1), 0\}} \leftarrow \text{Permute}'(k, (z_i, z_i + 1), 0)$.

$\text{Hyb}_{(i-1).2}$: $\text{iO}(1^\lambda, 1^s, \Gamma_i(\Pi(k^{(z_i \ z_{i+1}),1}, \cdot))), \text{iO}(1^\lambda, 1^s, \Pi^{-1}(k^{(z_i \ z_{i+1}),1}, \Gamma_i^{-1}(\cdot)))$. Here, we sample $k^{(z_i \ z_{i+1}),1} \leftarrow \text{Permute}'(k, (z_i \ z_{i+1}), 1)$.

Observe that $\Pi(k^{(z_i \ z_{i+1}),0}, \cdot)$ is functionally equivalent to $\Pi(k, \cdot)$, and this equivalence is preserved by composing both sides with Γ_{i-1} . Likewise for the Π^{-1} functions. The sizes of the programs being obfuscated are at most $s' + t$, where t is the maximum size of the circuit Π after hardcoding the key (including permuted keys by $\text{Permute}'$). Thus, by iO security, Hybrids $i - 1$ and $(i - 1).1$ are indistinguishable, except with probability at most ϵ .

The only difference between Hybrids $(i - 1).1$ and $(i - 1).2$ is that we switch from $k^{(z_i \ z_{i+1}),0}$ to $k^{(z_i \ z_{i+1}),1}$. By the ONS-security of $(\Pi, \Pi^{-1}, \text{Permute}')$, these two hybrids are indistinguishable except with probability at most δ .

Finally, observe that $\Pi(k^{(z_i \ z_{i+1}),1}, \cdot)$ is functionally equivalent to $(z_i \ z_{i+1}) \circ \Pi(k, \cdot)$. Therefore $\Gamma_{i-1} \circ \Pi(k^{(z_i \ z_{i+1}),1}, \cdot)$ is functionally equivalent to $\Gamma_{i-1} \circ (z_i \ z_{i+1}) \circ \Pi(k, \cdot)$, which in turn is equivalent to $\Gamma_i \circ \Pi(k, \cdot)$. Thus, by iO security, Hybrids $(i - 1).2$ and i are indistinguishable except with probability at most ϵ .

Thus we obtain a chain of hybrids $0 \rightarrow 0.1 \rightarrow 0.2 \rightarrow 1 \rightarrow 1.1 \rightarrow 1.2 \rightarrow 2 \rightarrow \dots \rightarrow T - 1 \rightarrow (T - 1).1 \rightarrow (T - 1).2 \rightarrow T$ where each transition is indistinguishable. The triangle inequality then gives the theorem. $\square$

6 One-Shot Signatures in the Standard Model

Following our construction in an oracle model from Section 4, we present our standard model construction of non-collapsing collision resistant hash functions, which imply OSS.

Construction 54. Let $\lambda \in \mathbb{N}$ the statistical security parameter. Define $s := 16 \cdot \lambda$ and let $n, r, k \in \mathbb{N}$ such that $r := s \cdot (\lambda - 1)$, $n := r + \frac{3}{2} \cdot s$, $k := n$. Let $d := \text{poly}_d(\lambda) \in \mathbb{N}$ the expansion parameter and $\kappa := \text{poly}_\kappa(\lambda) \in \mathbb{N}$ the cryptographic security parameter, for some sufficiently large polynomials in the statistical security parameter.

Let iO an iO scheme, (F, Punc) a puncturable PRF, and $(\Pi, \Pi^{-1}, \text{Permute})$ a permutable PRP for the class of all $(2^{\text{poly}(\kappa)}, \text{poly}(\kappa))$ -decomposable permutations. Then we construct a hash function $(\text{Gen}, \text{Hash})$ as follows:

- $\text{Gen}(1^\lambda)$: Sample $k_{\text{in}}, k_{\text{out}}, k_{\text{lin}} \leftarrow \{0, 1\}^\kappa$. $\Pi(k_{\text{in}}, \cdot)$ is a permutation with domain $\{0, 1\}^n$, $\Pi(k_{\text{out}}, \cdot)$ is a permutation with domain $\{0, 1\}^d$, and $F(k_{\text{lin}}, \cdot)$ is a PRF with inputs in $\{0, 1\}^d$ that outputs some polynomial number of bits. Let $H(\cdot)$ denote the first r output bits of $\Pi(k_{\text{in}}, \cdot)$ and $J(\cdot)$ denote the remaining $n - r$ bits. For each $y \in \{0, 1\}^d$, let $\mathbf{A}(y) \in \mathbb{Z}_2^{k \times (n-r)}$ be a matrix with full column-rank and $\mathbf{b}(y) \in \mathbb{Z}_2^k$ a vector, both are generated pseudorandomly by the output of $F(k_{\text{lin}}, y)$.

As the common reference string output $\text{CRS} = (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D})$ where $\mathcal{P} \leftarrow \text{iO}(1^\kappa, P)$, $\mathcal{P}^{-1} \leftarrow$

$\text{iO}(1^\kappa, P^{-1}), \mathcal{D} \leftarrow \text{iO}(1^\kappa, D)$ such that

$$\begin{aligned}
P(x) &= (y, \mathbf{A}(y) \cdot J(x) + \mathbf{b}(y)) \text{ where } y \leftarrow \Pi^{-1}(k_{\text{out}}, H(x) || 0^{d-r}) \\
P^{-1}(y, \mathbf{u}) &= \begin{cases} \Pi^{-1}(w || \mathbf{z}) & \exists w, \mathbf{z} : (\Pi(k_{\text{out}}, y) = w || 0^{d-r}) \wedge (\mathbf{A}(y) \cdot \mathbf{z} + \mathbf{b}(y) = \mathbf{u}) \\ \perp & \text{else} \end{cases} \\
D(y, \mathbf{v}) &= \begin{cases} 1 & \text{if } \mathbf{v}^T \cdot \mathbf{A}(y) = 0^{n-r} \\ 0 & \text{otherwise} \end{cases}
\end{aligned}$$

- **Hash**(CRS, x): Compute $(y, \mathbf{u}) \leftarrow \mathcal{P}(x)$ and output y .

Changes made from the oracle to standard model construction. The two first obvious differences between the oracle model Construction 28 and the standard model Construction 54 are that (1) The standard model construction pseudorandomly generates the permutation Π and matrices $\mathbf{A}_y, \mathbf{b}_y$ rather than truly at random, and (2) the standard model construction obfuscates the programs rather than putting them in oracles. The third, less obvious change, is that the standard model construction additionally pads $H(x)$ with 0's and then applies $\Pi^{-1}(k_{\text{out}}, \cdot)$ in order to get $y \in \{0, 1\}^d$, rather than simply setting $y = H(x) \in \{0, 1\}^r$, which was done in the oracle construction.

The rationale behind expanding y (the input to $F(k_{\text{lin}}, \cdot)$) from r to d bits, comes from the adaptation of the last step of the oracle construction's security proof (Section 4.3 in the oracle construction), to the last step of the the standard model construction's security proof (Section 6.3 in the standard model construction). Specifically, one can recall that in the oracle security proof, in the last step, we showed that collision finding in the dual-free case can be used for collision finding in 2-to-1 random functions. Here, in the standard model, our replacement for 2-to-1 collision resistant hash functions will be LWE-based and approximate, as elaborated in Section 6.3, and the expansion of y is intended to deal with that.

Finally, note that our choice to use the inverse $\Pi^{-1}(k_{\text{out}}, \cdot)$ in P (i.e., rather than computing the permutation in the forward direction $\Pi(k_{\text{out}}, \cdot)$) may seem arbitrary. However, our choice corresponds to us ultimately using the second permutation as an *input*-permutable permutation, which is in turn equivalent to treating the inverse as the permutation, and then asking that it is output-permutable. We accomplish this exactly by using the inverse of Π as our forward-computation of the permutation. While in Section 5 we show that output-permutable and input-permutable PRPs are in fact equivalent (and also bilaterally-permutable PRPs), the explicit use of the inverse $\Pi^{-1}(k_{\text{out}}, \cdot)$ intends to give insight to the security proof.

Non-collapsing. The non-collapsing property of our standard model construction is identical to the oracle model non-collapsing procedure, shown in Proposition 29.

Security in the Standard Model. The rest of this section is for dedicated for showing security in the standard model. The results in Sections 6.1 (Lemma 56), 6.2 (Lemma 57) and 6.3 (Theorem 58) together, eventually prove the below theorem.

Theorem 55 (Collision Resistance of Hash). *Let $\text{Gen}(1^\lambda)$ the generation algorithm from Construction 54. Then, for every quantum polynomial-time algorithm $\mathcal{A}$ there exists a negligible function negl such that,*

$$\Pr \left[(x_0 \neq x_1) \wedge (\text{Hash}(x_0) = \text{Hash}(x_1)) : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \text{Gen}(1^\lambda) \\ (x_0, x_1) \leftarrow \mathcal{A}(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \end{array} \right] \leq \text{negl}(\lambda) .$$

Direct Construction of One-Shot Signatures from our Non-collapsing CRH - Standard Model. Our standard-model OSS is constructed from our standard-model non-collapsing CRH, in the exact same way it was done in the oracle model. That is, the standard-model OSS uses the non-collapsing CRH circuits in a black-box manner. This is described in Construction 31. Also similarly to the security of the oracle model OSS, the strong unforgeability of our standard model OSS follows from the collision-resistance of the function Hash from Construction 54.

6.1 Bloating the Dual

For $n', r', k' \in \mathbb{N}$, $s \in \mathbb{N} \cup \{0\}$ such that $r' + s \leq n' \leq k'$, we define the modified generator $\widetilde{\text{Gen}}(1^\lambda, n', r', k', s)$, as follows. It samples a distribution over $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'$, where the values of n, r, k in Construction 54 are replaced by n', r', k' respectively. For s we let $\mathbf{A}^{(0)}(y) \in \mathbb{Z}_2^{k' \times s}$ denote the first s columns of $\mathbf{A}(y) \in \mathbb{Z}_2^{k' \times (n' - r')}$ and $\mathbf{A}^{(1)}(y) \in \mathbb{Z}_2^{k' \times (n' - r' - s)}$ denote the remaining $n' - r' - s$ columns. Note that the standard generator is defined as $\widetilde{\text{Gen}}(1^\lambda, n, r, k, 0)$. The functionality of $\mathcal{P}, \mathcal{P}^{-1}$ stays the same and we define $\mathcal{D}'(y, \mathbf{v})$ as an indistinguishability obfuscation of the following function:

$$D'(y, \mathbf{v}) = \begin{cases} 1 & \text{if } \mathbf{v}^T \cdot \mathbf{A}^{(1)}(y) = 0^{n' - r' - s} \\ 0 & \text{otherwise} \end{cases}$$

Observe that if $\mathbf{v}^T \cdot \mathbf{A}(y) = 0^{n-r}$, then $\mathbf{v}^T \cdot \mathbf{A}^{(0)}(y) = 0^{n-r-s}$. Thus $\mathcal{D}'$ accepts all points that are accepted by $\mathcal{D}$, but also accepts additional points as well, namely those for which $\mathbf{v}^T \cdot \mathbf{A}^{(0)}(y) \neq 0^s$ but $\mathbf{v}^T \cdot \mathbf{A}_y^{(1)} = 0^{n-r-s}$. We call this bloating the dual.

Lemma 56. *Let $\lambda, n, r, k \in \mathbb{N}$, and assume there is a quantum algorithm $\mathcal{A}$ with complexity $T_{\mathcal{A}}$ such that,*

$$\Pr \left[(y_0 = y_1) \wedge (x_0 \neq x_1) : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \widetilde{\text{Gen}}(1^\lambda, n, r, k, 0) \\ (x_0, x_1) \leftarrow \mathcal{A}(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \right] \geq \epsilon .$$

For our primitives in Construction 54, let the iO scheme be $(f_{\text{iO}}(\cdot), \frac{1}{f_{\text{iO}}(\cdot)})$ -secure, the puncturable PRF be $(f_{\text{PRF}}(\cdot), \frac{1}{f_{\text{PRF}}(\cdot)})$ -secure, the permutable PRP be $(f_{\text{PRP}}(\cdot), \frac{1}{f_{\text{PRP}}(\cdot)})$ -secure and assume that Lossy Functions (as in Definition 17) are $(f_{\text{LF}}(\cdot), \frac{1}{f_{\text{LF}}(\cdot)})$ -secure. Also assume that injective OWFs are $(f_{\text{OWF}}(\cdot), \frac{1}{f_{\text{OWF}}(\cdot)})$ -secure for $f_{\text{OWF}}(\lambda) := 2^{\lambda^\delta}$ for some constant real number $\delta > 0$ and let $w := \lambda^{\frac{\delta}{2}}$, $s' := s - (n - r - s)$.

Assume $f_{\text{iO}}(\kappa), f_{\text{PRF}}(\kappa), f_{\text{PRP}}(\kappa), f_{\text{LF}}(w) \geq T_{\mathcal{A}} + \text{poly}'(\lambda)$ for some fixed polynomial $\text{poly}'(\cdot)$ and all of the following:

1. For $f := \min(f_{\text{IO}}, f_{\text{PRF}}, f_{\text{PRP}})$, $\frac{2^r}{f(\kappa)} \leq \frac{\epsilon}{k^{2.512}}$,
2. $\frac{1}{f_{\text{LF}}(w)} \leq \frac{\epsilon}{k^{2.256}}$,
3. $\frac{2^w \cdot (k^5 + \text{poly}(n-r-s) + T_{\mathcal{A}})}{f_{\text{OWF}}(n-r-s)} \leq \epsilon$, and
4. $\frac{4 \cdot (n-r-s) \cdot 2^w}{2^{s'}} \leq \epsilon$.

Then, it follows that,

$$\Pr \left[\begin{array}{l} y_0 = y_1 := y, \\ (\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}^{(1)}(y)) \end{array} : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \widetilde{\text{Gen}}(1^\lambda, n, r, k, s) \\ (x_0, x_1) \leftarrow \mathcal{A}(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \right] \geq \frac{\epsilon}{512 \cdot k^2}.$$

Note that $\mathbf{u}_0 - \mathbf{u}_1 \notin \text{ColSpan}(\mathbf{A}^{(1)}(y))$ means in particular that $\mathbf{u}_0, \mathbf{u}_1$, and hence x_0, x_1 , are distinct. Thus, the second expression means that $\mathcal{A}$ is finding collisions, but these collisions satisfy an even stronger requirement.

Proof. Let $\lambda \in \mathbb{N}$ and assume there is a $T_{\mathcal{A}}$ -complexity algorithm $\mathcal{A}$ and a probability ϵ such that $\mathcal{A}$ gets $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \text{Gen}(1^\lambda)$ and outputs a pair (x_0, x_1) of n -bit strings such that with probability ϵ we have $x_0 \neq x_1$ and $y_0 = y_1$. We next define a sequence of hybrid experiments. Each hybrid defines a computational process, an output of the process and a predicate computed on the process output. The predicate defines whether the (hybrid) process execution was successful or not.

- **Hyb₀**: The original execution of $\mathcal{A}$.

The process **Hyb₀** is the above execution of $\mathcal{A}$ on input a sample from the distribution $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \widetilde{\text{Gen}}(1^\lambda, n, r, k, 0)$. We define the output of the process as (x_0, x_1) and the process execution is considered as successful if x_0, x_1 are both distinct and collide in their y values. By definition, the success probability of **Hyb₀** is ϵ .

- **Hyb₁**: Preparing to switch to a bounded number of cosets $(\mathbf{A}(y), \mathbf{b}(y))$, by using an obfuscated puncturable PRF and injective mode of a lossy function.

Let $(\text{LF.KeyGen}, \text{LF.F})$ a $\left(f(\cdot), \frac{1}{f(\cdot)}\right)$ -secure lossy function scheme (as in Definition 17). Sample $\text{pk}_{\text{LF}} \leftarrow \text{LF.KeyGen}(1^d, 0, 1^w)$ where w is as defined in our Lemma statement, and let $\text{LF.F}(\text{pk}_{\text{LF}}, \cdot) : \{0, 1\}^d \rightarrow \{0, 1\}^m$ the induced injective function.

We now consider two circuits in order to describe our current hybrid. $E_0(k_{\text{lin}}, \cdot)$ is the circuit that given an input from $\{0, 1\}^d$ applies $F(k_{\text{lin}}, \cdot)$ to get $\mathbf{A}(y), \mathbf{b}(y)$. $E_1(\text{pk}_{\text{LF}}, k'_{\text{lin}}, \cdot)$ is the circuit that for a LF key pk_{LF} and P-PRF key k'_{lin} for a P-PRF with input size m rather than d , given a d -bit input y , applies the lossy function with key pk_{LF} and then the P-PRF to get $\mathbf{A}'(y), \mathbf{b}'(y)$.

Note that in the previous hybrid, the circuit E_0 is used in all three circuits $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}$ in order to generate the cosets per input y , and furthermore, each of these three circuits access E_0 only as a black box. The change that we make to the current hybrid is that we are going to use $E_1(\text{pk}_{\text{LF}}, k'_{\text{lin}}, \cdot)$ for a freshly sampled $\text{pk}_{\text{LF}}, k'_{\text{lin}}$, instead of E_0 . Since we are sending obfuscations $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D})$ of the three circuits and due to the three circuits accessing the samplers E_0, E_1 only as black boxes, it

follows by Lemma 19 that the output of the previous hybrid and the current hybrid are $\left(f(\kappa), \frac{|\mathcal{X}|}{f(\kappa)}\right)$ -indistinguishable, where $f := \min(f_{\text{iO}}, f_{\text{PRF}})$ and $\mathcal{X}$ is the set of all possible values of y , which has size 2^r (recall that while the y 's are of length $d \gg r$, these are a sparse set inside $\{0, 1\}^d$ because we are padding with zeros), and we also recall that κ , the cryptographic security parameter is some polynomial in the statistical security parameter λ . It follows by Condition 1 from our Lemma statement, that the success probability of the current process is $:= \epsilon_1 \geq \epsilon - \frac{2^r}{f(\kappa)} \geq \epsilon - \frac{\epsilon}{32} = \frac{31 \cdot \epsilon}{32}$.

- **Hyb₂**: Switching to a bounded number of cosets $(\mathbf{A}(y), \mathbf{b}(y))$, by using the lossy function.

The change from the previous hybrid to the current hybrid is that we are going to sample a lossy key $\text{pk}_{\text{LF}}^1 \leftarrow \text{LF.KeyGen}(1^d, 1, 1^w)$ and use it inside E_1 from the previous hybrid, instead of using an injective key $\text{pk}_{\text{LF}}^0 \leftarrow \text{LF.KeyGen}(1^d, 0, 1^w)$, which was used in the previous hybrid. Note that in this hybrid, there are at most 2^w cosets (that is, some different values of y will have the same coset), by the correctness of the lossy function scheme. By the security of the lossy function scheme, the output of this hybrid is $\left(f_{\text{LF}}(w), \frac{1}{f_{\text{LF}}(w)}\right)$ -indistinguishable from the previous hybrid. It follows by Condition 2 from our Lemma statement that the success probability of the current process is

$$:= \epsilon_2 \geq \epsilon_1 - \frac{1}{f_{\text{LF}}(w)} \geq \frac{31 \cdot \epsilon}{32} - \frac{\epsilon}{32} \geq \frac{30 \cdot \epsilon}{32}.$$

- **Hyb₃**: Using obfuscated (instead of plain) circuits for membership checks inside D , by using the security of the obfuscator that obfuscates D .

Recall that in the previous hybrid, the circuit D executes as follows: $D(y, \mathbf{v})$ computes $\mathbf{A}(y)$ by access to $\mathbf{F}(k_{\text{lin}}, \cdot)$, and then checks membership in the dual of $\text{ColSpan}(\mathbf{A}(y)) := S_y$. In the current hybrid we make the following change to D : We sample an additional P-PRF key k_S at the beginning of the hybrid. Given $\mathbf{A}(y)$, we apply $\mathbf{F}(k_S, \cdot)$ to obtain pseudorandomness and generate an obfuscation $\mathcal{O}_{S_y^\perp} \leftarrow \text{iO}(1^\kappa, S_y^\perp)$ of the circuit that checks membership inside $S_y^\perp$. The circuit D decides on the membership check of $\mathbf{v}$ using the obfuscated circuit $\mathcal{O}_{S_y^\perp}$ instead of the plain circuit $S_y^\perp$.

Note that by the correctness of the obfuscation scheme (specifically, the inner obfuscation scheme that was used to obfuscate the circuit $S_y^\perp$, for every y), we did not change the functionality of D . It follows by the security of the iO that obfuscates D , that the obfuscations between the two cases are $\left(f(\kappa), \frac{1}{f(\kappa)}\right)$ -indistinguishable, and thus the same can be said of the outputs of the hybrids. It follows in particular the success probability of the current process is

$$:= \epsilon_3 \geq \epsilon_2 - \frac{1}{f(\kappa)} \geq \frac{30 \cdot \epsilon}{32} - \frac{\epsilon}{32} = \frac{29 \cdot \epsilon}{32}.$$

- **Hyb₄**: Relaxing dual verification inside D to accept a larger subspace $T_y^\perp$ for every y , by using an obfuscated puncturable PRF over subspace hiding.

We now consider two circuits in order to describe our current hybrid, both of which are used only inside the circuit D . The first circuit $E_S(k_S, \cdot)$ is the circuit that given an input $\mathbf{A}(y)$ applies $\mathbf{F}(k_S, \cdot)$ to obtain pseudorandomness and generate an obfuscation $\mathcal{O}_{S_y^\perp} \leftarrow \text{iO}(1^\kappa, S_y^\perp)$. The second circuit $E_T(k_T, \cdot)$ is the circuit that given an input $\mathbf{A}(y)$ applies $\mathbf{F}(k_T, \cdot)$ to obtain pseudorandomness for

two things: (1) to sample $T_y^\perp \subseteq \{0, 1\}^k$ a superspace that contains $S_y^\perp$ and has s more dimensions (recall that S_y has $n - r$ dimensions, $S_y^\perp$ has $k - (n - r)$ dimensions and thus $T_y^\perp$ has $k - (n - r) + s$ dimensions), and (2) to generate an obfuscation $O_{T_y^\perp} \leftarrow \text{iO}(1^\kappa, T_y^\perp)$.

For concreteness, the way we use the randomness from $F(k_T, \cdot)$ is by generating a pseudorandom full rank matrix $M_y \in \mathbb{Z}_2^{(n-r) \times (n-r)}$, multiplying by $\mathbf{A}(y) \in \mathbb{Z}_2^{k \times (n-r)}$ to get $\overline{\mathbf{A}}(y) := \mathbf{A}(y) \cdot M_y \in \mathbb{Z}_2^{k \times (n-r)}$. We then take the last $n - r - s$ columns of $\overline{\mathbf{A}}(y)$, denote this sub-matrix by $\overline{\mathbf{A}}^{(1)} \in \mathbb{Z}_2^{k \times (n-r-s)}$ and define $\text{ColSpan}(\overline{\mathbf{A}}^{(1)})^\perp := T_y^\perp$.

Let $\mathcal{X}$ the set of all possible cosets that arise from the scheme, which has size $\leq 2^w$ by the lossy function. We would first like to note the indistinguishability, per coset $i \in \mathcal{X}$, between $O_{S_i^\perp}$ and $O_{T_i^\perp}$, for a known S_i and uniformly random appropriate superspace T_i . Specifically, we look at the indistinguishability for truly random bits for sampling T and for sampling the obfuscation in either of the cases. For this, we would like to use Lemma 24. Note that in our case, k , $k - n + r$ and s , take the role of k' , r' and s' respectively in the Lemma 24 statement and thus we get $\left(f_{\text{OWF}}(n - r - s), \frac{s}{f_{\text{OWF}}(n - r - s)}\right)$ -indistinguishability, where the indistinguishability is that of an injective OWF.

Note that in the previous hybrid, the circuit E_S is used in D , and furthermore, the access of D to E_S is only as a black box. The change that we make to the current hybrid is that we are going to use $E_T(k_T, \cdot)$ for a freshly sampled P-PRF key k_T instead of E_S (which is also sampled for a freshly sampled P-PRF key k_S). Since we are sending an obfuscation $\mathcal{D}$ of D and due to D accessing the samplers E_S , E_T only as black boxes, it follows by Lemma 19 that the output of the previous hybrid and the current hybrid are $\left(f_{\text{OWF}}(n - r - s), \frac{|\mathcal{X}|}{f_{\text{OWF}}(n - r - s)}\right)$ -indistinguishable, where $\mathcal{X}$ is the set of all cosets derived from our scheme, which has size $\leq 2^w$ by the lossy function. It follows by Condition 3 that the success probability of the current process is

$$:= \epsilon_4 \geq \epsilon_3 - \frac{2^w}{f_{\text{OWF}}(n - r - s)} \geq \frac{29 \cdot \epsilon}{32} - \frac{\epsilon}{32} = \frac{28 \cdot \epsilon}{32}.$$

- **Hyb₅**: Asking for sum of collisions to be outside of T_y , by an obfuscated puncturable PRF over dual-subspace anti-concentration.

This hybrid is the same as the previous in terms of execution, but we change the definition of a successful execution, that is, we change the predicate computed on the output of the process. We still ask that $(y_0 = y_1 := y)$, but instead of only asking the second requirement to be $(x_0 \neq x_1)$, we ask for a stronger condition: $(\mathbf{u}_0 - \mathbf{u}_1) \in (S_y \setminus T_y)$. Note that we are not going to need to be able to efficiently check for the success of the condition, but we'll prove that it happens with a good probability nonetheless.

Let ϵ_5 be the success probability of the current hybrid and note that $\mathcal{A}$ finds collisions with probability ϵ_4 in the previous hybrid **Hyb₄** (and since this hybrid is no different, the same goes for the current hybrid). Let $\mathcal{X} \subseteq \{0, 1\}^m$ the image of the lossy function $\text{LF.F}(\text{pk}_{\text{LF}}, \cdot)$ which we use to map our images y to cosets $(\mathbf{A}(y), \mathbf{b}(y))$, that is, there are $|\mathcal{X}|$ cosets and by the lossyness we know that $|\mathcal{X}| \leq 2^w$. For every value $\mathbf{x} \in \mathcal{X}$ denote by $\epsilon_4^{(\mathbf{x})}$ the probability to find a collision on value $\mathbf{x}$, or formally, that $y_0 = y_1 := y$, $x_0 \neq x_1$ and $\mathbf{x} = \text{LF.F}(\text{pk}_{\text{LF}}, y)$. We deduce $\sum_{\mathbf{x} \in \mathcal{X}} \epsilon_4^{(\mathbf{x})} = \epsilon_4$. Let L be a subset of $\mathcal{X}$ such that $\epsilon_4^{(\mathbf{x})} \geq \frac{\epsilon_4}{2 \cdot |\mathcal{X}|}$ and note that $\sum_{\mathbf{x} \in L} \epsilon_4^{(\mathbf{x})} \geq \frac{\epsilon_4}{2}$. We further define $\epsilon_5^{(\mathbf{x})}$ as the probability to find a strong (as in the notion of ϵ_5) collision on value $\mathbf{x}$, or formally, that

$y_0 = y_1 := y$, $(\mathbf{u}_0 - \mathbf{u}_1) \in (S_y \setminus T_y)$ and $\mathbf{x} = \text{LF.F}(\text{pk}_{\text{LF}}, y)$. Note that S_y, T_y are really functions of $\mathbf{x}$ rather than of y , so $(\mathbf{u}_0 - \mathbf{u}_1) \in (S_{\mathbf{x}} \setminus T_{\mathbf{x}})$ and also observe that $\sum_{\mathbf{x} \in \mathcal{X}} \epsilon_5^{(\mathbf{x})} = \epsilon_5$.

We would now like to use Lemma 27, so we make sure that we satisfy its requirements. Let any $\mathbf{x} \in L$, we know that by definition $\epsilon_4^{(\mathbf{x})} \geq \frac{\epsilon_4}{2 \cdot |\mathcal{X}|}$ and also recall that $\epsilon_4 \geq \frac{28 \cdot \epsilon}{32}$, $|\mathcal{X}| \leq 2^w$ and thus

$$\epsilon_4^{(\mathbf{x})} \geq \frac{\epsilon_4}{2 \cdot |\mathcal{X}|} \geq \frac{28 \cdot \epsilon}{64} \cdot \frac{1}{2^w} \geq \frac{\epsilon}{2 \cdot 2^w}.$$

Let $s' := s - (n - r - s)$ and for any $\mathbf{x} \in L$ let $\ell_{\mathbf{x}} := \frac{k^2}{\epsilon_4^{(\mathbf{x})}} \leq \frac{k^2 \cdot 2^w \cdot 2}{\epsilon}$. Note that by our Lemma 56 statement's Condition 4 we have (1) $\frac{(n-r-s)}{2^{s'}} \leq \frac{\epsilon_4^{(\mathbf{x})}}{2}$, and by Condition 3 we have (2) $\frac{\ell_{\mathbf{x}} \cdot (k^3 + \text{poly}(n-r-s) + T_{\mathcal{A}})}{f_{\text{OWF}}(n-r-s)} \leq \frac{1}{8}$. Since this satisfies Lemma 27, it follows that for every $\mathbf{x} \in L$ we have $\epsilon_5^{(\mathbf{x})} \geq \frac{\epsilon_4^{(\mathbf{x})}}{16 \cdot k^2} - \frac{1}{f(\kappa)}$ for $f := \min(f_{\text{iO}}, f_{\text{PRF}})$. The expression $\frac{1}{f(\kappa)}$ is subtracted because for each $\mathbf{x} \in \mathcal{X}$, in order to use Lemma 27, we need the randomness for the experiment to be genuinely random, which will necessitate us to invoke the security of the iO and puncturable PRF, which incurs the loss of $\frac{1}{f(\kappa)}$. It follows that

$$\epsilon_5 = \sum_{\mathbf{x} \in \mathcal{X}} \epsilon_5^{(\mathbf{x})} \geq \sum_{\mathbf{x} \in L} \epsilon_5^{(\mathbf{x})} \geq \sum_{\mathbf{x} \in L} \frac{\epsilon_4^{(\mathbf{x})}}{16 \cdot k^2} - \frac{|\mathcal{X}|}{f(\kappa)} \geq \frac{\left(\frac{\epsilon_4}{2}\right)}{16 \cdot k^2} - \frac{|\mathcal{X}|}{f(\kappa)} \geq \frac{\left(\frac{28 \cdot \epsilon}{64}\right)}{16 \cdot k^2} - \frac{2^w}{f(\kappa)} \geq \frac{\epsilon}{64 \cdot k^2}.$$

- **Hyb₆**: For every y , de-randomizing T_y and defining it as the column span of $\mathbf{A}(y)^{(1)} \in \mathbb{Z}_2^{k \times (n-r-s)}$, the last $n - r - s$ columns of the matrix $\mathbf{A}(y)$, by using permutable PRPs, security of iO and an obfuscated puncturable PRF.

This hybrid is the same as the previous, with the following change. Recall how we compute dual membership check inside the circuit D : Given y , we compute $\mathbf{x} = \text{LF.F}(\text{pk}_{\text{LF}}, y)$ we take the P-PRF key k_{in} and compute $\mathbf{A}(\mathbf{x}) = \mathbf{A}(y)$, then use the additional P-PRF key k_T to generate the random superspace $T_y^\perp$ of $S_y^\perp$ and additional randomness, and use both to generate an obfuscation $\text{O}_{T_y^\perp} \leftarrow \text{iO}(1^\kappa, T_y^\perp)$. Also recall that the subspace $T_y^\perp$ is generated as follows: Generate the matrix M_y using the P-PRF, then multiply it by $\mathbf{A}(y)$ to get $\overline{\mathbf{A}}(y)$, and then take the last $n - r - s$ columns of it as a basis for T_y , and $T_y^\perp$ is defined as the dual of that space. In the current hybrid we will not sample M_y and simply define $\overline{\mathbf{A}}_y := \mathbf{A}_y$. This means that T_y is defined to be the column span of $\mathbf{A}(y)^{(1)}$. We will also not obfuscate the membership check for $T_y^\perp$ inside the circuit D , and check membership by using a basis of $T_y^\perp$ in the plain. We next define hybrid experiments (which go in the direction from the previous hybrid Hyb₅ to the current hybrid Hyb₆) and explain why each consecutive pair are appropriately indistinguishable.

- **Using the security of the permutable PRP.** Assume we sample all components of our scheme, excluding the key k_{in} for the initial permutation Π on $\{0, 1\}^n$, and define the following permutation Γ on $\{0, 1\}^n$. Denote by h the first r bits of the input and by j the last $n - r$ bits of the input to the permutation Γ . Recall that in the circuits P, P^{-1}, D , the value y , the coset $\mathbf{A}(y), \mathbf{b}(y)$, the superspace $T_y^\perp$ and the associated matrix $M_y \in \mathbb{Z}_2^{(n-r) \times (n-r)}$, are all computed as a function of r bits, which in the construction take the role of $H(x)$. In fact, all of the above variables can be written as a function of $h \in \{0, 1\}^r$ instead of as a function of y .

The permutation Γ takes h and computes M_h (the matrix for computing T_h from the matrix $\mathbf{A}(h)$), interprets j as a vector in $\mathbb{Z}_2^{n-r}$, and applies M_h to j . Note that this change means we are taking the puncturable PRF key k_T which is used to sample $T_y^\perp$ (and formally, is used to sample the pseudorandomness for generating the matrix M_y and for obfuscating membership check for $T_y^\perp$), which previously only existed inside the circuit D , and putting it also inside the circuits P, P^{-1} , because this key is needed in order to compute the permutation Γ .

Recall the examples of decomposable permutations in Section 5. For every value $h \in \{0, 1\}^r$ observe that multiplication by M_h is a permutation (and moreover an affine permutation, which is decomposable efficiently). Then, Γ is a controlled permutation as described in the examples of decomposable permutations in Section 5, which is controlled on decomposable permutations. Overall, we deduce that Γ is a $(2^{\text{poly}(\lambda)}, \text{poly}(\lambda))$ -decomposable permutation. We use the permutable PRP Π , and switch to a setting where we use the key k_{in}^Γ that applies Γ to the output of Π (and Γ^{-1} to the input of Π), instead of just applying Π and its inverse. By the security of the permutable PRP (Lemma 42), this change is $\left(f_{\text{PRP}}(\kappa), \frac{1}{f_{\text{PRP}}(\kappa)}\right)$ -indistinguishable.

- **Using the security of outside iO.** For every $h \in \mathbb{Z}_2^r$, we make the following change to P, P^{-1} . Instead of composing the permutation Γ to the output of Π , it applies Π as it is. However, when generating the matrix $\mathbf{A}(y)$, it multiplies by M_y to get $\overline{\mathbf{A}}(y) := \mathbf{A}(y) \cdot M_y$. An additional change we will make not to P, P^{-1} but to the circuit D , is that we will not obfuscate the membership check circuit for $T_y^\perp$ and simply use its available basis. One can observe that we did not change the functionality of P, P^{-1}, D in any of the above changes and thus by the security of the indistinguishability obfuscation that obfuscates $\mathcal{P} \leftarrow \text{iO}(1^\kappa, P)$, $\mathcal{P}^{-1} \leftarrow \text{iO}(1^\kappa, P^{-1})$, $\mathcal{D} \leftarrow \text{iO}(1^\kappa, D)$, this change is $\left(f_{\text{iO}}(\kappa), \frac{1}{f_{\text{iO}}(\kappa)}\right)$ -indistinguishable.
- **Using an obfuscated puncturable PRF argument over the choice of the matrix $\mathbf{A}(y)$, for every y .** Note that after we did the last step, our process for generating the cosets, in all tree circuits P, P^{-1}, D is the following: Given y we compute $\mathbf{x}$ with the lossy function, and then apply two different puncturable PRFs (that is, with i.i.d keys, k_{lin} and k_T) to obtain $(\mathbf{A}(\mathbf{x}), \mathbf{b}(\mathbf{x}))$ and the matrix $M_{\mathbf{x}} \in \mathbb{Z}_2^{(n-r) \times (n-r)}$. We then multiply to get the matrix that we are actually using, i.e., $\overline{\mathbf{A}}(\mathbf{x}) := \mathbf{A}(\mathbf{x}) \cdot M_{\mathbf{x}}$. The change we next make is to have one fresh puncturable PRF key k'_{lin} and use it to generate the coset $(\mathbf{A}(\mathbf{x}), \mathbf{b}(\mathbf{x}))$, without further generating the matrix $M_{\mathbf{x}}$.

By a standard argument using an obfuscated puncturable PRF like we used numerous times in this proof (i.e., we use lemma 19 and the fact that when using real randomness, the two ways to sample $\mathbf{A}(y)$ are statistically equivalent), we get that this change is $\left(f(\kappa), 2^w \cdot \frac{1}{f(\kappa)}\right)$ -indistinguishable, for $f := \min(f_{\text{iO}}, f_{\text{PRF}})$.

Observe that after the last change we are exactly in the setting of Hyb_6 , and we started in Hyb_5 . It follows in particular that the success probability of the current process is

$$:= \epsilon_6 \geq \epsilon_5 - \frac{2^w}{f(\kappa)} \geq \frac{\epsilon}{64 \cdot k^2} - \frac{\epsilon}{128 \cdot k^2} = \frac{\epsilon}{128 \cdot k^2}.$$

- **Hyb₇:** Going back to using 2^r cosets rather than $\leq 2^w$, by moving from lossy mode to injective mode in the lossy function.

We make the exact same change we made between Hyb_1 to Hyb_2 , but in the opposite direction. That is we sample an injective key $\text{pk}_{\text{LF}}^0 \leftarrow \text{LF.KeyGen}(1^d, 0, 1^w)$ and use it instead of the previous lossy key $\text{pk}_{\text{LF}}^1 \leftarrow \text{LF.KeyGen}(1^d, 1, 1^w)$, which is used in the previous hybrid. By the exact same argument (which relies on the security of the lossy function), the output of this hybrid is $(f_{\text{LF}}(w), \frac{1}{f_{\text{LF}}(w)})$ -indistinguishable. It follows in particular the success probability of the current process is

$$:= \epsilon_7 \geq \epsilon_6 - \frac{1}{f_{\text{LF}}(w)} \geq \frac{\epsilon}{128 \cdot k^2} - \frac{\epsilon}{256 \cdot k^2} = \frac{\epsilon}{256 \cdot k^2} .$$

- Hyb_8 : Stop using the lossy function, by an obfuscated puncturable PRF.

We make the exact same change we made between Hyb_0 to Hyb_1 , but in the opposite direction. That is, we drop the lossy function LF.F altogether and apply the PRF to y directly and not to $\mathbf{x}$, the output of the lossy function on input y . By the exact same argument (which relies on Lemma 19), the output of this hybrid is $(f(\kappa), \frac{|\mathcal{X}|}{f(\kappa)})$ -indistinguishable for $f := \min(f_{\text{IO}}, f_{\text{PRF}})$, where $\mathcal{X}$ is the set of all possible values of y , which has size 2^r (recall that while the y 's are of length $d \gg r$, they are a sparse set inside $\{0, 1\}^d$ because they are padded with zeros). It follows that the success probability of the current process is

$$:= \epsilon_8 \geq \epsilon_7 - \frac{2^r}{f(\kappa)} \geq \frac{\epsilon}{256 \cdot k^2} - \frac{\epsilon}{512 \cdot k^2} = \frac{\epsilon}{512 \cdot k^2} .$$

To conclude, note that the generated obfuscations in the final hybrid Hyb_8 form exactly the distribution $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \widetilde{\text{Gen}}(1^\lambda, n, r, k, s)$. This finishes our proof. $\square$

6.2 Simulating the Dual

Our next step is to show that an adversary which has access to the dual-free setting can simulate the CRS for an adversary in the restricted setting, where the dual verification check is bloated.

Lemma 57. *Let $\lambda, n, r, k \in \mathbb{N}$ and assume there is a quantum algorithm $\mathcal{A}$ running in time $T_{\mathcal{A}}$ such that,*

$$\Pr \left[\begin{array}{l} y_0 = y_1 := y, \\ (\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}^{(1)}(y)) \end{array} : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \widetilde{\text{Gen}}(1^\lambda, n, r, k, s) \\ (x_0, x_1) \leftarrow \mathcal{A}(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \right] \geq \epsilon .$$

Assume that the primitives used in Construction 54 are $(f(\cdot), \frac{1}{f(\cdot)})$ -secure, and assume $\frac{2^r}{f(\kappa)} \leq o(1)$. Then, there is a quantum algorithm $\mathcal{B}$ running in time $T_{\mathcal{A}} + \text{poly}(\lambda)$ such that

$$\Pr \left[\begin{array}{l} (\overline{y}_0 = \overline{y}_1) \wedge (\overline{x}_0 \neq \overline{x}_1) : \\ (\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}, \overline{\mathcal{D}}) \leftarrow \widetilde{\text{Gen}}(1^\lambda, r + s, r, k - (n - r - s), 0) \\ (\overline{x}_0, \overline{x}_1) \leftarrow \mathcal{B}(\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}) \\ (\overline{y}_b, \overline{\mathbf{u}}_b) \leftarrow \overline{\mathcal{P}}(x_b) \end{array} \right] \geq \frac{\epsilon}{2} .$$

Proof. We first describe the actions of the algorithm $\mathcal{B}$ (which will use the code of $\mathcal{A}$ as part of its machinery) and then argue why it breaks collision resistance with the appropriate probability. Given $\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}$ which comes from $(\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}, \overline{\mathcal{D}}) \leftarrow \widetilde{\text{Gen}}(1^\lambda, r + s, r, k - (n - r - s), 0)$, the algorithm $\mathcal{B}$ does the following:

- Sample a P-PRF key k_C that outputs some sufficient (polynomial) amount of random bits on an d -bit input, and sample a permutable PRP key k_Γ for a PRP on domain $\{0, 1\}^n$. Define the following circuits.
- $(y \in \mathbb{Z}_2^d, \mathbf{u} \in \mathbb{Z}_2^k) \leftarrow P(x \in \mathbb{Z}_2^n)$:
 - $(\bar{x} \in \mathbb{Z}_2^{r+s}, \tilde{x} \in \mathbb{Z}_2^{n-r-s}) \leftarrow \Pi(k_\Gamma, x)$.
 - $(y \in \mathbb{Z}_2^d, \bar{\mathbf{u}} \in \mathbb{Z}_2^{k-(n-r-s)}) \leftarrow \bar{P}(\bar{x})$.
 - $(\mathbf{C}(y) \in \mathbb{Z}_2^{k \times k}, \mathbf{d}(y) \in \mathbb{Z}_2^{n-r-s}) \leftarrow F(k_C, y)$.
 - $\mathbf{u} \leftarrow \mathbf{C}(y) \cdot \begin{pmatrix} \bar{\mathbf{u}} \\ \tilde{x} + \mathbf{d}(y) \end{pmatrix}$.
- $(x \in \mathbb{Z}_2^n) \leftarrow P^{-1}(y \in \mathbb{Z}_2^d, \mathbf{u} \in \mathbb{Z}_2^k)$:
 - $(\mathbf{C}(y) \in \mathbb{Z}_2^{k \times k}, \mathbf{d}(y) \in \mathbb{Z}_2^{n-r-s}) \leftarrow F(k_C, y)$.
 - $\begin{pmatrix} \bar{\mathbf{u}} \\ \tilde{x} \end{pmatrix} \leftarrow \mathbf{C}(y)^{-1} \cdot \mathbf{u} - \begin{pmatrix} 0^{k-(n-r-s)} \\ \mathbf{d}(y) \end{pmatrix}$.
 - $(\bar{x} \in \mathbb{Z}_2^{r+s}) \leftarrow \bar{P}^{-1}(y, \bar{\mathbf{u}})$.
 - $x \leftarrow \Pi^{-1}(k_\Gamma, (\bar{x}, \tilde{x}))$.
- $D'(y \in \mathbb{Z}_2^d, \mathbf{v} \in \mathbb{Z}_2^k) \in \{0, 1\}$:
 - $(\mathbf{C}(y) \in \mathbb{Z}_2^{k \times k}, \mathbf{d}(y) \in \mathbb{Z}_2^{n-r-s}) \leftarrow F(k_C, y)$.
 - $\mathbf{A}^{(1)}(y) := \text{last } n - r - s \text{ columns of } \mathbf{C}(y)$.
 - Output 1 iff $\mathbf{v}^T \cdot \mathbf{A}^{(1)}(y) = \mathbf{0}^{n-r-s}$.
- Use indistinguishability obfuscation in order to generate the input for $\mathcal{A}$: $\mathcal{P} \leftarrow \text{iO}(1^\kappa, P)$, $\mathcal{P}^{-1} \leftarrow \text{iO}(1^\kappa, P^{-1})$, $\mathcal{D}' \leftarrow \text{iO}(1^\kappa, D')$.

The remainder of the reduction is simple: $\mathcal{B}$ executes $(x_0, x_1) \leftarrow \mathcal{A}(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}')$ and then $(\bar{x}_b, \tilde{x}_b) \leftarrow \Pi(k_\Gamma, x_b)$ and outputs $(\bar{x}_0, \bar{x}_1)$. Assume that the output of $\mathcal{A}$ satisfies $y_0 = y_1 := y$ and also $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}(y)^{(1)})$, and recall that $\mathbf{A}(y)^{(1)} \in \mathbb{Z}_2^{k \times (n-r-s)}$ are the last $n - r - s$ columns of the matrix $\mathbf{A}(y) \in \mathbb{Z}_2^{k \times (n-r)}$, which is generated by the reduction. We explain why it is necessarily the case that $\bar{x}_0 \neq \bar{x}_1$.

First note that due to how we defined the reduction, $\mathbf{A}(y) := \mathbf{C}(y) \cdot \begin{pmatrix} \bar{\mathbf{A}}(y) \\ \mathbf{I}_{n-r-s} \end{pmatrix}$, where $\bar{\mathbf{A}}(y) \in \mathbb{Z}_2^{(k-(n-r-s)) \times s}$ is the matrix arising from $\bar{P}, \bar{P}^{-1}$ and $\mathbf{I}_{n-r-s} \in \mathbb{Z}_2^{(n-r-s) \times (n-r-s)}$ is the identity matrix of dimension $n - r - s$. Also note that because $\mathbf{C}(y), \bar{\mathbf{A}}(y)$ are full rank then $\mathbf{A}(y)$ is full rank. Now, since $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}(y)^{(1)})$ and since $\mathbf{A}(y)^{(1)}$ are the last $n - r - s$ columns of $\mathbf{A}(y)$, it follows that if we consider the coordinates vector $\mathbf{x} \in \mathbb{Z}_2^{n-r}$ of $(\mathbf{u}_0 - \mathbf{u}_1)$ with respect to $\mathbf{A}(y)$, the first s elements are not 0^s . By linearity of matrix multiplication it follows that if we look at each of the two coordinates vectors $\mathbf{x}_0, \mathbf{x}_1$ (each has $n - r$ bits) for $\mathbf{u}_0, \mathbf{u}_1$, respectively, somewhere in the first s bits, they differ. Now, recall how we obtain the first s bits of $\mathbf{x}_b$ – this is

exactly by applying $\bar{\Pi}$ (the permutation on $\{0,1\}^{r+s}$ arising from $\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}$) to $\bar{x}_b$ and taking the last s bits of the output. Since these bits differ in the output of the permutation, then the preimages have to differ, i.e., $\bar{x}_0 \neq \bar{x}_1$.

Define $\epsilon_{\mathcal{B}}$ as the probability that the output of $\mathcal{A}$ indeed satisfies $y_0 = y_1 := y$ and also $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}^{(1)}(y))$, and it remains to give a lower bound for the probability $\epsilon_{\mathcal{B}}$. We do this by a sequence of hybrids, eventually showing that the view which $\mathcal{B}$ simulates to $\mathcal{A}$ is computationally indistinguishable from a sample from $\widetilde{\text{Gen}}(1^\lambda, n, r, k, s)$. More precisely, each hybrid describes a process, it has an output, and a success predicate on the output.

- **Hyb₀**: The above distribution $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \mathcal{B}(\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1})$, simulated to the algorithm $\mathcal{A}$.

The first distribution is defined in the reduction above. The output of the process is the output (x_0, x_1) of $\mathcal{A}$. The process execution is considered as successful if $y_0 = y_1 := y$ and $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}(y)^{(1)})$.

- **Hyb₁**: Not applying the inner permutation $\bar{\Pi}_{\text{in}}$ (which comes from the circuits $\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}$), by using the security of an obfuscated permutable PRP.

Let $\bar{\Pi}_{\text{in}}$ the (first) permutable PRP that's inside $\bar{\mathcal{P}}$ (which is the obfuscation of the circuit $\bar{P}$). In the previous hybrid we apply the n -bit permutable PRP $\Pi(k_{\Gamma}, \cdot)$ to the input $x \in \mathbb{Z}_2^n$ and then proceed to apply the inner permutation $\bar{\Pi}_{\text{in}}(\bar{k}_{\text{in}}, \cdot)$ to the first (i.e. leftmost) $r + s$ output bits of the first permutation $\Pi(k_{\Gamma}, \cdot)$. The change we make to the current hybrid is that we simply apply only $\Pi(k_{\Gamma}, \cdot)$.

Recall two details: (1) By Theorem 41, the inner permutable PRP $\bar{\Pi}_{\text{in}}(\bar{k}_{\text{in}}, \cdot)$ is in and of itself $(2^{\text{poly}(\lambda)}, \text{poly}(\lambda))$ -decomposable, and (2) the circuits P, P^{-1} which apply the permutations are both obfuscated by iO to be generate the obfuscations $\mathcal{P}, \mathcal{P}^{-1}$. We can treat it as a fixed permutation that acts on the output of the permutation $\Pi(k_{\Gamma}, \cdot)$ and thus it follows by Lemma 42 that the current and previous hybrids are computationally indistinguishable, with indistinguishability $\frac{1}{f(\kappa)}$.

- **Hyb₂**: For every $y \in \mathbb{Z}_2^d$, taking $\mathbf{A}(y)$ to be the direct output of the PRF F , by using an obfuscated punctured PRF.

In order to describe the change between the current and previous hybrid we first recall the structure of the circuits from the previous hybrid: In the previous hybrid, for every $y \in \mathbb{Z}_2^r$ we defined $\mathbf{A}(y) := \mathbf{C}(y) \cdot \begin{pmatrix} \bar{\mathbf{A}}(y) \\ \mathbf{I}_{n-r-s} \end{pmatrix}$, where $\mathbf{C}(y) \in \mathbb{Z}_2^{k \times k}$ comes from the output $F(k_{\mathbf{C}}, y)$ and $\bar{\mathbf{A}}(y) \in \mathbb{Z}_2^{(k-(n-r-s)) \times s}$ is the output of the inner PRF $\bar{F}(\bar{k}_{\text{lin}})$ (which in turn comes from the inside of $(\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1})$). In the current hybrid we are going to ignore the PRFs $F(k_{\mathbf{C}}, y)$ and $\bar{F}(\bar{k}_{\text{lin}})$ and their generated values $\mathbf{C}(y)$, $\mathbf{d}(y)$ and $\bar{\mathbf{A}}(y)$ and instead, sample a fresh key $k_{\mathbf{A}}$, and on query y generate $\mathbf{A}(y) \leftarrow F(k_{\mathbf{A}}, y)$, for $\mathbf{A}(y) \in \mathbb{Z}_2^{k \times (n-r)}$.

First, note that the following two ways to sample $\mathbf{A}(y)$, are statistically equivalent for every y : (1) The matrix $\mathbf{A}(y)$ is generated by sampling a random full-rank matrix $\mathbf{C}(y) \in \mathbb{Z}_2^{k \times k}$ and letting $\mathbf{A}(y)$ be $\mathbf{C}(y) \cdot \begin{pmatrix} \bar{\mathbf{A}}(y) \\ \mathbf{I}_{n-r-s} \end{pmatrix}$. (2) For every $y \in \mathbb{Z}_2^r$ just sample a full-rank matrix

$\mathbf{A}(y) \in \mathbb{Z}_2^{k \times (n-r)}$. This means that when truly random bits are used for generating $\mathbf{A}(y)$ in the two cases, the distributions are statistically equivalent.

To see why the two distributions are computationally indistinguishable, a different description of the previous hybrid can be given as follows: We can consider a sampler E_0 that for every $y \in \{0,1\}^d$ samples $\mathbf{A}(y)$ according to the first algorithm, and another sampler E_1 that samples $\mathbf{A}(y)$ according to the second algorithm, and we know that for every y (and recall there are 2^r actual values of y which can appear as the output, and not 2^d) the outputs of E_0 and E_1 are statistically indistinguishable.

Since there are 2^r valid values for y , by Lemma 19, the current hybrid is computationally indistinguishable from the previous, with indistinguishability $\frac{2^r}{f(\kappa)}$.

- **Hyb₃**: Discarding the inner obfuscations $(\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1})$ completely, by using the security of the outer obfuscator.

The change between the current hybrid and the previous is that in the current hybrid we generate the circuits P, P^{-1}, D' without using $(\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1})$ at all. Note that this is possible, since in the previous hybrid, we moved to a circuit that did not use access to the circuits $(\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1})$ any longer during the execution of any of the three circuits P, P^{-1}, D' , except from using the second permutation $\overline{\Pi}_{\text{out}}$, which acts on $\{0,1\}^d$ and does not need to act from inside the inner obfuscations $(\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1})$ any more. This means that we can technically move the application of the inner permutation $\overline{\Pi}_{\text{out}}$ "outside of the inner circuits $(\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1})$ " and the functionality of the circuits P, P^{-1}, D' did not change between the current and the previous hybrids, and thus, by the security of the indistinguishability obfuscator that obfuscates the three circuits, the current hybrid is computationally indistinguishable from the previous one, with indistinguishability of $\frac{1}{f(\kappa)}$.

Finalizing the reduction. Finally, observe that the distribution generated in the above Hyb₃ is exactly a sample from $\widetilde{\text{Gen}}(1^\lambda, n, r, k, s)$. Also observe that the outputs of Hyb₀ and Hyb₃ are $O\left(\frac{2^r}{f(\kappa)}\right)$ -computationally indistinguishable. Recall that by the lemma's assumptions, with probability ϵ , on a sample from $\widetilde{\text{Gen}}(1^\lambda, n, r, k, s)$, the algorithm $\mathcal{A}$ outputs a pair (x_0, x_1) of n -bit strings such that $y_0 = y_1 := y$ and also $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}(\mathbf{A}^{(1)}(y))$. It follows that the probability for the same event when the input to $\mathcal{A}$ is generated by Hyb₀, is at least $\epsilon - O\left(\frac{2^r}{f(\kappa)}\right) \geq \frac{\epsilon}{2}$, which finishes our proof. $\square$

6.3 Hardness of the Dual-free Case from LWE and iO

Here, we explain how to prove the collision resistance of the dual-free case (where the adversary sees $\mathcal{P}, \mathcal{P}^{-1}$, but not $\mathcal{D}$). This will follow the blueprint used in Section 4.3 to reduce the dual-free case to a standard collision-resistance problem (which we recommend going over, before reading the following standard model security reduction). However, a few technical challenges arise. Some of these are due to needing certain steps to be efficient, where they are naively inefficient in Section 4.3. Another issue is that the underlying 2-to-1 function we use based on LWE [BCM⁺18] is not uniformly 2-to-1, but rather has some points that have no collisions and are only 1-to-1. By

careful arguments using the novel permutable PRPs as a tool, we are nevertheless able to resolve these issues and prove security. In the following section all arithmetic is implicitly modulo 2, unless explicitly noted otherwise (and then the arithmetic modulo or lack thereof will be specified).

The LWE-based hash function L . Here, we recall an *approximate* 2-to-1 function L based on LWE which is a simplified version of the *noisy claw-free trapdoor function* developed in [BCM⁺18]. Let $u, v, \sigma, B, \bar{B}, q$ be parameters with the relationships described in Equation 1.

$$\begin{aligned} \sigma &= u^{\Omega(1)}, & \bar{B} &= \sigma \times u^{\Omega(1)}, \\ B &\geq \bar{B} \times u^{\omega(1)}, & q &\geq B \times u^{\Omega(1)}, \\ v &\geq \Omega(u \log q), & \exists \delta \in (0, 1) : \frac{q}{\sigma} &\leq 2^{u^\delta}. \end{aligned} \tag{1}$$

The keys for the hash function have the form $\mathbf{pk} = (\mathbf{B}, \mathbf{c})$, where $\mathbf{B} \leftarrow \mathbb{Z}_q^{v \times u}$ and $\mathbf{c} \leftarrow \mathbf{B} \cdot \mathbf{s} + \mathbf{e} \bmod q$ where $\mathbf{s} \leftarrow \mathbb{Z}_q^u$ and the entries of $\mathbf{e} \in \mathbb{Z}_q^v$ are i.i.d. sampled from discrete Gaussians of width σ , which in turn are guaranteed (w.h.p) to have entries in $(-\bar{B}, \bar{B}]$.

We define the function $L(\mathbf{pk}, \cdot) : \mathbb{Z}_q^u \times (-B, B]^v \times \{0, 1\} \rightarrow \mathbb{Z}_q^v$ as follows.

$$L((\mathbf{B} \in \mathbb{Z}_q^{v \times u}, \mathbf{c} \in \mathbb{Z}_q^v), (\mathbf{t} \in \mathbb{Z}_q^u, \mathbf{f} \in (-B, B]^v), b \in \{0, 1\}) = \mathbf{B} \cdot \mathbf{t} + \mathbf{f} + b \cdot \mathbf{c} \bmod q.$$

By choosing B, q to be powers of 2, we can map the domain and range to bit-strings.

L is collision resistant. Observe that $L(\mathbf{pk}, \cdot)$ is collision resistant, under the LWE assumption. This is because (with probability exponentially close to 1 in the security parameter, over sampling $\mathbf{pk}$) a collision is always of the form $(\mathbf{t}'_0, 0), (\mathbf{t}'_1, 1)$, and subtraction reveals $\mathbf{s}' = \mathbf{t}_0 - \mathbf{t}_1$, which breaks the assumption that search LWE with sub-exponential modulus/noise ratio is hard for quantum polynomial-time algorithms.

L is approximately 2-to-1. Observe that with overwhelming probability over sampling $\mathbf{pk}$, the function $L(\mathbf{pk}, \cdot)$ is *almost* 2-to-1. Formally, with overwhelming probability over sampling $\mathbf{e}, \mathbf{B}$ we have that for any two different lattice points $\mathbf{B} \cdot \mathbf{u}, \mathbf{B} \cdot \mathbf{v}$, the sets $\mathbf{B} \cdot \mathbf{u} + (-B, B]^v \bmod q, \mathbf{B} \cdot \mathbf{v} + (-B, B]^v \bmod q$ are non-intersecting (this follows because $v \geq \Omega(u \log q)$ is known to imply that the lattice is sparse with high probability). This in turn implies that for any point in the set $S := \{\mathbf{B} \cdot \mathbf{u} + (-(B - \|\mathbf{e}\|), (B - \|\mathbf{e}\|))^v \mid \mathbf{u} \in \mathbb{Z}_q^u\}$, the only two points in the domain of L that reach it are $(\mathbf{t}', 0), (\mathbf{t}' - \mathbf{s}', 1)$. Also, due to our parameter choices, (1) From $\bar{B} \geq \sigma \times u^{\Omega(1)}$ it follows that with high probability over sampling $\mathbf{e}$, the norm $\|\mathbf{e}\|$ is bounded by $\bar{B}$ and (2) From $B \geq \bar{B} \times u^{\omega(1)}$ it follows that $\bar{B}$ (and thus also $\|\mathbf{e}\|$) is negligible compared to B , which makes the set S be an overwhelming fraction of the entire range $\{\mathbf{B} \cdot \mathbf{u} + (-B, B]^v \mid \mathbf{u} \in \mathbb{Z}_q^u\}$.

L has a trapdoor to find preimage sets. Similarly to the noisy trapdoor claw-free functions from [BCM⁺18], the function L has a trapdoor that can invert it. Specifically, it is known how to efficiently sample a pair $(\mathbf{pk}, \mathbf{td})$ where $\mathbf{pk}$ is statistically indistinguishable from an honestly sampled public key for L , and given y an element in the image of $L(\mathbf{pk}, \cdot)$ and $\mathbf{td}$, it is possible to efficiently compute the preimage set of y (which, as we know by now, is of size either 1 or 2).

Theorem 58. *Let $n, r, k \in \mathbb{N}$ such that $r < n \leq k$ and also $\frac{n}{n-r}$ is an integer (which implies that $\frac{r}{n-r}$ is also an integer). Suppose the hardness of LWE against quantum polynomial-time algorithms, for the parameters as set in Equation 1. Note that this in particular implies the collision resistance of the LWE-based function L defined above.*

*Then, for every quantum polynomial-time algorithm $\mathcal{A}$ there exists a negligible function negl , such that the probability to get $(\mathcal{P}, \mathcal{P}^{-1})$ sampled from $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \widetilde{\text{Gen}}(1^\lambda, n, r, k, 0)$, and then output a collision in **Hash** (the function derived from $\mathcal{P}$), is $\text{negl}\left(\frac{n}{n-r}\right)$.*

Proof. Suppose there exists an adversary $\mathcal{A}$ which, given $(\mathcal{P}, \mathcal{P}^{-1})$ as sampled from $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \widetilde{\text{Gen}}(1^\lambda, n, r, k, 0)$ (recall that $\mathcal{A}$ gets the sampled circuits without the dual $\mathcal{D}$), finds a collision in the associated function $\text{Hash}(x) := \Pi^{-1}(k_{\text{out}}, H(x) || 0^{d-r})$ with non-negligible probability ϵ . We will describe an adversary $\mathcal{B}$ that violates the quantum hardness of LWE, using the adversary $\mathcal{A}$.

The collision resistant, approximate coset partition, trapdoor hash Q . We will use the above function L in order to construct the function Q . We start with defining an instance of L : We choose u, q, B, v in a way which satisfies Equation 1 and also such that $2 \cdot q^u \cdot (2B)^v = 2^{n/(n-r)}$. In other words, this makes the domain of the hash function L be exactly $\frac{n}{n-r}$ bits, and its output size be $v \cdot \log_2(q)$ bits (recall we assume that $\frac{n}{n-r}$ is an integer and we set q to be a power of 2). There is one more requirement for our parameters: We take d (recall that d is the number of bits that the second permutation $\Pi(k_{\text{out}}, \cdot)$ acts on) to be at least the sum of the sizes (in bits) of the input and output of L , multiplied by $(n-r)$, that is,

$$(n-r) \cdot \left(\frac{n}{n-r} + v \cdot \log(q) \right) = n + (n-r) \cdot v \cdot \log(q) \leq d.$$

We set Q to be a $(n-r)$ -parallel repetition of L , namely we sample $n-r$ i.i.d. public keys $\text{pk}_1, \dots, \text{pk}_{n-r}$ for L , which form the key pk for Q , and compute the $n-r$ outputs of L given $n-r$ inputs.¹⁹ It follows that Q maps n input bits to $(n-r) \cdot v \cdot \log_2(q)$ output bits, and furthermore by our setting of parameters, d is at least the sum of sizes (in bits) of the input and output sizes of Q – this will come in handy later in the proof.

As a further specification of the format for computing Q , the input $\mathbf{w} \in \{0,1\}^n$ to Q is given in two parts: The first, vector part, consists of r bits and is written $(\mathbf{w}_1, \dots, \mathbf{w}_{n-r})$ (note $\frac{r}{n-r} = \frac{n}{n-r} - 1$) where for each $i \in [n-r]$ we have $\mathbf{w}_i \in \{0,1\}^{\frac{r}{n-r}}$, and the second, coordinates part, consists of $n-r$ bits $(b_1, \dots, b_{n-r})$. For computing Q , for every $i \in [n-r]$ we take $\mathbf{w}_i$ and b_i and apply $L(\text{pk}_i, \cdot)$. We then take the output $\mathbf{a}_i \in \{0,1\}^{v \cdot \log(q)}$ and set it as the i -th packet in the output $\mathbf{a} \in \{0,1\}^{(n-r) \cdot v \cdot \log(q)}$ of Q .

Note the following details about Q :

- **Collision resistance.** A collision in Q with input size n constitutes a collision in L with input size $\frac{n}{n-r}$. It follows that based on the quantum hardness of LWE, for every quantum polynomial time adversary there is a negligible function negl such that the probability to find a collision in Q is bounded by $\text{negl}\left(\frac{n}{n-r}\right)$.

¹⁹The construction of Q from L is almost identical to the oracle construction of a coset partition function from Theorem 37, where we construct a collision-resistant coset partition function from a random 2-to-1 function.

- **Approximate coset partition.** Due to L being approximately 2-to-1 and Q being a parallel repetition of it, it follows that Q is an approximate coset partition function (Definition 35). Specifically, we know that for an overwhelming fraction of the outputs of L , the preimage set is a coset of dimension 1, thus it follows that for an overwhelming fraction of the outputs of Q , the preimage set is a coset of dimension $n - r$.
- **Existence of a (secret) trapdoor that allows efficient computation of coset description.** The concatenation of the trapdoors $(\text{td}_1, \dots, \text{td}_{n-r})$ of the $n - r$ instances of L gives us a trapdoor td for Q in the following sense: Given an element $\mathbf{a}$ in the image of Q and the trapdoor, one can compute a coset description of the preimage set of $\mathbf{a}$.

To see this, recall that the output of Q is given by the concatenation of the $n - r$ outputs of L , and for each of them, the number of preimages is either 1 or 2, and furthermore, these can be computed by the independent trapdoors for each of the instances of L . Eventually, the computed coset description of $\mathbf{a}$ is given as follows: For each $i \in [n - r]$ and $b \in \{0, 1\}$, let $\mathbf{w}_{i,b} \in \{0, 1\}^{\frac{n}{n-r}}$ the preimage of the i -th part $\mathbf{a}_i \in \{0, 1\}^{v \cdot \log(q)}$ (of $\mathbf{a} \in \{0, 1\}^{(n-r) \cdot v \cdot \log_2(q)}$), in the function $L(\text{pk}_i, \cdot)$, such that it ends with b . If there is no such preimage that ends with b we set $\mathbf{w}_{i,b} = 0^{\frac{n}{n-r}}$. Note that while it isn't necessarily the case that there are two images (one for each $b \in \{0, 1\}$), it is guaranteed that at least one of them always exists.

- The coset shift is set to

$$\bar{\mathbf{b}}_{\mathbf{a}} := \sum_{i \in [n-r]} \mathbf{w}'_{i,0} ,$$

where the vector $\mathbf{w}'_{i,0} \in \mathbb{Z}_2^n$ is derived from $\mathbf{w}_{i,0} \in \mathbb{Z}_2^{\frac{n}{n-r}}$ by using the previously described input format to Q . Namely, we split $\mathbf{w}'_{i,0} \in \mathbb{Z}_2^n$ into two parts, first part of r bits and a second part of $n - r$ bits. We put the first $\frac{n}{n-r} - 1 = \frac{r}{n-r}$ bits of $\mathbf{w}_{i,0}$ as the i -th packet in the first, r -bit part of $\mathbf{w}'_{i,0}$, and the remaining bit of $\mathbf{w}_{i,0}$ we set as the i -th bit in the second, $n - r$ -bit part of $\mathbf{w}'_{i,0}$, and the rest of the bits of $\mathbf{w}'_{i,0}$ are set to 0.

- As for the subspace associated with the coset, denote by $\bar{\mathbf{A}}_{\mathbf{a}} \in \mathbb{Z}_2^{n \times n-r}$ the subspace basis, and for $i \in [n - r]$ we set the i -th column of the basis to be $\mathbf{w}'_{i,0} + \mathbf{w}'_{i,1}$, where for $b \in \{0, 1\}$, the vector $\mathbf{w}'_{i,b} \in \{0, 1\}^n$ is derived from $\mathbf{w}_{i,b} \in \{0, 1\}^{\frac{n}{n-r}}$ in the same way described above when computing the coset shift.
- **Public efficient computation of coordinate vector for preimage element.** Q allows anyone to efficiently compute coordinate vectors of preimages with respect to its corresponding cosets. Given any input $\mathbf{w} \in \{0, 1\}^n$ to Q , one can take the last $n - r$ bits of $\mathbf{w}$ to obtain a coordinates vector $\mathbf{z} \in \mathbb{Z}_2^{n-r}$. Then, if one further gets access to the coset description $(\bar{\mathbf{A}}_{\mathbf{a}}, \bar{\mathbf{b}}_{\mathbf{a}})$, we have the equality

$$\mathbf{w} = \bar{\mathbf{A}}_{\mathbf{a}} \cdot \mathbf{z} + \bar{\mathbf{b}}_{\mathbf{a}} .$$

In that sense, the vector $\mathbf{z}$ which is efficiently computable from the preimage $\mathbf{w}$ is the coordinates vector of $\mathbf{w}$ with respect to the coset $(\bar{\mathbf{A}}_{\mathbf{a}}, \bar{\mathbf{b}}_{\mathbf{a}})$.

Now, given our definition of the hash function Q and its above special properties, we are ready to describe our reduction.

The reduction $\mathcal{B}$ from collision finding in Q to collision finding in Hash. The following reduction is an adaptation of the oracle reduction from Theorem 36 to the standard model, using LWE, obfuscation and permutable PRPs. Given $\mathbf{pk} = (\mathbf{pk}_1, \dots, \mathbf{pk}_{n-r})$ a public key for Q (where $\mathbf{pk}_i$ is the i.i.d. sampled public key for L), the reduction $\mathcal{B}$ samples permutable PRP keys $k_{\text{in}}, k_{\text{out}}$ and a puncturable PRF key k'_{lin} . Denote by $\ell := (n-r) \cdot v \cdot \log_2(q)$ the output size of Q , and we next define the functions P, P^{-1} which we then obfuscate to get the circuits $\mathcal{P}, \mathcal{P}^{-1}$, which we will feed to $\mathcal{A}$.

- $P(x \in \{0, 1\}^n)$:
 1. $\mathbf{w} \leftarrow \Pi(k_{\text{in}}, x)$,
 2. $\mathbf{a} \leftarrow Q(\mathbf{pk}, \mathbf{w})$,
 3. $y \leftarrow \Pi^{-1}(k_{\text{out}}, (\mathbf{a} || 0^{d-\ell}))$,
 4. $(\mathbf{C}_y \in \mathbb{Z}_2^{k \times n}, \mathbf{d}_y \in \mathbb{Z}_2^k) \leftarrow F(k'_{\text{lin}}, y)$,
 5. $\mathbf{u} \leftarrow \mathbf{C}_y \cdot \mathbf{w} + \mathbf{d}_y$,
 6. Output $(y \in \{0, 1\}^d, \mathbf{u} \in \mathbb{Z}_2^k)$.
- $P^{-1}(y \in \{0, 1\}^d, \mathbf{u} \in \mathbb{Z}_2^k)$:
 1. $x \leftarrow \begin{cases} \Pi^{-1}(k_{\text{in}}, \mathbf{w}) & \exists \mathbf{w} \in \mathbb{Z}_2^n \text{ such that } \mathbf{C}_y \cdot \mathbf{w} + \mathbf{d}_y = \mathbf{u} \\ \perp & \text{if no such } \mathbf{w} \text{ exists} \end{cases}$
 2. Output $\begin{cases} x & \text{if } x \neq \perp \text{ and } y = \Pi^{-1}(k_{\text{out}}, (\mathbf{a} || 0^{d-\ell})), \text{ for } \mathbf{a} \leftarrow Q(\mathbf{pk}, \mathbf{w}) \\ \perp & \text{otherwise} \end{cases}$

$\mathcal{B}$ obfuscates the two circuits to get $\mathcal{P}, \mathcal{P}^{-1}$ and executes $(x_0, x_1) \leftarrow \mathcal{A}(\mathcal{P}, \mathcal{P}^{-1})$. The output of $\mathcal{B}$ is $(\mathbf{w}_0 := \Pi(k_{\text{in}}, x_0), \mathbf{w}_1 := \Pi(k_{\text{in}}, x_1))$ as a collision in Q . Let $\epsilon_{\mathcal{B}}$ the probability that $\mathcal{B}$ outputs a collision in Q and we will show that if ϵ is non-negligible then so is $\epsilon_{\mathcal{B}}$. We next define a sequence of hybrid experiments. Each hybrid defines a computational process, an output of the process and a predicate computed on the process output. The predicate defines whether the (hybrid) process execution was successful or not.

- Hyb_0 : The original execution of the reduction $\mathcal{B}$.

Here, $\mathbf{pk}$ is sampled as a public key for the function Q and we execute $(\mathbf{w}_0, \mathbf{w}_1) \leftarrow \mathcal{B}(\mathbf{pk})$. The output of this hybrid is $(\mathbf{w}_0, \mathbf{w}_1)$ and the process is defined as successful if the pair constitutes a collision in $Q(\mathbf{pk}, \cdot)$. By definition, the success probability of this hybrid is $\epsilon_0 := \epsilon_{\mathcal{B}}$. The program P in this hybrid is described in Figure 8.

- Hyb_1 : Generating $\mathbf{u}$ from a coordinates vector $\mathbf{z}$ and the coset $(\overline{\mathbf{A}}_{\mathbf{a}}, \overline{\mathbf{b}}_{\mathbf{a}})$ instead of the preimage $\mathbf{w}$, by using the trapdoor of Q and the security of the iO. Also, changing back to asking for collisions in the original $\mathcal{P}$.

Here, the public key $\mathbf{pk}$ for Q is sampled together with a trapdoor td . The change we make to the following hybrid is this: In the previous hybrid, we took $\mathbf{a}$ and computed $y \in \{0, 1\}^d$, then

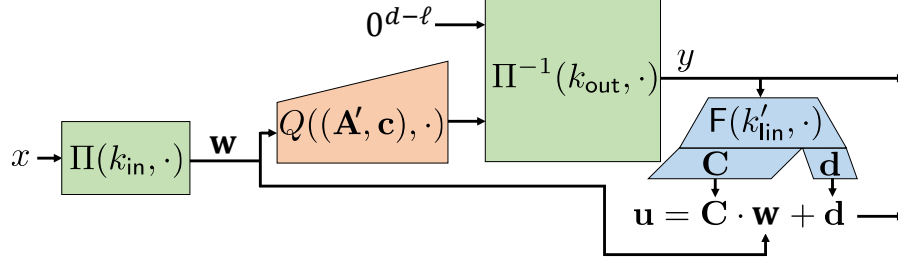


Figure 8: The obfuscated program P in Hybrid 0. We simply apply Q to the output $\mathbf{w}$ of $\Pi(k_{\text{in}}, \cdot)$. The output $\mathbf{u}$ is set to a pseudorandomly-generated affine function of $\mathbf{w}$.

computed $(\mathbf{C}_y, \mathbf{d}_y) \leftarrow F(k'_{\text{lin}}, y)$ and set $\mathbf{u} \leftarrow \mathbf{C}_y \cdot \mathbf{w} + \mathbf{d}_y$. Here, we compute $(\mathbf{C}_y, \mathbf{d}_y)$ all the same, but instead of using $\mathbf{w}$ to compute $\mathbf{u}$, we use the coordinates vector $\mathbf{z}$ of $\mathbf{w}$, the output y and the trapdoor td . In other words, we will maintain the same functionality but move to a circuit structure that more resembles the original distribution P, P^{-1} , by (1) having a circuit that given $y \in \{0, 1\}^d$ computes the pseudorandom coset description $(\mathbf{A}_y, \mathbf{b}_y)$, and the vector $\mathbf{u}$ is generated by $\mathbf{A}_y \cdot \mathbf{z} + \mathbf{b}_y$ where $\mathbf{z}$ will be the last $n - r$ output bits of the first permutation $\Pi(k_{\text{in}}, \cdot)$.

Specifically, recall that given an image $\mathbf{a} \in \{0, 1\}^{(n-r) \cdot v \cdot \log(q)}$ of $Q(\text{pk}, \cdot)$ and the trapdoor td , we can efficiently compute the coset $(\bar{\mathbf{A}}_{\mathbf{a}}, \bar{\mathbf{b}}_{\mathbf{a}})$ and also, given an input $\mathbf{w}$ we can compute $\mathbf{z}$ the coordinates vector of $\mathbf{w}$ with respect to the coset of the output $\mathbf{a}$. Given y we use the key k'_{lin} to compute $\mathbf{a}$ and then td to compute $(\bar{\mathbf{A}}_{\mathbf{a}}, \bar{\mathbf{b}}_{\mathbf{a}})$. Now, we take $\mathbf{z}$ the last $n - r$ bits of $\mathbf{w}$ and set $\mathbf{u} \leftarrow \mathbf{A}_y \cdot \mathbf{z} + \mathbf{d}_y$ for $\mathbf{A}_y := \mathbf{C}_y \cdot \bar{\mathbf{A}}_{\mathbf{a}}, \mathbf{d}_y := \mathbf{d}_y + \mathbf{C}_y \cdot \bar{\mathbf{b}}_{\mathbf{a}}$.

The sampling of pk with a trapdoor td is statistically indistinguishable from sampling pk without one, and furthermore by the correctness of the trapdoor, the functionality of the circuits did not change. Thus, the circuits given to $\mathcal{A}$ in Hyb_0 are computationally indistinguishable by the security of the iO that obfuscates the circuits P, P^{-1} . It follows in particular that the success probability of the current process is $:= \epsilon_1$ such that $\epsilon_{\mathcal{B}} \geq \epsilon_1 - \text{negl}(\lambda)$.

Another change that we make to this hybrid the definition of successful execution: Instead of asking for collisions in Q , we ask for collisions in the original $\mathcal{P}$. Since we move back and forth between collisions in Q and $\mathcal{P}$ using a permutation, any collision in Q can be translated into a collision in $\mathcal{P}$. Formally the reduction $\mathcal{B}$ changes minimally: After executing $\mathcal{A}$ and obtaining (x_0, x_1) we do not apply $\Pi(k_{\text{in}}, \cdot)$ in the end to obtain $\mathbf{w}_0, \mathbf{w}_1$. The success probability still satisfies $\epsilon_{\mathcal{B}} \geq \epsilon_1 - \text{negl}(\lambda)$.

- **Hyb₂**: Discarding the trapdoor td and computing the coset $(\mathbf{A}_y, \mathbf{b}_y)$ as a function of y alone, using an obfuscated puncturable PRF.

The change we will make to the following hybrid is to the circuit that samples the coset $(\mathbf{A}_y, \mathbf{b}_y)$. Specifically, instead of sampling the coset through the process from previous hybrid (setting $\mathbf{A}_y := \mathbf{C}_y \cdot \bar{\mathbf{A}}_{\mathbf{a}}, \mathbf{d}_y := \mathbf{d}_y + \mathbf{C}_y \cdot \bar{\mathbf{b}}_{\mathbf{a}}$ for the pair $(\bar{\mathbf{A}}_{\mathbf{a}}, \bar{\mathbf{b}}_{\mathbf{a}})$ arising from the trapdoor td and the pseudorandomly generated $(\mathbf{C}_y, \mathbf{d}_y) \leftarrow F(k'_{\text{lin}}, y)$), we just sample a fresh puncturable PRF key k_{lin} and sample the coset description from scratch $(\mathbf{A}_y \in \mathbb{Z}_2^{k \times (n-r)}, \mathbf{b}_y \in \mathbb{Z}_2^k) \leftarrow F(k_{\text{lin}}, y)$.

Note that in the first distribution (arising from the sampling process of Hyb_1), the matrix $\bar{\mathbf{A}}_{\mathbf{a}}$ is full rank and thus for a truly random $(\mathbf{C}_y, \mathbf{d}_y)$, the pair $(\mathbf{A}_y, \mathbf{b}_y)$ is a truly random coset. To see why

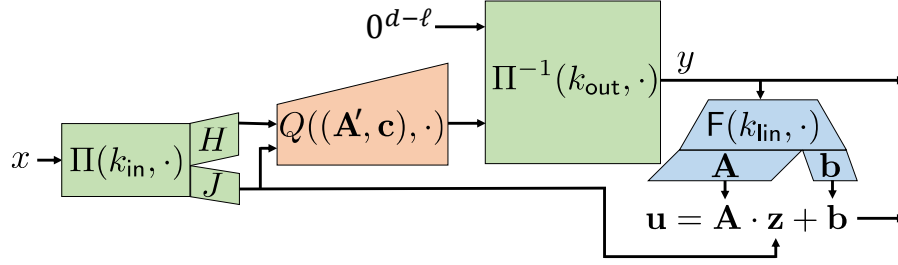


Figure 9: The obfuscated program P in Hybrid 2. The output $\mathbf{w}$ of $\Pi(k_{\text{in}}, \cdot)$ is still the input to Q . However, instead of generating the output vector $\mathbf{u}$ as a function of $\mathbf{w}$, it is a pseudorandomly-generated affine function of $\mathbf{z}$, the last $n - r$ bits of $\mathbf{w}$.

the two distributions are computationally indistinguishable, a different description of the previous hybrid can be given as follows: We can consider a sampler E_0 that for every $y \in \{0, 1\}^d$ samples $\mathbf{A}(y)$ according to the first algorithm, and another sampler E_1 that samples $\mathbf{A}(y)$ according to the second algorithm, and we know that for every y (and recall there are at most 2^ℓ actual values of y which can appear as the output, and not 2^d) the outputs of E_0 and E_1 are statistically equivalent.

Since there are $\leq 2^\ell$ valid values for y , by Lemma 19, the current hybrid is computationally indistinguishable from the previous, with indistinguishability $\frac{2^\ell}{f(\kappa)}$. It follows in particular that the success probability of the current process is $:= \epsilon_2$ such that $\epsilon_B \geq \epsilon_2 - \text{negl}(\lambda)$. Note that once we moved to this hybrid, the program P in the current hybrid applies the original circuit P from the Construction 54, only applying Q in the middle, between the two permutations $\Pi(k_{\text{in}}, \cdot)$ and $\Pi(k_{\text{out}}, \cdot)$, this is described in Figure 9. The rest of our hybrids are intended to get rid of Q and gradually embed it into the obfuscated circuits.

- **Hyb₃**: Describing a circuit C with equivalent functionality to Q and switching to using it instead, by using the security of iO.

We next describe a circuit $C : \{0, 1\}^n \rightarrow \{0, 1\}^\ell$ with identical functionality (but different circuit) than Q . The goal of using C is that (1) we can switch to it securely by the security of the iO that obfuscates the circuits P , P^{-1} , and (2) the structure of C will be described in steps, such that each step is securely removable. Specifically, by the security of an obfuscated permutable PRP (in some instances using the security of the first PRP $\Pi(k_{\text{in}}, \cdot)$, and sometimes the second one $\Pi(k_{\text{out}}, \cdot)$) we will be able to provably remove it from the insides of the programs P , P^{-1} , using these steps. Recall that the only difference between the current hybrid and the original distribution over P , P^{-1} (from Construction 54) is the circuit C in between the permutations $\Pi(k_{\text{in}}, \cdot)$ and $\Pi(k_{\text{out}}, \cdot)$, and thus, once we remove all of its parts indistinguishably, our proof will be complete.

Recall that the original Q is a $(n - r)$ -parallel repetition of $L : \{0, 1\}^{\frac{n}{n-r}} \rightarrow \{0, 1\}^{v \cdot \log(q)}$, where for each $L_i(\mathbf{pk}_i, \cdot) := L_i((\mathbf{B}_i, \mathbf{c}_i), \cdot)$ we have $\mathbf{c}_i = \mathbf{B}_i \cdot \mathbf{s}_i + \mathbf{e}_i$ for the secret vectors $\mathbf{s}_i \in \mathbb{Z}_q^u$, $\mathbf{e}_i \in (-B, B]^v$. As we no longer rely on the security of L (nor Q) for the remainder of the hybrids, we can use this knowledge in the following circuit C , which has the following 4 steps.

1. **Permutation π , correctly acts on almost all inputs.** We apply a permutation π which

acts on $\{0, 1\}^n$, and for each of the $n - r$ slices (each of which takes $\frac{n}{n-r}$ bits) acts as follows:

$$\begin{aligned} \left(\mathbf{w}_i \in \{0, 1\}^{\frac{r}{n-r}}, b_i \in \{0, 1\} \right) &:= \left(\mathbf{t}_i \in \mathbb{Z}_q^u, \mathbf{f}_i \in (-B, B]^v, b_i \in \{0, 1\} \right) \\ &\mapsto \left(\mathbf{t}'_i = \mathbf{t}_i + b_i \cdot \mathbf{s}_i \bmod q, \mathbf{f}'_i = \mathbf{f}_i + b_i \cdot \mathbf{e}_i \bmod (-B, B], b_i \right) \\ &:= \left(\mathbf{w}'_i \in \{0, 1\}^{\frac{r}{n-r}}, b_i \right), \end{aligned}$$

where $h \bmod (-B, B]$ means the unique integer $h' \in (-B, B]$ such that $h - h'$ is a multiple of $2B$. Thanks to the fact that we sum $\bmod (-B, B]$ in $\mathbf{f}'_i$ we can say that π is indeed a permutation, but observe that for an exponentially small fraction of the inputs (specifically, inputs such that $\mathbf{f}'_i = \mathbf{f}_i + b_i \cdot \mathbf{e}_i \bmod (-B, B]$ does not equal $\mathbf{f}_i + b_i \cdot \mathbf{e}_i \bmod q$), we are losing information in $\mathbf{f}'_i$ with respect to the larger range q . Our next two steps of C are intended to correct this loss of information.

2. **Computing correction signal for faulty inputs.** This step maps n bits to n bits in the following way, which sometimes erases information. Given n input bits $\mathbf{w} \in \{0, 1\}^n$ first we allocate $n - r$ new bits $w \in \{0, 1\}^{n-r}$, initially set to value 0^{n-r} . Intuitively, for $i \in [n - r]$ the bit w_i records whether or not the adding of $\mathbf{e}_i$ to $\mathbf{f}_i$ modulo $(-B, B]$ from the previous step caused an out-of-bounds vector. Formally, we set w_i to be 1 iff both $b_i = 1$ and $(\mathbf{f}'_i - \mathbf{e}_i \bmod q) \notin (-B, B]^v$ (we call such instances of $\mathbf{f}_i$ that satisfy this condition, "bad"). The output of C is $((w_1, \dots, w_{n-r}), (\mathbf{w}'_1, \dots, \mathbf{w}'_{n-r}))$. From hereon the circuit C disregards $\mathbf{z} := (b_1, \dots, b_{n-r})$ the last $n - r$ bits of $\mathbf{w}$.

3. **Injective function τ , correcting the faulty inputs from π .** Let $\tilde{B} := 3 \cdot (B + \overline{B})$ and recall $n := (n-r) + (n-r) \cdot (u \cdot \log(q) + v \cdot \log(2 \cdot B))$. This part of the computation injectively maps n input bits into $(n-r) \cdot (u \cdot \log(q) + v \cdot \log(2 \cdot \tilde{B})) > n$ output bits. τ computes in slices and for every $i \in [n - r]$ the input is $(w_i, \mathbf{w}'_i)$ where $\mathbf{w}'_i := (\mathbf{t}'_i \in \mathbb{Z}_q^u, \mathbf{f}'_i \in [-B, B])$ and the output is $(\mathbf{t}''_i \in \mathbb{Z}_q^u, \mathbf{f}''_i \in [-\tilde{B}, \tilde{B}])$ (discarding the bit w_i , but enlarging the range of $\mathbf{f}'_i$). Ideally, to compute the correction, all we need to do is

$$(w_i, \mathbf{f}'_i) \mapsto \mathbf{f}''_i = (\mathbf{f}'_i - w_i \cdot \mathbf{e}_i \bmod (-B, B]) + w_i \cdot \mathbf{e}_i \bmod (-(B + \overline{B}), (B + \overline{B}]) ,$$

discarding the input bit w_i . However one can see that this will sometimes lose information and not be injective. Specifically, observe that only when $w_1 = 1$ and it is *not* the case that $(\mathbf{f}'_i - \mathbf{e}_i \bmod (-(B + \overline{B}), (B + \overline{B}])) \notin (-B, B]^v$, we lose information. So, we simply treat these inputs differently, by slightly artificially enlarging the output range of the function.

Formally, for every $i \in [n - r]$ the function τ computes

- If $w_i = 0$ or $(\mathbf{f}'_i - \mathbf{e}_i \bmod (-(B + \overline{B}), (B + \overline{B}])) \notin (-B, B]^v$ then

$$\begin{aligned} (w_i \in \{0, 1\}, \mathbf{t}'_i \in \mathbb{Z}_q^u, \mathbf{f}'_i \in (-B, B]^v) \\ \mapsto (\mathbf{t}''_i = \mathbf{t}'_i, \mathbf{f}''_i = (\mathbf{f}'_i - w_i \cdot \mathbf{e}_i \bmod (-B, B]) + w_i \cdot \mathbf{e}_i \bmod (-(B + \overline{B}), (B + \overline{B}])) . \end{aligned}$$

- Else, let $\mathbf{shift} \in (-\tilde{B}, \tilde{B}]^v$ the vector such that each of its coordinates has the same value $2 \cdot (B + \overline{B})$. Then, compute

$$\begin{aligned} (w_i \in \{0, 1\}, \mathbf{t}'_i \in \mathbb{Z}_q^u, \mathbf{f}'_i \in (-B, B]^v) \\ \mapsto (\mathbf{t}''_i = \mathbf{t}'_i, \mathbf{f}''_i = \mathbf{f}'_i + \mathbf{shift} \bmod (-\tilde{B}, \tilde{B}]) . \end{aligned}$$

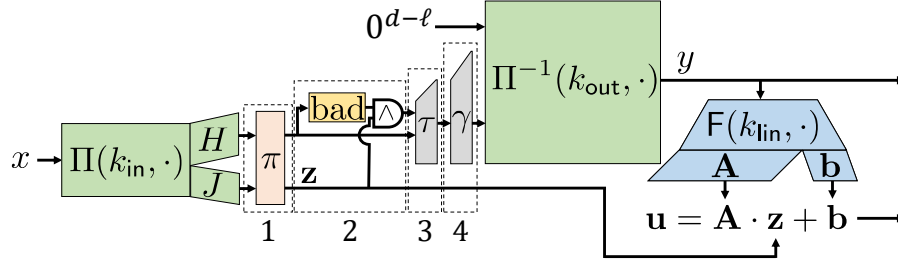


Figure 10: The obfuscated program P in Hybrid 3. We insert the program C instead of the original circuit for Q , which was used in Hybrid 2. There are 4 steps to the computation of C : 1 is the permutation π , 2 is the correction signal, 3 and 4 are the injective functions τ and γ , respectively. The 4 steps of the computation of C , presented in the picture, are formally described in Hybrid 3.

One can verify that τ is indeed injective and efficiently invertible: If $\mathbf{f}_i'' \in (-B, B]^v$ we know $w_i = 0$ and $\mathbf{f}_i' = \mathbf{f}_i''$. Otherwise if $\mathbf{f}_i'' \in (-(B + \bar{B}), (B + \bar{B})]^v$, we know that $w_i = 1$ and $\mathbf{f}_i' = \mathbf{f}_i'' - \mathbf{e}_i \bmod (-B, B]^v$. Otherwise, if $\mathbf{f}_i'' \in (-\tilde{B}, \tilde{B}]^v$, we know that $w_i = 1$ and $\mathbf{f}_i' = \mathbf{f}_i'' - \mathbf{shift} \bmod (-B, B]^v$.

One can verify that in terms of correctness, at the end of this step in C , we have

$$\mathbf{t}_i'' = \mathbf{t}_i + b_i \cdot \mathbf{s}_i \bmod q, \quad \mathbf{f}_i'' = \mathbf{f}_i + b_i \cdot \mathbf{e}_i \bmod q.$$

4. **Injective function γ computing the final output.** The last step of C injectively maps $(n-r) \cdot (u \cdot \log(q) + v \cdot \log(2 \cdot \tilde{B}))$ bits to $(n-r) \cdot v \cdot \log(q)$. The input is $(\mathbf{w}_1'', \dots, \mathbf{w}_{n-r}'')$ and for each $i \in [n-r]$ we have $\mathbf{w}_i'' := (\mathbf{t}_i'' \in \mathbb{Z}_q^u, \mathbf{f}_i'' \in (-\tilde{B}, \tilde{B}]^v)$ and the output is

$$\begin{aligned} & \mathbf{B}_i \cdot \mathbf{t}_i'' + \mathbf{f}_i'' \bmod q \\ &= \mathbf{B}_i \cdot \mathbf{t}_i + \mathbf{f}_i + b_i \cdot (\mathbf{B}_i \cdot \mathbf{s}_i + \mathbf{e}_i) \bmod q \\ &:= \mathbf{B}_i \cdot \mathbf{t}_i + \mathbf{f}_i + b_i \cdot \mathbf{c}_i \bmod q \\ &:= L((\mathbf{B}_i, \mathbf{c}_i), (\mathbf{t}_i, \mathbf{f}_i, b_i)) \end{aligned}$$

which implies the correct computation of the original circuit Q .

The circuit C replaces Q in a specific way: We connect the output $\mathbf{w}$ of the permutation $\Pi(k_{\text{in}}, \cdot)$ into C (all n bits, including $\mathbf{z}$), then the output $\mathbf{a} \in \mathbb{Z}_q^v$ is padded with zeros and goes into $\Pi^{-1}(k_{\text{out}}, \cdot)$ to get the value $y \in \{0, 1\}^d$ as before, and the value $\mathbf{z}$, which was not changed during the computation C , goes as usual to be a coordinate vector for the coset $(\mathbf{A}_y, \mathbf{b}_y)$. The circuit P in the current hybrid is described in Figure 10. One can verify that when C is inserted to replace Q as described, the functionality of P , P^{-1} does not change. It follows that the change of the obfuscated circuits $\mathcal{P}$, $\mathcal{P}^{-1}$ is indistinguishable by the security of the iO. It follows in particular that the success probability of the current process is $:= \epsilon_3$ such that $\epsilon_B \geq \epsilon_3 - \text{negl}(\lambda)$.

- **Hyb₄:** Eliminating layers 1, 3 and 4 from the circuit C (and keeping only layer 2), by using permutable PRPs and the security of iO.

In this hybrid we will execute only layer 2 of C instead of the entire circuit C (recall that layer 2 computes the correction bits). To indistinguishably remove layers 1, 3, 4 from C we argue the following:

- Both permutations $\Pi(k_{\text{in}}, \cdot)$, $\Pi(k_{\text{out}}, \cdot)$ are output-permutable, which means that $\Pi^{-1}(k_{\text{out}}, \cdot)$ is input-permutable. One can verify that it thus only remains to explain why the permutation π (from layer 1 of C) is composable with $\Pi(k_{\text{in}}, \cdot)$ and why the injective functions τ, γ (from layers 3, 4 in C , respectively) are composable with $\Pi^{-1}(k_{\text{out}}, \cdot)$. This will invoke Lemma 42 which uses both the fact that permutations (or permutations that are extensions of injective functions that use ancillas of zeros, as explained in Section 5) are decomposable and the fact that the entire circuit is obfuscated under iO.
- π can be described as a controlled permutation, that under the controlled value (i.e., the value of $\mathbf{z} \in \mathbb{Z}_2^{n-r}$) applies a (modular) addition of a vector. Since a modular addition of a vector is decomposable and π is a controlled version of a decomposable permutation it is overall decomposable (all of these arguments are explained at the beginning of Section 5). Both functions τ and γ are injective as explained as part of the description of C . Also, the input to $\Pi^{-1}(k_{\text{out}}, \cdot)$ has size d and is padded with zeros. Due to our parameter choices for d , one can verify that d is at least as large as the sum of sizes for the input and output for both functions τ and γ . This means again that these functions are composable with the permutation.

The above means that we can indeed move to obfuscating only the computation of layer 2 instead of the entirety of C . It follows in particular that the success probability of the current process is $:= \epsilon_4$ such that $\epsilon_B \geq \epsilon_4 - \text{negl}(\lambda)$.

- **Hyb₅**: Eliminating the remaining layer 2 from the circuit C , by using fixed permuted sparse triggers and the security of iO.

In this hybrid we will eliminate the final layer 2 of the circuit C . Note that once we do this, the obfuscated programs that are given to $\mathcal{A}$ are identical to the ones given to it in the dual-free setting. To indistinguishably remove the final part of C let us observe why it matches the setting of Lemma 45, which is also described in Figure 7. Referring to the notations in Lemma 45, in our setting we do not have the circuits P_0 and P_2 and also there is no wire w_1 . Recall that in previous hybrids, in our circuit we denoted by $(\mathbf{w}_1, \dots, \mathbf{w}_{n-r})$ the first r output bits of $\Pi(k_{\text{in}}, \cdot)$ and by $\mathbf{z} \in \mathbb{Z}_2^{n-r}$ the remaining bits. Going back to matching to the setting of Lemma 45: The role of x_2 is taken by $(\mathbf{w}_1, \dots, \mathbf{w}_{n-r})$, and the role of w_2 is taken by $\mathbf{z}$. Now we will consider two cases for the middle program P_1 (which will accordingly represent the two hybrids **Hyb₄** and **Hyb₅**).

In one instance, the program in the middle is P'_1 and given $w_2 := \mathbf{z}$ and $x_2 := (\mathbf{w}_1, \dots, \mathbf{w}_{n-r})$ uses the trigger program R on x_2 , which acts as follows: For every $i \in [n-r]$, if $\mathbf{w}_i$ contains a "bad" $\mathbf{f}_i$, then R is triggered, outputs 1 and then bit i of w_3 is set to be the i -th bit of w_2 . w_4 is always set to be $w_2 := \mathbf{z}$ regardless. Note that this exactly describes our process in **Hyb₅**, as w_4 continues to be a coordinates vector for the coset $(\mathbf{A}_y, \mathbf{b}_y)$ and w_3 continues as the signal for correction (which, in the original circuit C , was used in the function τ).

In a different instance, the program in the middle is P_1 and given $w_2 := \mathbf{z}$ always outputs $w_3 = 0^{n-r}$ (and similarly to the previous process, w_4 is always set to be $w_2 := \mathbf{z}$). Note that this exactly describes our **Hyb₄**. The program P^{-1} is changed analogously, just like the program Q in Lemma 45 is changed to Q' .

The above means that we can indeed move to obfuscating the original circuits P, P^{-1} from the dual-free setting of Construction 54, and indistinguishably remove the remaining layer 2 of C . It follows in particular that the success probability of the current process is $:= \epsilon_5$ such that $\epsilon_B \geq \epsilon_5 - \text{negl}(\lambda)$. To conclude the proof, note that we assume that $\mathcal{A}$ succeeds in finding a collision with a non-negligible probability, i.e., ϵ_5 is non-negligible, which makes ϵ_B non-negligible, which in turn implies that the quantum polynomial time algorithm $\mathcal{B}$ breaks the collision resistance of Q (and thus search LWE), in contradiction.

□

References

- [Aar03] S. Aaronson. Quantum certificate complexity. In *18th IEEE Annual Conference on Computational Complexity, 2003. Proceedings.*, pages 171–178, 2003.
- [Aar09] Scott Aaronson. Quantum copy-protection and quantum money. In *Proceedings of the 2009 24th Annual IEEE Conference on Computational Complexity, CCC '09*, page 229–242, USA, 2009. IEEE Computer Society.
- [AC12] Scott Aaronson and Paul Christiano. Quantum money from hidden subspaces. In Howard J. Karloff and Toniann Pitassi, editors, *44th Annual ACM Symposium on Theory of Computing*, pages 41–60, New York, NY, USA, May 19–22, 2012. ACM Press.
- [AGKZ20] Ryan Amos, Marios Georgiou, Aggelos Kiayias, and Mark Zhandry. One-shot signatures and applications to hybrid quantum/classical authentication. In Konstantin Makarychev, Yury Makarychev, Madhur Tulsiani, Gautam Kamath, and Julia Chuzhoy, editors, *52nd Annual ACM Symposium on Theory of Computing*, pages 255–268, Chicago, IL, USA, June 22–26, 2020. ACM Press.
- [AGQY22] Prabhanjan Ananth, Aditya Gulati, Luowen Qian, and Henry Yuen. Pseudorandom (function-like) quantum state generators: New definitions and applications. In *Theory of Cryptography Conference*, pages 237–265. Springer, 2022.
- [AMR22] Navid Alamati, Giulio Malavolta, and Ahmadsreza Rahimi. Candidate trapdoor claw-free functions from group actions with applications to quantum protocols. In *Theory of Cryptography Conference*, pages 266–293. Springer, 2022.
- [ARU14] Andris Ambainis, Ansis Rosmanis, and Dominique Unruh. Quantum attacks on classical proof systems: The hardness of quantum rewinding. In *55th Annual Symposium on Foundations of Computer Science*, pages 474–483, Philadelphia, PA, USA, October 18–21, 2014. IEEE Computer Society Press.
- [AS04] Scott Aaronson and Yaoyun Shi. Quantum lower bounds for the collision and the element distinctness problems. *J. ACM*, 51(4):595–605, July 2004.
- [AS15] Gilad Asharov and Gil Segev. Limits on the power of indistinguishability obfuscation and functional encryption. In Venkatesan Guruswami, editor, *56th Annual Symposium on Foundations of Computer Science*, pages 191–209, Berkeley, CA, USA, October 17–20, 2015. IEEE Computer Society Press.

- [AS16] Gilad Asharov and Gil Segev. On constructing one-way permutations from indistinguishability obfuscation. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A: 13th Theory of Cryptography Conference, Part II*, volume 9563 of *Lecture Notes in Computer Science*, pages 512–541, Tel Aviv, Israel, January 10–13, 2016. Springer Berlin Heidelberg, Germany.
- [Bac90] Eric Bach. Explicit bounds for primality testing and related problems. *Mathematics of Computation*, 55:355–380, 1990.
- [Bar23] James Bartusek. Personal communication, and also announced at the ntt research quantum money workshop, 2023.
- [BCM⁺18] Zvika Brakerski, Paul Christiano, Urmila Mahadev, Umesh V. Vazirani, and Thomas Vidick. A cryptographic test of quantumness and certifiable randomness from a single quantum device. In Mikkel Thorup, editor, *59th Annual Symposium on Foundations of Computer Science*, pages 320–331, Paris, France, October 7–9, 2018. IEEE Computer Society Press.
- [BGI⁺01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. In Joe Kilian, editor, *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 1–18, Santa Barbara, CA, USA, August 19–23, 2001. Springer Berlin Heidelberg, Germany.
- [BGI14] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. Functional signatures and pseudorandom functions. In Hugo Krawczyk, editor, *PKC 2014: 17th International Conference on Theory and Practice of Public Key Cryptography*, volume 8383 of *Lecture Notes in Computer Science*, pages 501–519, Buenos Aires, Argentina, March 26–28, 2014. Springer Berlin Heidelberg, Germany.
- [BGK⁺24] James Bartusek, Vipul Goyal, Dakshita Khurana, Giulio Malavolta, Justin Raizes, and Bhaskar Roberts. Software with certified deletion. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part IV*, volume 14654 of *Lecture Notes in Computer Science*, pages 85–111, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
- [BK23] James Bartusek and Dakshita Khurana. Cryptography with certified deletion. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 192–223, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
- [BKP23] James Bartusek, Dakshita Khurana, and Alexander Poremba. Publicly-verifiable deletion via target-collapsing functions. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 99–128, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.

- [BKW17] Dan Boneh, Sam Kim, and David J. Wu. Constrained keys for invertible pseudorandom functions. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017: 15th Theory of Cryptography Conference, Part I*, volume 10677 of *Lecture Notes in Computer Science*, pages 237–263, Baltimore, MD, USA, November 12–15, 2017. Springer, Cham, Switzerland.
- [BLVW19] Zvika Brakerski, Vadim Lyubashevsky, Vinod Vaikuntanathan, and Daniel Wichs. Worst-case hardness for LPN and cryptographic hashing via code smoothing. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part III*, volume 11478 of *Lecture Notes in Computer Science*, pages 619–635, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland.
- [BPW16] Nir Bitansky, Omer Paneth, and Daniel Wichs. Perfect structure on the edge of chaos - trapdoor permutations from indistinguishability obfuscation. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A: 13th Theory of Cryptography Conference, Part I*, volume 9562 of *Lecture Notes in Computer Science*, pages 474–502, Tel Aviv, Israel, January 10–13, 2016. Springer Berlin Heidelberg, Germany.
- [BW13] Dan Boneh and Brent Waters. Constrained pseudorandom functions and their applications. In Kazue Sako and Palash Sarkar, editors, *Advances in Cryptology – ASIACRYPT 2013, Part II*, volume 8270 of *Lecture Notes in Computer Science*, pages 280–300, Bangalore, India, December 1–5, 2013. Springer Berlin Heidelberg, Germany.
- [CL18] Ran Canetti and Amit Lichtenberg. Certifying trapdoor permutations, revisited. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018: 16th Theory of Cryptography Conference, Part I*, volume 11239 of *Lecture Notes in Computer Science*, pages 476–506, Panaji, India, November 11–14, 2018. Springer, Cham, Switzerland.
- [CLG09] Denis X. Charles, Kristin E. Lauter, and Eyal Z. Goren. Cryptographic hash functions from expander graphs. *Journal of Cryptology*, 22(1):93–113, 2009.
- [CLTV15] Ran Canetti, Huijia Lin, Stefano Tessaro, and Vinod Vaikuntanathan. Obfuscation of probabilistic circuits and applications. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015: 12th Theory of Cryptography Conference, Part II*, volume 9015 of *Lecture Notes in Computer Science*, pages 468–497, Warsaw, Poland, March 23–25, 2015. Springer Berlin Heidelberg, Germany.
- [CS20] Andrea Coladangelo and Or Sattath. A quantum money solution to the blockchain scalability problem. *Quantum*, 4:297, 2020.
- [DH76] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
- [Dra23] Justin Drake. One-shot signatures. talk given at the programmable cryptography conference progcrypto, 2023. https://www.youtube.com/watch?v=VmqkH3NPG_s.
- [DS23] Marcel Dall’Agnol and Nicholas Spooner. On the Necessity of Collapsing for Post-Quantum and Quantum Commitments. In Omar Fawzi and Michael Walter, editors, *18th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2023)*, volume 266 of *Leibniz International Proceedings in Informatics*

- (*LIPICs*), pages 2:1–2:23, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [FGH⁺12] Edward Farhi, David Gosset, Avinatan Hassidim, Andrew Lutomirski, and Peter W. Shor. Quantum money from knots. In Shafi Goldwasser, editor, *ITCS 2012: 3rd Innovations in Theoretical Computer Science*, pages 276–289, Cambridge, MA, USA, January 8–10, 2012. Association for Computing Machinery.
 - [GGH⁺13] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. In *54th Annual Symposium on Foundations of Computer Science*, pages 40–49, Berkeley, CA, USA, October 26–29, 2013. IEEE Computer Society Press.
 - [GP07] Louis Granboulan and Thomas Pornin. Perfect block ciphers with small blocks. In Alex Biryukov, editor, *Fast Software Encryption – FSE 2007*, volume 4593 of *Lecture Notes in Computer Science*, pages 452–465, Luxembourg, Luxembourg, March 26–28, 2007. Springer Berlin Heidelberg, Germany.
 - [GPSZ17] Sanjam Garg, Omkant Pandey, Akshayaram Srinivasan, and Mark Zhandry. Breaking the sub-exponential barrier in obfustopia. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part III*, volume 10212 of *Lecture Notes in Computer Science*, pages 156–181, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland.
 - [GR13] Oded Goldreich and Ron D. Rothblum. Enhancements of trapdoor permutations. *Journal of Cryptology*, 26(3):484–512, 2013.
 - [HMR12] Viet Tung Hoang, Ben Morris, and Phillip Rogaway. An enciphering scheme based on a card shuffle. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 1–13, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany.
 - [HPPY24] Alexander Hoover, Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. Plinko: Single-server PIR with efficient updates via invertible PRFs. Cryptology ePrint Archive, Report 2024/318, 2024.
 - [KNY23] Fuyuki Kitagawa, Ryo Nishimaki, and Takashi Yamakawa. Publicly verifiable deletion from minimal assumptions. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023: 21st Theory of Cryptography Conference, Part IV*, volume 14372 of *Lecture Notes in Computer Science*, pages 228–245, Taipei, Taiwan, November 29 – December 2, 2023. Springer, Cham, Switzerland.
 - [KPTZ13] Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. Delegatable pseudorandom functions and applications. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013: 20th Conference on Computer and Communications Security*, pages 669–684, Berlin, Germany, November 4–8, 2013. ACM Press.
 - [KSS22] Daniel M. Kane, Shahed Sharif, and Alice Silverberg. Quantum money from quaternion algebras. *Mathematical Cryptology*, 2(1):60–83, Oct. 2022.

- [LAF⁺09] Andrew Lutomirski, Scott Aaronson, Edward Farhi, David Gosset, Avinatan Hassidim, Jonathan Kelner, and Peter Shor. Breaking and making quantum money: toward a new quantum cryptographic protocol, 2009.
- [LLQ22] Jiahui Liu, Qipeng Liu, and Luowen Qian. Beating classical impossibility of position verification. In Mark Braverman, editor, *ITCS 2022: 13th Innovations in Theoretical Computer Science Conference*, volume 215, pages 100:1–100:11, Berkeley, CA, USA, January 31 – February 3, 2022. Leibniz International Proceedings in Informatics (LIPIcs).
- [LMZ23] Jiahui Liu, Hart Montgomery, and Mark Zhandry. Another round of breaking and making quantum money: How to not build it from lattices, and more. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part I*, volume 14004 of *Lecture Notes in Computer Science*, pages 611–638, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
- [Mor05] Ben Morris. The mixing time of the Thorp shuffle. In Harold N. Gabow and Ronald Fagin, editors, *37th Annual ACM Symposium on Theory of Computing*, pages 403–412, Baltimore, MA, USA, May 22–24, 2005. ACM Press.
- [MR14] Ben Morris and Phillip Rogaway. Sometimes-recurse shuffle - almost-random permutations in logarithmic expected time. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 311–326, Copenhagen, Denmark, May 11–15, 2014. Springer Berlin Heidelberg, Germany.
- [MY23] Tomoyuki Morimae and Takashi Yamakawa. Proofs of quantumness from trapdoor permutations. In Yael Tauman Kalai, editor, *ITCS 2023: 14th Innovations in Theoretical Computer Science Conference*, volume 251, pages 87:1–87:14, Cambridge, MA, USA, January 10–13, 2023. Leibniz International Proceedings in Informatics (LIPIcs).
- [MZRA22] Yiping Ma, Ke Zhong, Tal Rabin, and Sebastian Angel. Incremental Offline/Online PIR. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 1741–1758, Boston, MA, August 2022. USENIX Association.
- [NTT23] NTT Research Quantum Money Workshop, 2023.
- [Pat01] Jacques Patarin. Generic attacks on Feistel schemes. In Colin Boyd, editor, *Advances in Cryptology – ASIACRYPT 2001*, volume 2248 of *Lecture Notes in Computer Science*, pages 222–238, Gold Coast, Australia, December 9–13, 2001. Springer Berlin Heidelberg, Germany.
- [QSi24] QSig Workshop, 2024. <https://informatics.ed.ac.uk/blockchain/events/previous-events/qsig>.
- [RY13] Thomas Ristenpart and Scott Yilek. The mix-and-cut shuffle: Small-domain encryption secure against N queries. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 392–409, Santa Barbara, CA, USA, August 18–22, 2013. Springer Berlin Heidelberg, Germany.

- [SACM21] Elaine Shi, Waqar Aqeel, Balakrishnan Chandrasekaran, and Bruce M. Maggs. Puncturable pseudorandom sets and private information retrieval with near-optimal online bandwidth and time. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part IV*, volume 12828 of *Lecture Notes in Computer Science*, pages 641–669, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.
- [Sat22] Or Sattath. Quantum prudent contracts with applications to bitcoin, 2022.
- [Shm22a] Omri Shmueli. Public-key quantum money with a classical bank. In Stefano Leonardi and Anupam Gupta, editors, *54th Annual ACM Symposium on Theory of Computing*, pages 790–803, Rome, Italy, June 20–24, 2022. ACM Press.
- [Shm22b] Omri Shmueli. Semi-quantum tokenized signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part I*, volume 13507 of *Lecture Notes in Computer Science*, pages 296–319, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [Sho94] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th Annual Symposium on Foundations of Computer Science*, pages 124–134, Santa Fe, NM, USA, November 20–22, 1994. IEEE Computer Society Press.
- [SW14] Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In David B. Shmoys, editor, *46th Annual ACM Symposium on Theory of Computing*, pages 475–484, New York, NY, USA, May 31 – June 3, 2014. ACM Press.
- [Unr16a] Dominique Unruh. Collapse-binding quantum commitments without random oracles. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology – ASIACRYPT 2016, Part II*, volume 10032 of *Lecture Notes in Computer Science*, pages 166–195, Hanoi, Vietnam, December 4–8, 2016. Springer Berlin Heidelberg, Germany.
- [Unr16b] Dominique Unruh. Computationally binding quantum commitments. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 497–527, Vienna, Austria, May 8–12, 2016. Springer Berlin Heidelberg, Germany.
- [Wie83] Stephen Wiesner. Conjugate coding. *SIGACT News*, 15(1):78–88, January 1983.
- [Yao82] Andrew Chi-Chih Yao. Theory and applications of trapdoor functions (extended abstract). In *23rd Annual Symposium on Foundations of Computer Science*, pages 80–91, Chicago, Illinois, November 3–5, 1982. IEEE Computer Society Press.
- [YZW⁺19] Yu Yu, Jiang Zhang, Jian Weng, Chun Guo, and Xiangxue Li. Collision resistant hashing from sub-exponential learning parity with noise. In Steven D. Galbraith and Shihō Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019, Part II*, volume 11922 of *Lecture Notes in Computer Science*, pages 3–24, Kobe, Japan, December 8–12, 2019. Springer, Cham, Switzerland.
- [Zha15] Mark Zhandry. A note on the quantum collision and set equality problems. *Quantum Info. Comput.*, 15(7–8):557–567, May 2015.

- [Zha16] Mark Zhandry. A note on quantum-secure prps. *arXiv preprint arXiv:1611.05564*, 2016.
- [Zha19] Mark Zhandry. Quantum lightning never strikes the same state twice. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part III*, volume 11478 of *Lecture Notes in Computer Science*, pages 408–438, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland.
- [Zha21] Mark Zhandry. How to construct quantum random functions. *Journal of the ACM (JACM)*, 68(5):1–43, 2021.
- [Zha22] Mark Zhandry. New constructions of collapsing hashes. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part III*, volume 13509 of *Lecture Notes in Computer Science*, pages 596–624, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- [Zha24] Mark Zhandry. Quantum money from abelian group actions. In Venkatesan Guruswami, editor, *ITCS 2024: 15th Innovations in Theoretical Computer Science Conference*, volume 287, pages 101:1–101:23, Berkeley, CA, USA, January 30 – February 2, 2024. Leibniz International Proceedings in Informatics (LIPIcs).